

Escuela Técnica Superior de Ingenieria Informática
Grado en Ingeniería Informática. Ingeniería de Computadores

Trabajo Fin de Grado

Sistema de Computación Paralelo Low-cost: un estudio experimental de viabilidad

Realizado por:
Marina Delgado Trinidade

Dirigido por:
D. Fernando Sancho Caparrini

Departamento:
Ciencias de la Computación e Inteligencia Artificial (CCIA)

Sevilla, Octubre 2026

Resumen

Esta memoria se centra en el estudio experimental de la viabilidad de aprovechar múltiples GPUs heterogéneas —distintas generaciones, arquitecturas y fabricantes— dentro de un mismo sistema, siendo lo óptimo poder utilizar todas ellas sin necesidad de descartar ninguna. El objetivo general del trabajo es determinar hasta qué punto es posible construir una máquina de cómputo de tal forma que se consiga integrar tarjetas de características distintas para así obtener un rendimiento conjunto útil. Resulta especialmente interesante emplear hardware de segunda mano e incluso *legacy*, ya que es el hardware que con mayor probabilidad encontraremos en el proceso de búsqueda de dispositivos a los que darle una segunda vida útil.

Se aborda el problema mediante una metodología experimental. En primer lugar se estudian las alternativas que permiten compartir el hardware gráfico entre múltiples entornos —máquinas virtuales con paso directo de dispositivos (VFIO) y contenedores LXC—, analizando las limitaciones técnicas de cada una. El caso de estudio lo forman dos tarjetas NVIDIA de la microarquitectura Tesla (2008–2009), de propósitos distintos: una Tesla C1060, orientada al cómputo, y una GeForce GTS 250 de consumo. Sobre él se documentan en detalle los obstáculos encontrados: la limitación de un único driver propietario por sistema, la instalación de un driver *legacy* de 2019 sobre un kernel y un *toolchain* de 2026 mediante parches comunitarios y DKMS, y la instalación del último toolkit CUDA compatible con estas arquitecturas (CUDA 6.5). A continuación se realiza una batería de pruebas de rendimiento (cómputo y ancho de banda), un experimento de ejecución concurrente sobre ambas GPUs, una prueba de estabilidad bajo carga sostenida y un análisis de las limitaciones del hardware en materia de telemetría energética.

A partir de los obstáculos y resultados obtenidos, se propone un marco teórico de viabilidad formado por cinco filtros —driver, aislamiento, tipo de carga, coste y estabilidad— que permite evaluar *a priori* si un sistema de estas características compensa, y se aplica al caso de estudio.

Abstract

This work focuses on the experimental study of the feasibility of taking advantage of multiple heterogeneous GPUs —different generations, architectures and vendors— within a single system, without having to discard any of them. The general goal is to determine to what extent it is possible to build a computing machine that integrates very different graphics cards and obtains useful aggregate performance, mainly using second-hand or legacy hardware.

The problem is approached through an experimental methodology. First, the alternatives that allow sharing graphics hardware between multiple environments —virtual machines with device passthrough (VFIO) and LXC containers— are studied, analysing the technical limitations of each one. The case study consists of two NVIDIA cards of the Tesla microarchitecture (2008–2009) with different purposes: a Tesla C1060, aimed at computing, and a consumer GeForce GTS 250. The obstacles found are documented in detail: the single proprietary driver limitation per system, the installation of a 2019 legacy driver on a 2026 kernel and toolchain by means of community patches and DKMS, and the installation of the last CUDA toolkit compatible with these architectures (CUDA 6.5). Then, a set of performance benchmarks (compute and bandwidth), a concurrent execution experiment on both GPUs, a sustained-load stability test and an analysis of the hardware limitations regarding energy telemetry are carried out.

Based on the obstacles and results obtained, a theoretical feasibility framework made up of five filters —driver, isolation, workload type, cost and stability— is proposed to assess *a priori* whether a system of this kind is worthwhile, and it is applied to the case study.

Índice general

Introducción **1**

Objetivos . . . 2

Motivación . . . 2

Metodología de trabajo . . . 3

Preliminares **4**

1. Cómputo con GPUs y la plataforma CUDA . . . 4

   1.1. Arquitectura de una GPU NVIDIA . . . 4

   1.2. Las GPUs del caso de estudio . . . 5

   1.3. Anatomía de una GPU de esta generación . . . 6

   1.4. Qué les falta frente a una GPU actual . . . 7

2. El ecosistema de software NVIDIA . . . 9

3. Virtualización de dispositivos: VFIO, IOMMU, ACS y FLR . . . 9

   3.1. IOMMU . . . 10

   3.2. ACS . . . 10

   3.3. Function Level Reset (FLR) . . . 11

4. Máquinas virtuales y contenedores . . . 11

   4.1. Máquinas virtuales con QEMU/KVM y VFIO . . . 11

   4.2. Contenedores LXC . . . 13

5. Proxmox VE . . . 14

1. Marco teórico de viabilidad de sistemas multi-GPU heterogéneos **16**

1.1. Leyes de escalado (Amdahl y Gustafson) . . . 17

1.2. Modelo Roofline . . . 20

1.3. Interconexión PCIe: *lanes*, generaciones y topología consumo vs. servidor . 24

1.4. Modelo económico y sostenibilidad . . . 25

1.5. Filtro de estabilidad: fallo contenido vs. fallo propagado 26

2. Planteamiento del problema y primer acercamiento 29

2.1. La limitación del driver único 29

2.2. Descripción del entorno hardware 30

2.2.1. Carriles PCIe disponibles: cuántas GPUs admite la plataforma . . . 30

2.2.2. Configuración de firmware de la plataforma 32

2.2.3. Arranque dual LXC/VFIO 33

2.3. Primer acercamiento: Ubuntu 18.04 LTS con el driver 340.40 34

3. Aislamiento de GPUs con máquinas virtuales 35

3.1. Planteamiento 35

3.1.1. Ajustes de configuración específicos para VFIO 36

3.2. Evaluación experimental de passthrough mediante VFIO 37

3.2.1. El primer tropiezo: la ASUS ENGTS250 no responde 37

3.2.2. Comprobaciones y ajustes en busca de una solución 37

3.2.3. La prueba de fuego en la máquina virtual y el cuelgue definitivo . . 40

3.2.4. Un fallo que alcanzó al host completo 41

3.3. Resultado y decisión 41

3.4. Análisis comparativo: máquinas virtuales frente a contenedores 42

4. Virtualización ligera con LXC: integración del driver legacy en Proxmox VE 44

4.1. Por qué LXC 44

4.2. Diagnóstico de la incompatibilidad fundamental 45

4.3. Bitácora de resolución de problemas 46

4.3.1. Preparación del host y del entorno de compilación 46

4.3.2. Intentos de compilación manual sin parches 47

4.4. Los parches comunitarios y la solución con DKMS 47

4.4.1. De la compilación manual a DKMS 48

4.5. Verificación y consideraciones sobre el módulo UVM 49

4.6. Discusión 50

5. Contenedor LXC con acceso a GPU y stack CUDA 6.5 **52**

5.1. Las capas del software de cómputo 52

5.2. Acceso a las GPUs desde el contenedor 53

5.3. Instalación del stack CUDA 54

5.3.1. Stack *userspace* del driver 54

5.3.2. *Toolkit* de CUDA 6.5 54

5.4. Validación del cómputo real 55

5.5. Valoración de la fase 56

6. Experimentación y resultados **57**

6.1. Metodología de evaluación 57

6.2. Compilación de los benchmarks 58

6.3. Resultados de rendimiento 60

6.3.1. Cómputo: `matrixMul` 60

6.3.2. Transferencias entre host y GPU 62

6.3.3. Transferencias internas en la GPU 66

6.3.4. Comparativa entre ambas tarjetas 66

6.4. Ejecución concurrente 68

6.5. Condición de carrera en la inicialización 69

6.5.1. Qué se observó 69

6.5.2. Un punto de entrada compartido 69

6.5.3. La pista del registro del kernel 69

6.5.4. Interpretación 70

6.5.5. Por qué el kernel moderno advierte de esta práctica 70

6.5.6. Mitigación 71

6.6. Prueba de estabilidad bajo carga sostenida 71

6.6.1. Planteamiento 71

6.6.2. Resultado: reinicio completo del host 71

6.6.3. Qué estaba haciendo el sistema 72

6.6.4. Interpretación 72

6.6.5. Implicaciones 72

6.7. Consumo eléctrico 73

6.7.1. Ausencia de telemetría 73

6.7.2. Protocolo de medición 73
6.8. Influencia del bus PCIe y rentabilidad 74
6.8.1. Por qué el bus no influyó en las pruebas de cómputo 74
6.8.2. Qué ocurriría con una carga limitada por el bus 75
6.8.3. Cuándo compensa: comparativa con una alternativa moderna 75
6.9. Aplicación del marco de viabilidad al caso de estudio 77
6.10. Discusión de resultados 79

Conclusiones **80**

A. Procedimiento reproducible **89**
A.1. Configuración del firmware 90
A.2. Fases del procedimiento 91
A.3. Material complementario 92

Referencias **93**

Introducción

El cómputo con unidades de procesamiento gráfico (GPUs) se ha consolidado como una de las vías principales para acelerar cargas de trabajo de alto coste computacional, tal y como puede verse en ámbitos tan destacados como el aprendizaje automático o la simulación científica [15]. El modelo habitual de despliegue consiste en adquirir una o varias tarjetas de una misma familia y generación, gestionadas por un *vendor* de software (driver, *runtime*) homogéneo. No obstante, la constante demanda de mejora de rendimiento en los sistemas fomenta recurrir a la compra de hardware cada vez más nuevo, dejándose atrás un parque de dispositivos muy amplio compuesto por GPUs de años, generaciones y fabricantes muy distintos. Ello es consecuencia directa de la rápida obsolescencia de las tarjetas, unida a la tendencia de reciclar equipos entre otros factores.

Surge entonces una pregunta natural: *¿es posible aprovechar todas esas tarjetas a la vez dentro de una misma máquina?* La respuesta intuitiva es que sí, pero la práctica revela que existen numerosas restricciones técnicas —a nivel de driver, de arquitectura y de virtualización— que condicionan e incluso impiden dicha integración. Este trabajo se plantea como un estudio experimental para responder a esa pregunta de forma rigurosa: construiremos un sistema real con varias GPUs de características diferentes y documentaremos, paso a paso, una serie de pruebas explorando qué es posible hacer con ellas y qué no.

El caso de estudio dispone de tres tarjetas NVIDIA de la microarquitectura Tesla (2008–2009): dos **NVIDIA Tesla C1060** (tarjetas de cómputo, *compute capability* 1.3) y una **ASUS ENGTS250** (tarjeta gráfica de consumo, *compute capability* 1.1). La configuración experimental empleó dos GPUs, una Tesla C1060 y la ENGTS250. Ambos modelos están limitados a la última rama de driver NVIDIA denominada *legacy*, versión 340.108, cuyo último *release* oficial data de 2019. Este detalle, aparentemente anecdótico, condiciona todo el trabajo y lo convierte en un excelente banco de pruebas para estudiar la viabilidad de sistemas multi-GPU heterogéneos con limitaciones tecnológicas especialmente remarcables.

Objetivos

El propósito general de este trabajo es doble: estudiar experimentalmente la viabilidad de aprovechar múltiples GPUs heterogéneas en un mismo sistema mediante un caso de estudio real, y sistematizar los criterios de viabilidad empleados en dicho estudio en un marco teórico aplicable *a priori* a otras combinaciones de hardware. La manera de proceder para la construcción del marco teórico ha sido de forma inductiva: se formula y se completa a partir de los propios obstáculos que la experimentación ha ido revelando, en lugar de imponerse como un criterio fijado de antemano. Ambos propósitos se concretan en los siguientes objetivos:

- Diseñar y construir un sistema que integre varias GPUs de generaciones y propósitos distintos, identificando las posibles configuraciones que lo hacen posible.
- Analizar la limitación que supone disponer de un único driver para un sistema multiGPU y evaluar los dos escenarios que derivan de esta condición: emplear un mismo driver para todas las tarjetas (opción no siempre disponible), o aislar cada tarjeta en un entorno propio con su versión de driver y su sistema operativo correspondiente.
- Instalar y validar un driver *legacy* de NVIDIA sobre un sistema con kernel y *toolchain* modernos, resolviendo las incompatibilidades que surjan entre ambos.
- Proporcionar acceso a las GPUs desde un entorno aislado mediante el paso de dispositivos, e instalar el toolkit CUDA compatible con las arquitecturas disponibles.
- Someter a pruebas las tarjetas para medición y comparación de rendimientos (cómputo y ancho de banda de memoria), así como evaluar la contención al ejecutar cargas de trabajo concurrentes y el comportamiento del sistema bajo carga sostenida.
- Documentar los fallos y limitaciones encontrados, tanto de hardware como de software, y plasmarlos en una serie de criterios dentro de un marco teórico de viabilidad generalizable a otros sistemas de características similares.

Motivación

La motivación de este trabajo es doble. En primer lugar, existe una motivación económica y de sostenibilidad: la adquisición de hardware de cómputo nuevo es costosa y el ciclo de vida útil de las tarjetas gráficas es corto, por lo que reutilizar tarjetas de segunda mano o rescatadas de otros equipos resulta atractivo tanto por su bajo coste como por reducir el residuo electrónico. Si se demostrase que es posible combinar tarjetas de distintas

generaciones y marcas en una misma máquina, cualquier usuario podría construir un nodo de cómputo aprovechando hardware que de otro modo se descartaría.

En segundo lugar, el problema tiene un interés técnico genuino. Las GPUs no son componentes fácilmente intercambiables: dependen de un driver propietario, de arquitecturas de cómputo concretas y de un *runtime* (CUDA en el caso de nuestras GPUs NVIDIA). La heterogeneidad introduce además complejidad en el aislamiento, la gestión de memoria y la contención de recursos. Estudiar dónde están los límites de esta heterogeneidad, y cómo pueden salvarse con las distintas herramientas actuales, es un problema abierto que combina conocimientos de sistemas operativos, arquitectura de computadores y virtualización.

Como se comprobará a lo largo de la memoria, el trabajo también se justifica por la gran cantidad de problemas reales —y a menudo no documentados— que surgen al combinar componentes *legacy* con software y hardware modernos. Cada uno de esos problemas constituye un resultado en sí mismo.

Metodología de trabajo

Dado el carácter experimental del trabajo, la metodología seguida es la propia de una bitácora de experimentación estructurada. En cada fase se ha procedido de la siguiente manera:

1. **Planteamiento**: se describe el escenario que se quiere conseguir y se estudian las alternativas técnicas disponibles.
2. **Experimentación**: se implementa el escenario sobre el hardware real y se documentan los obstáculos encontrados, con su causa raíz cuando es posible determinarla.
3. **Validación**: cada fase concluye con una comprobación objetiva del resultado.
4. **Discusión**: se analizan los resultados obtenidos y su relevancia para el objetivo general de viabilidad.

En los casos en los que una vía resulta en descarte, el fracaso se documenta y se utiliza como justificación técnica de la opción alternativa, siguiendo la dinámica de que en un trabajo experimental los resultados negativos son tan informativos como los positivos.

Preliminares

En esta sección se introducen los elementos necesarios para seguir el resto del trabajo: el cómputo con GPUs y la plataforma CUDA, el ecosistema de software NVIDIA, la virtualización de dispositivos, los contenedores LXC, el hipervisor Proxmox VE y la gestión de drivers de kernel en Linux. El lector familiarizado con estos temas puede omitir este capítulo sin pérdida de continuidad.

1. Cómputo con GPUs y la plataforma CUDA

Una unidad de procesamiento gráfico (GPU) es un procesador especializado que integra un gran número de núcleos de ejecución sencillos, capaces de ejecutar de forma simultánea miles de hilos. Esta organización, radicalmente distinta de la de una CPU (pocos núcleos complejos con grandes caches), hace que las GPUs sean especialmente adecuadas para cargas de trabajo masivamente paralelas, como el procesamiento de imágenes o las operaciones con matrices y vectores [24].

Aunque las GPUs nacieron para el renderizado gráfico, pronto se generalizó su uso para cómputo de propósito general (GPGPU). NVIDIA acompañó este cambio con la plataforma **CUDA** (*Compute Unified Device Architecture*), presentada en 2007, que permite programar las GPUs con extensiones del lenguaje C y un *runtime* propio [24].

1.1. Arquitectura de una GPU NVIDIA

El modelo de ejecución de una GPU NVIDIA se organiza jerárquicamente. La GPU se compone de varios *Streaming Multiprocessors* (SM), cada uno de los cuales contiene un conjunto de núcleos CUDA que ejecutan las instrucciones. El programador define *kernels*, funciones que se ejecutan en la GPU, y las lanza sobre una rejilla (*grid*) de bloques de hilos; el planificador de la GPU distribuye esos bloques entre los SM disponibles [24]. Dentro de cada SM, los hilos de un bloque se agrupan en conjuntos de 32 llamados ***warps***: los 32 hilos de un *warp* ejecutan la misma instrucción a la vez, cada uno sobre sus propios datos, y el planificador de *warps* del SM decide en cada momento cuál de ellos avanza.

Para que el mismo código fuente funcione en GPUs distintas, NVIDIA asigna a cada arquitectura una ***compute capability***, un número (p. ej. 1.1, 1.3, 5.0) que describe las capacidades de cómputo de la arquitectura [27, 16]. El compilador de CUDA, `nvcc`, genera código para arquitecturas concretas mediante directivas `-gencode arch=compute_-X,code=sm_X`. Una misma aplicación puede incluir varias versiones de un kernel, una por arquitectura, de modo que el *runtime* seleccione la adecuada en tiempo de ejecución. La evolución de la *compute capability* es la razón última de muchas de las incompatibilidades de software que encontraremos en este trabajo: cada generación añade características (memoria unificada, *ECC*, etc.) que el software más reciente da por supuestas.

1.2. Las GPUs del caso de estudio

Las dos tarjetas utilizadas en este trabajo pertenecen a la microarquitectura **Tesla** de NVIDIA, aunque a chips de distinta generación dentro de ella, lanzados entre 2008 y 2009: la Tesla C1060 utiliza el chip GT200, y la GTS 250, el G92b [27, 16]. A pesar de compartir microarquitectura, son productos de propósitos muy distintos:

	Tesla C1060	**ASUS ENGTS250**
Propósito	Cómputo (Tesla)	Gráfico de consumo (GeForce)
Chip	GT200	G92b
Compute capability	1.3	1.1
Multiprocesadores (SM)	30	16
Núcleos CUDA	240	128
Frecuencia de los núcleos	1296 MHz	1620 MHz
Rendimiento pico teórico	622 GFLOP/s	415 GFLOP/s
Memoria global	4096 MB	1024 MB
Ancho de bus de memoria	512-bit	256-bit
Frecuencia de memoria	800 MHz	1000 MHz
Ancho de banda de memoria teórico	102.4 GB/s	64.0 GB/s
Soporte UVA (Unified Addressing)	No	No

Cuadro 1: Especificaciones de las GPUs del caso de estudio (obtenidas con `deviceQuery` y contrastadas con las fichas de [50, 49])

Como se aprecia en la tabla 1, la Tesla C1060 tiene casi el doble de núcleos que la GTS 250, pero estos funcionan a menor frecuencia, por lo que su rendimiento pico teórico —calculado como núcleos × frecuencia × 2 operaciones por ciclo— es solo 1.5 veces superior. Además, ninguna de las dos soporta memoria unificada, lo que condiciona la configuración del software.

1.3. Anatomía de una GPU de esta generación

Para entender qué cabe esperar de estas tarjetas, conviene ver de qué se componen. La figura 1 lo muestra en tres niveles, de la tarjeta al multiprocesador, tomando como ejemplo la Tesla C1060.

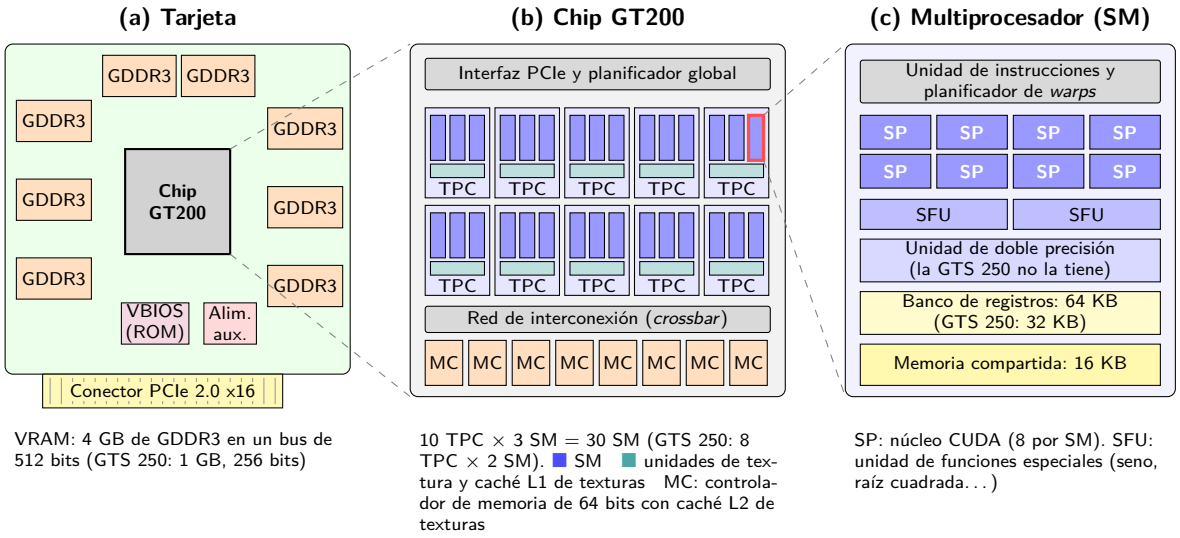


Figura 1: Organización de una GPU de la generación del caso de estudio, con la Tesla C1060 como ejemplo; entre paréntesis, las diferencias de la GTS 250. Elaboración propia a partir de [16, 34].

- **La tarjeta** reúne el chip, la memoria de vídeo (VRAM), la VBIOS y la alimentación auxiliar. La VRAM es la memoria propia de la GPU: los datos deben copiarse a ella a través del bus PCIe antes de poder procesarlos. La VBIOS es el programa, grabado en una memoria ROM de la tarjeta, que el firmware de la placa base ejecuta durante el arranque para inicializarla. En estas tarjetas no incluye GOP, el protocolo con el que las plataformas UEFI inicializan la salida de vídeo.

- **El chip** agrupa los multiprocesadores en TPC (*Texture/Processor Clusters*). Un TPC es un bloque que reúne varios SM junto con las unidades de textura que estos comparten, encargadas de leer y filtrar imágenes almacenadas en memoria. La GT200 tiene diez TPC de tres SM cada uno, y la G92b de la GTS 250, ocho TPC de dos SM. Una red de interconexión (*crossbar*) une los TPC con los controladores de memoria, de 64 bits cada uno: ocho en la GT200, que juntos forman el bus de 512 bits. Junto a cada controlador hay una pequeña caché, reservada a las texturas [16].

- **El multiprocesador (SM)** es la unidad que ejecuta los hilos. Contiene 8 núcleos CUDA (*Streaming Processors*, SP) para operaciones en precisión simple y dos unidades de funciones especiales (SFU), que calculan funciones más complejas y costosas como el seno o la raíz cuadrada. Solo la Tesla C1060 añade una unidad de

doble precisión por SM; la GTS 250 carece de ella, y de ahí su *compute capability* 1.1 frente al 1.3 de la Tesla. Todos los núcleos del SM comparten un banco de registros y 16 KB de memoria compartida, una memoria rápida dentro del chip que los hilos de un mismo bloque usan para cooperar [16, 34].

1.4. Qué les falta frente a una GPU actual

La arquitectura de las GPUs de NVIDIA ha cambiado mucho desde 2008. La figura 2 compara el SM de la GT200 con el de la arquitectura Ada Lovelace (2022), la de la RTX 4060, una GPU actual de gama media [29].

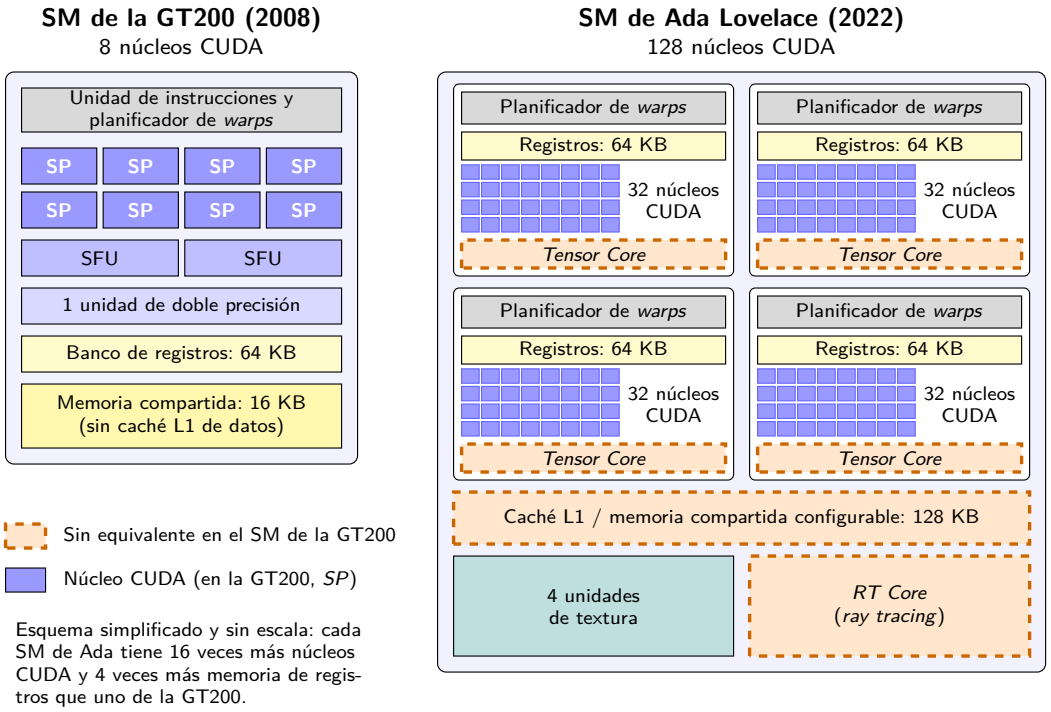


Figura 2: Comparación entre el multiprocesador (SM) de la GT200 y el de la arquitectura Ada Lovelace. Elaboración propia a partir de [16, 29].

Además de tener 16 veces más núcleos CUDA por SM, el SM actual se divide en cuatro particiones con su propio planificador e incorpora tres elementos sin equivalente en la arquitectura GT200:

- **Caché L1 de datos.** La GT200 no tiene cachés de datos de propósito general: salvo las de texturas y constantes, cada acceso a la memoria global llega hasta la VRAM. La generación siguiente, Fermi, introdujo una caché L1 por SM, que comparte espacio con la memoria compartida, y una caché L2 común a todo el chip [34]. En Ada, esa memoria configurable alcanza los 128 KB por SM [29].

- ***Tensor Cores***: unidades especializadas en operaciones de multiplicación y acumulación matricial (*fused multiply-add*), optimizadas para formatos de precisión reducida empleados en el entrenamiento y la inferencia de redes neuronales. La arquitectura Ada incorpora cuatro unidades de cuarta generación por cada SM [29].

- ***RT Cores***: unidades que aceleran el *ray tracing*, una técnica de generación de imágenes que simula la trayectoria de los rayos de luz para calcular de forma realista la iluminación, las sombras y los reflejos de una escena. Se introdujeron con la arquitectura Turing en 2018, y Ada incluye uno por SM [29]. No intervienen en el cómputo de propósito general, pero ilustran la tendencia de las GPUs actuales a incorporar unidades especializadas.

La tabla 2 completa la comparación con las características de la tarjeta y de su ecosistema.

Característica	**Tesla C1060 / GTS 250**	**GPU actual (RTX 4060)**
Núcleos CUDA	240 / 128	3072
Memoria de vídeo	GDDR3: 4 GB / 1 GB	GDDR6: 8 GB
Doble precisión	Solo la Tesla C1060 (una unidad por SM)	Sí
Cachés L1 y L2 de datos	No	Sí (desde Fermi)
Memoria con corrección de errores (ECC)	No	En la gama profesional (desde Fermi)
Ejecución concurrente de varios *kernels*	No	Sí (desde Fermi)
Espacio de direcciones unificado (UVA)	No	Sí (desde Fermi)
Memoria unificada con migración de páginas	No	Sí (desde Pascal)
Tensor Cores y *RT Cores*	No	Sí
GOP en la VBIOS (arranque UEFI)	No	Sí
Telemetría de consumo en `nvidia-smi`	No	Sí
Interfaz	PCIe 2.0 x16	PCIe 4.0
Versión máxima de CUDA	6.5	12.x
Driver	Rama 340.xx, sin actualizaciones desde 2019	Rama actual

Cuadro 2: Características de las GPUs del caso de estudio frente a una GPU actual de gama media. Fuentes: [34, 29, 26, 14, 27].

2. El ecosistema de software NVIDIA

El software de NVIDIA para Linux se estructura en dos niveles con responsabilidades muy distintas [28]:

- **Módulos de kernel**: el driver propiamente dicho (`nvidia.ko` y, en las versiones modernas también, `nvidia-uvm.ko`), compilado contra los *headers* del kernel en ejecución. Es la capa que controla físicamente las tarjetas: gestión de la VRAM, frecuencias y voltajes de los chips, así como la asignación de trabajo a los núcleos CUDA.
- **Stack *userspace***: librerías y herramientas que se ejecutan en espacio de usuario, entre las que destacan `libcuda` (la interfaz del *runtime* CUDA), `nvidia-smi` (herramienta de administración y monitorización [31]) y las librerías de OpenGL/CUDA. A diferencia del módulo de kernel, este stack es un binario precompilado por NVIDIA que no depende de la versión del kernel.

Esta separación es crucial para entender el trabajo: el módulo de kernel del driver es el que sufre las incompatibilidades frente a actualizaciones en el kernel del sistema operativo, mientras que el stack *userspace* es portable entre sistemas. La gestión del módulo se realiza tradicionalmente con **DKMS** (*Dynamic Kernel Module Support*), un framework que automatiza la recompilación de módulos ante cambios de kernel, manteniendo así la supervivencia del sistema ante actualizaciones [11].

UVM (Unified Virtual Memory) es un subsistema del driver NVIDIA que presenta a la CPU y a la GPU una única visión del espacio de direcciones, de modo que RAM y VRAM dejan de ser regiones aisladas: el programador realiza una única reserva de memoria y el mecanismo de *page faulting* por hardware se encarga de mover las páginas entre ambos espacios [14]. La memoria unificada simplifica enormemente la programación CUDA, ya que elimina la necesidad de copias explícitas entre CPU y GPU. Las GPUs del caso de estudio no soportan esta característica (tabla 1), lo que impone las técnicas de gestión de memoria clásicas.

3. Virtualización de dispositivos: VFIO, IOMMU, ACS y FLR

Cuando se plantea integrar en un mismo sistema GPUs heterogéneas que previsiblemente requieren controladores distintos, resulta indispensable abordar el uso de máquinas virtuales con paso directo de dispositivos (GPU passthrough). No obstante, dado que este escenario no se corresponde con el caso de estudio inicial de este trabajo, se introduce también la

tecnología de contenedores de Linux (LXC) como una alternativa de virtualización a nivel de sistema operativo.

3.1. IOMMU

La unidad **IOMMU** (*I/O Memory Management Unit*) es un componente lógico integrado en el procesador y respaldado por el soporte de la BIOS/UEFI y el *chipset*, cuya función principal es traducir las direcciones de memoria virtuales y físicas para los dispositivos de entrada/salida [23]. Mediante este mecanismo, se restringe y aísla el acceso directo a memoria (DMA) de los periféricos, asignando a cada uno un espacio de direcciones exclusivo.

Este aislamiento resulta fundamental en entornos de virtualización. En ausencia de IOMMU, cualquier periférico con capacidades DMA podría leer o escribir en posiciones arbitrarias de la memoria física, comprometiendo la seguridad del anfitrión y de otras instancias virtualizadas. Al habilitar IOMMU, el hipervisor confina las operaciones del hardware en un espacio de direcciones acotado (**sandbox**) que este percibe como la memoria física completa.

A nivel práctico, el sistema operativo organiza los dispositivos en **grupos IOMMU** (*IOMMU groups*), que representan la unidad mínima indivisible de aislamiento de hardware que el kernel de linux puede gestionar y proteger de forma segura. Para que un dispositivo (como una GPU) conforme un grupo IOMMU independiente y pueda asignarse en exclusiva a una máquina virtual, se requiere una conexión directa y dedicada a las líneas PCIe de la CPU respaldada por mecanismos de control de acceso (*Access Control Services*, ACS). Este es precisamente el requisito que la literatura aplicada a la virtualización con paso directo de GPU identifica como condición previa: sin separación en grupos IOMMU independientes, el paso directo de una tarjeta gráfica no resulta viable [45]. Por el contrario, cuando los periféricos carecen de enlace directo y comparten conmutadores PCIe o líneas subordinadas al *chipset* de la placa base sin aislamiento ACS, la IOMMU no puede discriminar sus transferencias DMA de forma individualizada; como consecuencia, dichos dispositivos quedan forzosamente dentro de un mismo grupo IOMMU, debiendo asignarse en bloque a la misma máquina virtual o permanecer en el sistema anfitrión.

3.2. ACS

En las generaciones antiguas de chipsets no existía la granularidad de aislamiento actual: esto se debía, por un lado, a la dependencia de buses compartidos así como a arquitecturas previas a la implementación de ACS, medida extra de seguridad a nivel de hardware.

El estándar PCIe contempla de forma nativa la comunicación directa entre pares (*peer-to-peer*, P2P), permitiendo que dos dispositivos conectados a un mismo conmutador o *chipset*

transfieran datos entre sí sin que el tráfico ascienda a la CPU. Si bien esta característica optimiza la latencia y descarga al bus principal, plantea un compromiso de seguridad en entornos de virtualización: una transferencia DMA directa entre periféricos resulta invisible para la IOMMU, eludiendo sus tablas de traducción y políticas de protección. Ante la imposibilidad de supervisar dicho tráfico, el sistema operativo se veía forzado a vincular preventivamente a todos los dispositivos interconectados dentro de un mismo grupo indivisible. Para cerrar esa brecha se introdujo la especificación **ACS** (*Access Control Services*) en el estándar PCIe [45, 23]. ACS permite la redirección de paquetes (*P2P Request Redirect*) de tal forma que se eleva la comunicación entre dos dispositivos a un nivel en el que la CPU (o la IOMMU) debe validarla antes de que se produzca, cortando de raíz la problemática de las comunicaciones no supervisadas.

La adopción de este mecanismo permite alcanzar la granularidad de aislamiento anteriormente mencionada a nivel de puerto individual, haciendo viable la asignación exclusiva de periféricos a máquinas virtuales independientes. No obstante, este desacoplamiento encuentra su límite en los dispositivos multifunción; en una tarjeta gráfica, por ejemplo, el núcleo de cómputo suele compartir grupo IOMMU con su controladora de audio digital integrada, ya que el propio silicio del punto final (*endpoint*) carece de aislamiento ACS entre sus funciones internas, obligando al hipervisor a transferir el conjunto en bloque.

3.3. Function Level Reset (FLR)

El estándar PCIe define un mecanismo para que el *host* pueda devolver un dispositivo a su configuración por defecto: el reinicio a nivel de función (*Function Level Reset*, FLR), que se activa por *software* escribiendo en un registro del propio dispositivo [3]. Lo característico de FLR es que la operación afecta única y exclusivamente al dispositivo objetivo, sin perturbar al resto de dispositivos del bus. En el contexto de la virtualización, FLR es imprescindible: al apagar o reiniciar una máquina virtual que tiene asignada un dispositivo, es necesario garantizar un punto de partida limpio del mismo para que en el siguiente arranque, la configuración del dispositivo afectado sea conocida. De este modo se garantiza que las siguientes inicializaciones no hereden contextos corruptos en sus registros o memoria interna, previniendo fallos en el *host* o bloqueos críticos en el sistema.

4. Máquinas virtuales y contenedores

4.1. Máquinas virtuales con QEMU/KVM y VFIO

En el ecosistema Linux, la virtualización moderna se sustenta habitualmente en el uso conjunto de dos tecnologías complementarias: **KVM** y **QEMU**.

Por un lado, **KVM** (*Kernel-based Virtual Machine*) es un módulo del *kernel* de Linux que dota a este de capacidades de hipervisor, apoyándose en las extensiones de virtualización por hardware del procesador (como AMD-V o Intel VT-x). La función de KVM como hipervisor es proporcionar la capa de coordinación necesaria que permite al procesador interactuar con el sistema operativo de la máquina virtual de forma prácticamente nativa, sin necesidad de traducción de instrucciones ni de emulación por software para CPU y RAM, lo que reduce la sobrecarga a niveles marginales.

Por otro lado, **QEMU** (*Quick Emulator*) actúa en el espacio de usuario gestionando el ciclo de vida de la máquina virtual y modelando la topología de la plataforma (controladores de bus, temporizadores y periféricos estándar). Si bien QEMU cuenta con la capacidad autónoma de emular arquitecturas completas por software, dicho mecanismo introduce una severa penalización de rendimiento. Al combinarse con KVM, QEMU delega la CPU y la memoria en el procesador físico, asumiendo únicamente la creación de los periféricos virtuales básicos (discos, red o controladores de bus). Sin embargo, cuando se requiere asignar una tarjeta física real (como una GPU) a la máquina virtual, QEMU no la emula: se apoya en el subsistema **VFIO** para conectar ese dispositivo del *host* directamente con el sistema invitado.

VFIO (*Virtual Function I/O*) es el *framework* del núcleo de Linux que expone hacia el espacio de usuario (en este caso, QEMU) el acceso directo y exclusivo a los dispositivos físicos conectados al bus PCIe de forma segura y controlada.

Conviene enfatizar la naturaleza de este acceso exclusivo: en la práctica, VFIO opera como un mecanismo de aislamiento preventivo que impide que el sistema operativo anfitrión reclame o inicialice el dispositivo como lo haría en un arranque convencional sin virtualización. Al vincular el hardware al controlador `vfio-pci`, el *kernel* renuncia a gestionar la tarjeta con sus controladores habituales, reteniéndola en un estado neutro para garantizar que solo el proceso de virtualización tenga potestad sobre sus registros y memoria.

La característica distintiva de VFIO es que su diseño se fundamenta de manera estricta en el concepto de **grupo IOMMU**:

- **Aislamiento mandatorio:** VFIO no permite asociar un único dispositivo a una máquina virtual si este comparte grupo IOMMU con otros periféricos, exigiendo que la totalidad de los componentes del grupo se desvinculen de sus controladores habituales del *host* y pasen a estar bajo el control de VFIO. De este modo, garantiza que ningún dispositivo adyacente pueda realizar accesos no supervisados hacia o desde la memoria de la máquina virtual.

- **El controlador `vfio-pci`:** Para llevar a cabo la cesión, el sistema anfitrión retira el dispositivo de su controlador original (como el controlador gráfico estándar) y lo

enlaza al módulo genérico `vfio-pci`. Este controlador no inicializa la tarjeta para su uso local, sino que actúa como una pasarela que mapea los registros de configuración, las interrupciones (MSI/MSI-X) y los espacios de memoria física (BAR) de la GPU directamente dentro del espacio de direcciones de la máquina virtual.

Gracias a esta arquitectura, la máquina virtual obtiene acceso directo de lectura y escritura sobre el silicio de la tarjeta gráfica mediante operaciones DMA traducidas por la IOMMU, logrando un rendimiento equivalente al de una ejecución nativa.

No obstante, esta aproximación introduce dos compromisos fundamentales:

- **Pérdida de concurrencia del hardware:** El periférico deja de operar como un recurso compartido para convertirse en un componente estrictamente dedicado, quedando inaccesible tanto para el sistema anfitrión como para otras instancias concurrentes.
- **Dependencia de capacidades del silicio:** Para que la alternancia entre arranques sea viable y predecible, el hardware cedido debe implementar mecanismos rigurosos de reinicio (como el estándar FLR analizado previamente) y, de forma preferente en plataformas contemporáneas, firmware compatible con arranque UEFI [3, 4]. De lo contrario, se corre el riesgo de que el dispositivo quede en estados de memoria indeterminados tras el apagado de la máquina invitada.

4.2. Contenedores LXC

Los contenedores **LXC** (*Linux Containers*) constituyen una tecnología de virtualización ligera a nivel de sistema operativo. A diferencia de las máquinas virtuales completas, el contenedor no ejecuta un *kernel* independiente, sino que se sustenta en dos primitivas nativas del *kernel* de Linux para gobernar los procesos: los **espacios de nombres** (*namespaces*) y los **grupos de control** (*cgroups*).

Mientras que los *namespaces* delimitan el ámbito de visibilidad del proceso —proporcionando vistas aisladas del árbol de procesos (`pid`), puntos de montaje (`mnt`), interfaces de red (`net`) e identificadores de usuario (`user`)—, los *cgroups* regulan la asignación y consumo de recursos físicos (cuotas de CPU y memoria principal). Asimismo, el subsistema *cgroup devices* actúa como un mecanismo de filtrado que autoriza o restringe el acceso a nodos de hardware específicos en función de los permisos de los que disponga el grupo asignado en la jerarquía existente.

A diferencia del paradigma de virtualización con VFIO, LXC no desvincula el dispositivo PCIe del anfitrión. El control del *driver* recae íntegra en el *host*, el cual expone hacia el espacio de usuario la interfaz de comunicación con el hardware mediante los denominados

archivos *character devices*. En la arquitectura de Linux, estos nodos especiales no representan ficheros con datos ordinarios en disco, sino puntos de entrada al subsistema de entrada/salida que canalizan flujos continuos de datos e instrucciones directamente hacia el controlador.

Para conceder acceso a estos recursos desde un entorno LXC, es necesario hacer un *bind mount*, técnica que proyecta dicho nodo en el sistema de archivos del contenedor sin duplicados ni emulación. Este mecanismo permite una discriminación selectiva y granular, ya que conociendo qué archivos se quieren montar, se habilita acceso exclusivo al hardware deseado sin afectar al resto.

En consecuencia, esta aproximación prescinde de requisitos como la pertenencia a un grupo IOMMU independiente o la compatibilidad con el mecanismo de reinicio FLR; sin embargo, no establece una frontera física de aislamiento: la contención es estrictamente lógica y a nivel de proceso, no de silicio.

La otra implicación directa de esta arquitectura es la dependencia absoluta del módulo del *kernel* cargado en el anfitrión: el contenedor se ve imposibilitado para ejecutar una versión de *driver* divergente. Por consiguiente, las bibliotecas de cómputo CUDA en espacio de usuario instaladas en el contenedor deben mantener compatibilidad binaria estricta con el controlador del *host*. Finalmente, en lo relativo al modelo de privilegios, un contenedor puede configurarse en modo privilegiado, en el que el usuario `root` del contenedor preserva el identificador `UID 0` del *host*, o en modo no privilegiado, en el que los identificadores se reasignan. El modo privilegiado simplifica el acceso directo a los *character devices*, a costa de un aislamiento de seguridad menor, por lo que resulta adecuado en entornos de pruebas controlados y dedicados en exclusiva a tareas de cómputo.

5. Proxmox VE

Proxmox VE (*Proxmox Virtual Environment*) es una plataforma de virtualización de código abierto basada en Debian que integra, bajo una interfaz unificada de administración, las dos tecnologías descritas previamente: máquinas virtuales basadas en KVM/QEMU y contenedores *LXC* [41].

Su relevancia en este trabajo es estrictamente metodológica: proporciona una infraestructura homogénea para desplegar y contrastar, sobre una misma máquina, dos paradigmas distintos de abstracción y gestión del hardware. Al estar basado en Debian, ofrece un ecosistema predecible para la administración de paquetes y herramientas de depuración a bajo nivel. Su almacenamiento gestionado, sus plantillas de sistemas mínimos y su configuración declarativa reducen además la variabilidad al reconstruir el entorno experimental. Asimismo, permite seleccionar el kernel, utilizar sus *headers* para compilar módulos y fijar el arranque a una versión concreta; esta capacidad es relevante para cualquier trabajo

con módulos del kernel, sin anticipar aquí la versión empleada. Si bien una instalación mínima de Debian sería más ligera, exigiría articular y mantener manualmente la gestión de máquinas virtuales, contenedores, almacenamiento, plantillas y versiones del kernel. Por ello, Proxmox VE se elige como entorno de pruebas por reproducibilidad experimental y consistencia operativa, no como requisito indispensable de la arquitectura.

Capítulo 1

Marco teórico de viabilidad de sistemas multi-GPU heterogéneos

En este capítulo se propone un marco teórico que permite decidir *a priori* si un sistema de reaprovechamiento con GPUs de consumo es viable. El marco consta de **cinco filtros secuenciales**:

$$V = F_{\text{driver}} \wedge F_{\text{aislamiento}} \wedge F_{\text{carga}} \wedge F_{\text{coste}} \wedge F_{\text{estabilidad}} \tag{1.1}$$

Basta con que uno falle para que el despliegue no compense:

- $F_{\textbf{driver}}$ **(Compatibilidad del software base):** Se estudia si el código del controlador propietario puede compilarse y enlazarse en *kernels* contemporáneos (mediante DKMS u otro gestor de módulos de kernel y parches de compatibilidad) sin comprometer la estabilidad del sistema anfitrión, y si todas las GPUs pueden funcionar con una misma rama del controlador.

- $F_{\textbf{aislamiento}}$ **(Virtualización y orquestación):** Si las GPUs requieren versiones de drivers incompatibles entre sí, no pueden convivir en el mismo *kernel* anfitrión y deben aislarse en máquinas virtuales mediante VFIO *passthrough*, lo que exige una segmentación IOMMU limpia y soporte de reinicio (FLR). En caso contrario, no es necesario el aislamiento mediante VFIO y este parámetro no aplica, por lo que se considera favorable.

- $F_{\textbf{carga}}$ **(Intensidad de la carga de trabajo):** Determina si el cuello de botella reside en las limitaciones de las comunicaciones del bus PCIe o si, por el contrario, reside en el tiempo de cálculo. Es decir, si el problema es de tipo *Transfer-Bound* o *Compute-Bound* (sección 1.2).

- $F_{\textbf{coste}}$ **(Viabilidad económica y operativa):** Comprueba si el valor del cómputo obtenido supera el coste energético, las horas de ingeniería invertidas y el riesgo de obsolescencia (sección 1.4).

- $F_{\textbf{estabilidad}}$ **(Resiliencia operativa):** Comprueba si los fallos del hardware quedan contenidos en el propio dispositivo y pueden resolverse sin intervención física (sección 1.5). Dado que los fallos de hardware no son deterministas, pueden haberse superado con éxito los cuatro filtros anteriores y aun así encontrarse situaciones limitantes en la práctica.

Las secciones siguientes desarrollan los fundamentos en los que se apoya la evaluación de cada filtro.

1.1. Leyes de escalado (Amdahl y Gustafson)

Se eligen estas leyes ya que nos permiten analizar el escalado en el rendimiento de un sistema a medida que aumentan los recursos, que en nuestro caso concreto, se traduce en ir añadiendo GPUs.

Ley de Amdahl. La ley de Amdahl es la siguiente:

$$T_{\text{mejorado}} = T_{\text{antes}} \left((1 - f_{\text{m}}) + \frac{f_{\text{m}}}{A_{\text{m}}} \right) \tag{1.2}$$

- T_{antes}: Tiempo total de ejecución base previo al aumento de recursos.
- f_{m}: Fracción del tiempo de cómputo original que se ve directamente beneficiada por la mejora ($0 \leq f_{\text{m}} \leq 1$).
- $1 - f_{\text{m}}$: Fracción inalterada por ser estrictamente secuencial, y que por tanto, no experimenta aceleración alguna.
- A_{m}: Factor de aceleración local (*speedup*) obtenido exclusivamente en la parte mejorada (adimensional; en paralelismo ideal con n GPUs homogéneas, $A_{\text{m}} = n$).

En arquitectura de computadores, la aceleración de un sistema por definición es:

$$S(n) = \frac{T_{\text{antes}}}{T_{\text{mejorado}}} \tag{1.3}$$

Por lo que si sustituimos la expresión de tiempo mejorado de (1.2) en (1.3):

$$S(n) = \frac{T_{\text{antes}}}{T_{\text{mejorado}}} = \frac{T_{\text{antes}}}{T_{\text{antes}} \left((1 - f_{\text{m}}) + \frac{f_{\text{m}}}{A_{\text{m}}} \right)} \tag{1.4}$$

Simplificando se nos queda en:

$$S(n) = \frac{1}{(1 - f_{\mathrm{m}}) + \dfrac{f_{\mathrm{m}}}{A_{\mathrm{m}}}} \tag{1.5}$$

La Ley de Amdahl permite modelar la aceleración teórica máxima (S) al paralelizar una carga de trabajo sobre n recursos [5]. Sin embargo, impone una limitación asintótica insalvable: incluso en el caso ideal en el que el número de recursos tienda a infinito ($n \to \infty$, tomando $A_{\mathrm{m}} = n$), la aceleración queda estrictamente acotada por la fracción no paralelizable:

$$\lim_{n \to \infty} S(n) = \frac{1}{1 - f_{\mathrm{m}}} \tag{1.6}$$

En un sistema con varias GPUs, dicho término secuencial $(1 - f_{\mathrm{m}})$ no comprende únicamente las secciones de código no paralelizables, sino que abarca también los tiempos de sincronización y las latencias físicas de transferencia de datos del bus PCIe. Esta sobrecarga puede situar al sistema en un régimen limitado por transferencia (*transfer-bound*), donde el bus actúa como el cuello de botella crítico frente a la capacidad de cálculo pura de las GPUs.

Para mitigar esta limitación de tanto peso, la estrategia consiste en reducir el tráfico que atraviesa el bus, reutilizando cada byte transferido en el mayor número posible de operaciones. Esta idea se formaliza mediante la intensidad aritmética y el modelo Roofline (sección 1.2).

En sistemas heterogéneos, sin embargo, la igualdad $A_{\mathrm{m}} = n$ deja de cumplirse, ya que no todas las GPUs aportan la misma capacidad de cálculo. Tomando como referencia el rendimiento de la GPU más rápida (P_{ref}) y suponiendo un reparto de la carga proporcional a la capacidad de cada tarjeta, el factor de aceleración local es:

$$A_{\mathrm{m}} = \frac{\sum_{i=1}^{n} P_i}{P_{\mathrm{ref}}} \tag{1.7}$$

donde P_i es el rendimiento individual de cada GPU. Por ejemplo, si un sistema combina dos GPUs y la segunda rinde la mitad que la primera ($P_2 = P_{\mathrm{ref}}/2$), la combinación de ambas tarjetas proporciona $A_{\mathrm{m}} = 1 + 0{,}5 = 1{,}5$ en lugar de 2. Si se añade una tercera GPU igual que la primera, el sistema alcanzaría $A_{\mathrm{m}} = 2{,}5$ en lugar de 3.

Si, por el contrario, la carga se reparte a partes iguales entre las GPUs, la más lenta ($P_{\mathrm{mín}}$) determina el tiempo total:

$$A_{\mathrm{m}} = n \cdot \frac{P_{\mathrm{mín}}}{P_{\mathrm{ref}}} \tag{1.8}$$

lo que en el ejemplo anterior de dos GPUs resulta en $A_{\mathrm{m}} = 2 \cdot 0{,}5 = 1$: la segunda tarjeta no aporta ninguna aceleración, porque la primera termina su mitad del trabajo y queda

esperando a que la lenta termine la suya. Por tanto, en sistemas heterogéneos el criterio de reparto de la carga resulta tan determinante como el número de GPUs que se añaden.

Ley de Gustafson. La Ley de Gustafson matiza la cota impuesta por Amdahl, según la cual el factor de aceleración siempre queda limitado por la fracción secuencial sin importar con cuántos recursos se mejore un sistema. Más específicamente, la formulación de Gustafson complementa la perspectiva de Amdahl adaptando el modelo al escalado débil (*weak scaling*), donde el tamaño del problema crece proporcionalmente con la capacidad en recursos del sistema, manteniendo constante el tiempo de ejecución (T_{mejorado}). En otras palabras, se pone énfasis en que en un sistema con mayores recursos, pueden abordarse problemas de mayor complejidad computacional. En un sistema con varias GPUs conectadas por PCIe, la compensación se obtiene en la medida en que el tiempo dedicado al cálculo crece más deprisa que el invertido en transferencias, justo la situación que se busca para paliar la asimetría del bus PCIe [13].

A diferencia de la formulación de Amdahl, donde la fracción de tiempo mejorada f_{m} se define sobre el tiempo del sistema base (T_{antes}), en la Ley de Gustafson la fracción susceptible de mejora (f'_{m}) se mide directamente en el sistema paralelo:

$$\begin{aligned} S = \frac{T_{\text{antes}}}{T_{\text{mejorado}}} &= (1 - f'_{\text{m}}) + A_{\text{m}} \cdot f'_{\text{m}} \\ &= 1 + (A_{\text{m}} - 1) \cdot f'_{\text{m}} \end{aligned} \tag{1.9}$$

donde:

- S es la aceleración escalada del sistema completo. Dado que en el escalado débil T_{mejorado} se mantiene constante, T_{antes} no es un tiempo medido, sino el tiempo que tardaría el sistema base en resolver el problema ya escalado.
- f'_{m} es la fracción del tiempo de cómputo dedicada a la fase paralela en el sistema mejorado ($0 \leq f'_{\text{m}} \leq 1$).
- $1 - f'_{\text{m}}$ representa la fracción de tiempo consumida por el código secuencial, las sincronizaciones y las transferencias por el bus.
- A_{m} es la capacidad de paralelización del entorno (en sistemas homogéneos de n unidades de cálculo, $A_{\text{m}} = n$).

Esta ley evidencia que, al aumentar la dimensión de los datos a procesar, el término $A_{\text{m}} \cdot f'_{\text{m}}$ domina frente a las sobrecargas del bus ($1 - f'_{\text{m}}$), permitiendo un crecimiento prácticamente lineal de la aceleración con respecto al número de recursos y amortizando la latencia del enlace PCIe. Esta amortización, no obstante, exige que la intensidad aritmética del algoritmo crezca con el tamaño del problema (como ocurre en el producto denso de

matrices, con $O(N^3)$ FLOPs frente a $O(N^2)$ bytes): en algoritmos de intensidad constante o decreciente con la escala, agrandar el problema no reduce la fracción de tiempo dedicada al bus, y la Ley de Gustafson no ofrece ninguna escapatoria. Esta conclusión respalda la hipótesis de trabajo: un sistema de estas características puede resultar viable para problemas de índole *Compute-Bound* cuya intensidad alcanzable supere el punto de cresta (sección 1.2), pero no para el caso de *Transfer-Bound*.

1.2. Modelo Roofline

El modelo Roofline [52] delimita el rendimiento máximo teórico que puede alcanzar una GPU ante una carga de trabajo concreta. Caracteriza cada algoritmo en función de su *Intensidad Aritmética*, la cual se define como:

$$I_{\mathrm{A}} = \frac{\text{FLOPs}}{\text{Bytes transferidos}} \quad [\text{FLOP/Byte}] \tag{1.10}$$

donde los FLOPs son el número de operaciones en punto flotante que realiza el algoritmo. La intensidad aritmética mide, por tanto, cuánto trabajo se hace con cada byte que hay que mover. Es una propiedad del algoritmo: sumar dos vectores, por ejemplo, utiliza cada número una sola vez (una operación por cada dato leído), mientras que multiplicar dos matrices utiliza cada número en cientos o miles de operaciones, tantas como el tamaño de la matriz.

En su formulación clásica, los bytes transferidos son los que se mueven entre la memoria de la GPU (VRAM) y sus núcleos. En este trabajo el modelo se adapta tomando como referencia los bytes que atraviesan el bus PCIe, ya que es el canal por el que llegan los datos desde el anfitrión y el principal cuello de botella de la plataforma. Ambos techos coexisten: el de la VRAM limita la ejecución del *kernel* una vez los datos residen en la GPU, mientras que el del bus PCIe limita el trabajo completo (transferencia de entrada, cálculo y devolución de resultados), que es el que interesa en un sistema con varias GPUs que comparten el bus.

El valor de esta variable marca la frontera entre los dos tipos de problemas: *Transfer-Bound* y *Compute-Bound*.

- ¿I_{A} alta? Problema de naturaleza *Compute-Bound*. FLOPs $\gg$ Bytes, ergo el tiempo de intervención del bus es despreciable frente al tiempo de cálculo.
- ¿I_{A} baja? Problema de naturaleza *Transfer-Bound*. Bytes $\gg$ FLOPs, predomina la comunicación frente al tiempo de cómputo, por lo que el ancho de banda del bus se convierte en el factor crítico de rendimiento.

Basándonos en este modelo, el rendimiento alcanzable se define como:

$$P = \text{mín}\,(P_{\text{pico}},\ I_{\text{A}} \times AB) \quad [\text{FLOP/s}] \tag{1.11}$$

donde:

- P_{pico} (FLOP/s): Techo de cómputo. Potencia de cálculo máxima teórica de la GPU (tabla 1).
- AB (Bytes/s): Ancho de banda del bus PCIe, que depende de la generación y del número de carriles de la ranura que ocupa cada GPU (tabla 1.1), de modo que cada GPU tiene su propio techo de transferencia.
- $I_{\text{A}} \times AB$ (FLOP/s): Techo de transferencia. Marca la tasa máxima de operaciones que la tarjeta puede procesar atendiendo a la velocidad con la que le llegan los datos.

De entre ambos techos, se toma como indicador de rendimiento del sistema el mínimo, ya que el rendimiento queda siempre limitado por la medida más restrictiva. Cabe destacar que esta cota supone un solapamiento perfecto entre transferencia y cálculo, es decir, que la GPU procesa unos datos mientras recibe los siguientes (copias asíncronas mediante *streams*, colas de operaciones que la GPU puede ejecutar en paralelo). Si las transferencias y el cálculo se ejecutan de forma secuencial, los tiempos se suman, caso que recoge el modelo de la sección 1.3. Ambos modelos son, por tanto, complementarios: el Roofline proporciona la cota superior con solapamiento y el modelo de contención, el tiempo sin él.

El punto de cresta: el papel del hardware. Los dos techos se cruzan en la intensidad aritmética del punto de cresta (*ridge point*):

$$I_{\text{A}}^{*} = \frac{P_{\text{pico}}}{AB} \tag{1.12}$$

La carga es *compute-bound* en cuanto $I_{\text{A}} \geq I_{\text{A}}^{*}$, y *transfer-bound* por debajo de ese valor. La clave del modelo está en que cada lado de la comparación depende de una cosa distinta. La intensidad aritmética depende solo del algoritmo: cuántas operaciones hace por cada byte que mueve. El punto de cresta depende solo del hardware: cuántas operaciones puede hacer la GPU por cada byte que le llega. Por eso la frontera no es una propiedad absoluta del algoritmo, sino del par algoritmo–hardware: una misma carga puede ser *compute-bound* en una máquina moderna y *transfer-bound* en una plataforma con enlaces más lentos, como una ranura de pocos carriles conectada a través del *chipset*.

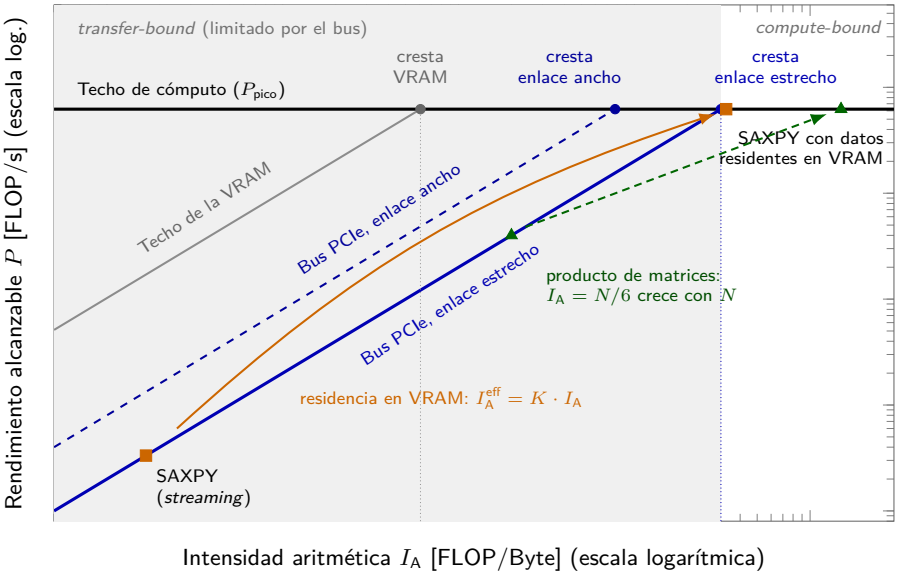


Figura 1.1: Esquema del modelo Roofline, en escala logarítmica. Cada línea inclinada es el techo de transferencia de un canal, y su cruce con el techo de cómputo es el punto de cresta de ese canal. La zona sombreada es la región *transfer-bound* respecto al enlace estrecho. Los puntos muestran cargas de ejemplo, y las flechas, cómo aumenta su intensidad al mantener los datos en la VRAM o al crecer el tamaño del problema. Elaboración propia a partir de [52].

La figura 1.1 representa el modelo de forma esquemática. Como el bus PCIe es mucho más estrecho que el canal entre la VRAM y los núcleos de la GPU, su techo de transferencia queda muy por debajo, y su punto de cresta, mucho más a la derecha: para aprovechar el cómputo cuando los datos llegan por el bus hace falta una intensidad muy superior a la que basta cuando ya están en la VRAM. Además, cuanto más estrecho es el enlace (menos carriles o una generación más antigua), más baja es su rampa y más a la derecha se desplaza la cresta, de modo que una misma carga puede pasar de *compute-bound* a *transfer-bound* solo por cambiar de ranura.

Intensidad aritmética efectiva. La intensidad de (1.10) dice cuántas operaciones hace el algoritmo por cada byte que necesita, pero no cuántas veces hay que enviar esos bytes por el bus. Y eso depende de cómo se organice el programa. Supóngase un cálculo que se repite K veces sobre los mismos datos, como una simulación que actualiza el mismo vector en cada paso. Hay dos formas de llevarlo a la GPU:

- **Enviar los datos en cada repetición** (*streaming*). Cada repetición paga su propia transferencia por el bus.
- **Enviar los datos una sola vez y dejarlos en la VRAM** (residencia). Las K repeticiones se hacen sobre los datos que ya están en la tarjeta, y el bus solo se usa al principio.

En los dos casos la GPU hace exactamente las mismas operaciones: K repeticiones de W FLOPs, es decir, $K \cdot W$ en total. Lo único que cambia es cuántos bytes cruzan el bus: $K \cdot B$ en el primer caso, porque los datos se envían K veces, y solo B en el segundo. Como la intensidad divide las operaciones entre los bytes transferidos, en el primer caso vale $KW/(KB) = W/B$, la del propio algoritmo, mientras que en el segundo vale KW/B: K veces mayor, porque el mismo trabajo se hace con K veces menos tráfico por el bus. Para recoger esta diferencia se define la **intensidad efectiva**, la que realmente "ve" el bus PCIe:

$$I_{\mathrm{A}}^{\mathrm{eff}} = \begin{cases} \dfrac{W}{B} = I_{\mathrm{A}}, & \textit{streaming}\text{: cada byte cruza el bus una vez} \\ \dfrac{K \cdot W}{B} = K \cdot I_{\mathrm{A}}, & \text{residencia: } K \text{ iteraciones sobre datos en VRAM} \end{cases} \tag{1.13}$$

donde W son los FLOPs de cada repetición y B, los bytes que cruzan el bus. La residencia tiene un requisito: el *working set*, el conjunto de datos que el programa necesita durante el cálculo, debe caber en la VRAM de la tarjeta.

El ejemplo de SAXPY de la figura 1.1 lo ilustra. Esta operación ($y \leftarrow ax + y$) hace 2 FLOPs por cada elemento y mueve 12 bytes por el bus (lee x e y y devuelve y, a 4 bytes cada uno), así que $I_{\mathrm{A}} = 2/12 \approx 0{,}17$ FLOP/B. Si se envían los datos en cada repetición, esa es la intensidad que ve el bus, muy por debajo de cualquier punto de cresta: la GPU pasa casi todo el tiempo esperando datos. Si, en cambio, se envían una vez y se repite la operación K veces en la VRAM, la intensidad efectiva se multiplica por K y, con suficientes repeticiones, supera la cresta: la misma operación pasa a estar limitada por el cálculo, como indica la flecha de la figura.

En el producto de matrices $N \times N$ ocurre algo distinto: no hace falta repetir el cálculo, porque la propia intensidad crece con el tamaño. Con $2N^3$ FLOPs sobre $12N^2$ bytes, $I_{\mathrm{A}} = N/6$, de modo que, a partir de cierto tamaño, supera cualquier punto de cresta.

En resumen, que una carga sea paralelizable no basta para aprovechar la GPU: importa cuántas veces cruzan sus datos el bus. La frontera práctica separa las cargas que reutilizan mucho sus datos, o los mantienen en la VRAM, de las que los transmiten de forma continua.

El techo de cada implementación. Para mitigar la limitación del bus, la estrategia consiste en elevar la intensidad efectiva de la implementación: reutilizar cada byte transferido en el mayor número posible de operaciones, mediante el bloqueo o *tiling* (dividir el cálculo en bloques que caben en la memoria rápida y se reutilizan antes de descartarlos), la residencia de los datos en VRAM o el solapamiento de transferencia y cálculo mediante *streams*. Conviene subrayar que esta adaptación no siempre es posible: una implementación concreta de un algoritmo posee un techo de intensidad alcanzable $I_{\mathrm{A}}^{\mathrm{máx}}$, impuesto por su estructura y por la capacidad de reutilización de memoria disponible. La compensación de

la asimetría del bus solo es viable mientras ese techo supere el punto de cresta del sistema:

$$I_{\mathrm{A}}^{\mathrm{máx}} \geq I_{\mathrm{A}}^{*} = \frac{P_{\mathrm{pico}}}{AB} \tag{1.14}$$

Si, por el contrario, $I_{\mathrm{A}}^{\mathrm{máx}} < I_{\mathrm{A}}^{*}$, el problema es intrínsecamente *transfer-bound* para esa GPU y esa ranura: ninguna optimización de software elimina la cota y la única vía de viabilidad pasa por aumentar el ancho de banda del enlace.

El filtro de carga. En el marco, F_{carga} se evalúa comparando la intensidad efectiva alcanzable con el punto de cresta. Con varias GPUs, cada una tiene su propia cresta, y la condición la fija la más exigente del conjunto: la GPU con el enlace más estrecho en relación con su capacidad de cálculo.

La consecuencia práctica es que un mismo hardware puede resultar adecuado para cargas de alta intensidad y no para las de baja, sin cambiar el código.

Este criterio ayuda también a valorar el uso de un sistema así para aprendizaje automático (*Machine Learning*). Las fases que mueven grandes volúmenes de datos entre la CPU y las GPUs, o entre las propias GPUs, como la carga de lotes de datos o la sincronización de gradientes en el entrenamiento distribuido, son las más sensibles al ancho de banda del bus, y en una plataforma de consumo quedarían limitadas por él.

1.3. Interconexión PCIe: *lanes*, generaciones y topología consumo vs. servidor

PCI Express es un bus serie punto a punto. El ancho de banda efectivo depende de dos factores: generación y número de *lanes* [51].

Generación	x1	x4	x8	x16
PCIe 1.0	0.25 GB/s	1.0 GB/s	2.0 GB/s	4.0 GB/s
PCIe 2.0	0.5 GB/s	2.0 GB/s	4.0 GB/s	8.0 GB/s
PCIe 4.0	2.0 GB/s	8.0 GB/s	16 GB/s	32 GB/s
PCIe 5.0	4.0 GB/s	16 GB/s	32 GB/s	64 GB/s

Cuadro 1.1: Ancho de banda teórico por sentido, según la generación y el número de lanes, una vez descontada la codificación de línea (8b/10b en PCIe 1.0 y 2.0; 128b/130b en PCIe 4.0 y 5.0) y sin *overhead* de protocolo. El throughput real es ∼75–80 % del teórico.

Plataformas de consumo y de servidor. El número de carriles disponibles depende de la plataforma. En una plataforma de consumo, el procesador ofrece un número limitado de carriles: normalmente reserva un bloque de dieciséis para la ranura gráfica principal y conecta el resto de ranuras a través del *chipset*, un circuito auxiliar de la placa base que comparte su propio enlace con el procesador entre todos los dispositivos que cuelgan de él

(discos, red, puertos USB). Una segunda GPU en una de esas ranuras dispone, por tanto, de menos carriles y comparte el ancho de banda con el resto del sistema. Las plataformas de servidor, en cambio, ofrecen muchos más carriles directos al procesador, del orden de 64 a 128, de modo que cada GPU puede disponer de su enlace x16 completo sin compartirlo. En servidor, el ancho de banda PCIe es un parámetro de diseño; en consumo, es una restricción compartida.

Gestión de energía del enlace. PCIe permite además que un enlace adopte estados de bajo consumo cuando los dispositivos extremos no los utilizan, mecanismo denominado *Active-State Power Management* (ASPM) y gobernado en el kernel por las políticas `default`, `powersave` y `performance` [44, 21]. El precio de esta gestión es una latencia añadida en cada transición entre estados del enlace, y un riesgo con dispositivos antiguos, que pueden no responder a tiempo al salir de un estado de bajo consumo.

Modelo de contención. Para una carga con volumen B bytes a transferir, el tiempo total se modela como:

$$T_{\text{total}} = T_{\text{cómputo}} + \frac{B}{\text{BW}_{\text{PCIe}}(\text{gen}, \text{lanes}) \cdot \eta \cdot \gamma} \tag{1.15}$$

donde $\eta \approx 0{,}75$ es la eficiencia del protocolo PCIe y $\gamma \geq 1$ representa el factor de contención o congestión derivado del tráfico compartido en el *chipset*. Si $T_{\text{cómputo}} \gg T_{\text{transfer}}$, el número de carriles es irrelevante. Si $T_{\text{transfer}} \gtrsim T_{\text{cómputo}}$, la plataforma de consumo deja de compensar. Para conocer la velocidad y el ancho realmente negociados por cada enlace, puede consultarse el campo `LnkSta` con `lspci -vv`.

1.4. Modelo económico y sostenibilidad

Para discernir si el reaprovechamiento compensa la adquisición de hardware moderno, se formaliza el balance de beneficio neto:

$$\text{Ganancia} = (S_{\text{real}} \cdot V_{\text{hora}} \cdot h) - (C_{\text{kWh}} \cdot P \cdot h + C_{\text{ing}} \cdot h_{\text{setup}} + C_{\text{riesgo}}) \quad [€] \tag{1.16}$$

considerándose el reaprovechamiento económicamente viable cuando Ganancia > 0, donde se cuantifica el valor del cálculo acelerado ($S_{\text{real}} \cdot V_{\text{hora}} \cdot h$) frente al coste eléctrico ($C_{\text{kWh}} \cdot P \cdot h$), el tiempo de ingeniería invertido en la puesta a punto ($C_{\text{ing}} \cdot h_{\text{setup}}$) y la deuda técnica derivada de la dependencia de parches comunitarios no oficiales (C_{riesgo}) [9, 12, 11].

Haciendo un desglose más exhaustivo de los parámetros quedaría:

- S_{real} **(Factor de aceleración empírico):** Mejora de tiempo obtenida por el sistema ante aumento de recursos frente a sistema base.

- $V_{\mathbf{hora}}$ **(Coste de oportunidad temporal):** Valoración económica por unidad de tiempo de cómputo ahorrada, calculada en función de las tarifas horarias de servicios en la nube equivalentes o del coste del personal técnico asignado.
- $C_{\mathbf{kWh}}$ **(Coste unitario de energía):** Precio de la energía eléctrica consumida por el sistema bajo carga de trabajo activa (€/kWh).
- P **(Potencia consumida):** Potencia eléctrica media demandada por el sistema bajo carga de trabajo activa (kW).
- h **(Tiempo de operación acumulado):** Horas totales en las que el sistema realizó cómputo efectivo.
- $C_{\mathbf{ing}}$ **(Coste horario de ingeniería):** Tarifa por hora del personal especializado requerido para la puesta a punto, resolución de incompatibilidades e integración del entorno.
- $h_{\mathbf{setup}}$ **(Esfuerzo de despliegue inicial):** Tiempo total invertido en la adaptación del software base, resolución de dependencias, configuración de la cadena de compilación y habilitación de los mecanismos de aislamiento.
- $C_{\mathbf{riesgo}}$ **(Coste por riesgo operativo y deuda técnica):** Penalización económica que cuantifica la probabilidad de fallo imprevisto, interrupción del servicio o coste asociado a migraciones de urgencia ante la falta de soporte oficial en el ciclo de vida del hardware.

En un sistema de reaprovechamiento, el coste de adquisición del hardware es prácticamente nulo, de modo que el término dominante suele ser h_{setup}: el tiempo necesario para adaptar un software antiguo a un sistema actual. C_{riesgo}, por su parte, depende de que exista una comunidad que mantenga las adaptaciones necesarias; sin ella, $C_{\text{riesgo}} \to \infty$ y el sistema es inviable aunque el hardware funcione. Este es el criterio que distingue un despliegue puntual de un despliegue sostenible.

1.5. Filtro de estabilidad: fallo contenido vs. fallo propagado

Los cuatro filtros anteriores comparten una asunción implícita: que, superados los obstáculos de compatibilidad de *software* y de rendimiento, el hardware se comporta de forma determinista y su fallo, si ocurre, queda contenido al propio dispositivo. Esta asunción no siempre se sostiene, especialmente con hardware antiguo, y motiva la incorporación de un quinto filtro: el filtro de estabilidad. En él, un fallo se considera **contenido** cuando

sus efectos se limitan al dispositivo que falla, y **propagado** cuando alcanzan al resto del sistema: al *host*, a los servicios que se ejecutan en él o a sus datos.

El objetivo de este filtro no es exigir que el hardware no falle —con dispositivos de más de quince años de antigüedad, los fallos son previsibles—, sino que, cuando falle, el fallo quede contenido en el propio dispositivo y pueda resolverse sin intervención física sobre la máquina. Como no es posible enumerar de antemano todos los fallos posibles, el filtro se descompone en dos componentes verificables: una que puede comprobarse antes del despliegue y otra que solo puede comprobarse mediante una prueba de campo.

$$F_{\text{estabilidad}} = F_{\text{recuperación}} \wedge F_{\text{carga sostenida}} \tag{1.17}$$

Componente previa al despliegue: $F_{\text{recuperación}}$. Evalúa si existe algún mecanismo para devolver un dispositivo colgado a un estado funcional sin intervención física. Apoyándose en los mecanismos de recuperación de dispositivos PCIe disponibles en el kernel Linux, que este expone en el atributo `reset_method` de cada dispositivo y en el directorio de ranuras con control de alimentación (`/sys/bus/pci/slots`) [47, 21]:

$$\begin{aligned} F_{\text{recuperación}} = \text{cierto} \iff & (\texttt{flr} \in \texttt{reset_method}) \\ & \vee\ (\exists\,\text{recurso de energía ACPI a nivel de slot}) \end{aligned} \tag{1.18}$$

Es decir: un dispositivo es recuperable por *software* ante un fallo de enlace o de contexto si soporta *Function Level Reset* real, **o** si la plataforma expone un mecanismo alternativo de corte/restauración de alimentación a nivel de *slot* (*hotplug* de energía), que permita un ciclo de encendido dirigido sin intervención física sobre el hardware.

Componente posterior al despliegue: $F_{\text{carga sostenida}}$. La existencia de un mecanismo de reinicio no garantiza por sí sola que los fallos queden contenidos: un fallo puede bloquear el propio host antes de que ningún mecanismo llegue a actuar. Por ello, la segunda componente exige superar una prueba de carga prolongada, una práctica habitual en la validación de hardware reacondicionado conocida como *burn-in*. $F_{\text{carga sostenida}}$ es cierto si el sistema, con todas las GPUs sometidas a carga máxima de forma continua durante un tiempo mínimo T, no presenta ninguno de los siguientes eventos:

- bloqueos del kernel (*soft lockups* o *hard lockups*);
- errores internos del driver (en el driver de NVIDIA, los mensajes `Xid` del registro del kernel);
- reinicios del host;
- pérdida de algún dispositivo del bus;

- temperaturas por encima del máximo especificado por el fabricante.

La duración T debe fijarse en función del uso previsto del sistema: como mínimo, igual a la del trabajo más largo que se espere ejecutar de forma continua. Una prueba más corta que el uso real no aporta ninguna garantía sobre él, mientras que un fallo dentro del intervalo T basta para que la componente sea falsa.

Conviene subrayar que los mecanismos de *detección* del kernel no son mecanismos de *recuperación*: los *watchdogs* de *softlockup* y *hardlockup* se limitan a registrar el diagnóstico y, si se les configura así, a provocar el reinicio del sistema [22]. Por tanto, un fallo de este tipo nunca puede considerarse contenido al dispositivo, sino que exige la intervención del operador.

Coste asociado a $F_{\text{estabilidad}}$ falso. Cuando el filtro de estabilidad falla y el fallo no queda contenido al dispositivo, el coste de riesgo de la ecuación (1.16) no es simplemente alto, sino potencialmente **no acotado**:

$$F_{\text{estabilidad}} = \text{falso} \ \wedge \ \text{el fallo se propaga fuera del dispositivo asignado} \implies C_{\text{riesgo}} \to \infty \tag{1.19}$$

Un fallo que se propaga al resto del sistema no solo detiene el cómputo en la GPU afectada: puede bloquear el *host*, interrumpir todos los servicios que se ejecutan en él e incluso comprometer la integridad de sus datos. Por eso, un sistema que no supera este filtro no es apto para un uso continuado, con independencia de que los restantes cuatro filtros sean favorables.

Capítulo 2

Planteamiento del problema y primer acercamiento

En este capítulo se presenta el dilema general del trabajo, el hardware sobre el que se experimenta y el primer acercamiento al objetivo: la integración de GPUs de distinta naturaleza en un mismo sistema.

2.1. La limitación del driver único

El punto de partida del trabajo es una restricción técnica fundamental: **en un mismo sistema operativo no puede haber más de una versión de driver propietario cargada al mismo tiempo**. El motivo es que el módulo del driver (`nvidia.ko`) controla elementos globales del *kernel*, no solo de cada tarjeta individual: gestiona la memoria unificada, registra los archivos de comunicación con el sistema (`character devices`) y expone funciones globales en el núcleo. Cargar dos versiones distintas sobre el mismo sistema generaría un conflicto directo por el control de estos recursos. Esta limitación es ampliamente conocida en sistemas de esta índole y es precisamente la línea conductora que define todas las decisiones tomadas a lo largo de este proyecto.

Partiendo de esta premisa, surgen dos escenarios posibles:

1. **Un único driver para todas las GPUs:** Es viable únicamente si todas las tarjetas instaladas comparten una misma versión de driver. Esto limita bastante el hardware que se puede combinar en una misma máquina, ya que exige utilizar tarjetas de épocas similares que compartan soporte. Este es precisamente nuestro caso de estudio: se dispone de dos Tesla C1060 y una ASUS ENGTS 250, modelos compatibles con la misma rama de drivers *legacy* de NVIDIA. La configuración experimental empleó una Tesla C1060 y la ENGTS250, porque la tercera tarjeta no fue detectada a través del *riser* PCIe.

2. **Aislamiento de cada tarjeta en un entorno propio:** Cada GPU se asigna de forma exclusiva a una máquina virtual con su propio sistema operativo y su propia versión de driver. Esta vía evita por completo la limitación del driver único, a cambio de la complejidad técnica que supone configurar el paso directo (*passthrough*) con VFIO.

Es digno de mención que el uso de contenedores (como Docker o LXC) no resuelve este problema: los contenedores aíslan aplicaciones y librerías en el espacio de usuario, pero todos comparten obligatoriamente el mismo *kernel* y el mismo módulo de driver cargado en el anfitrión. Esta diferencia entre aislar procesos por software y aislar hardware real es una de las piedras angulares de este trabajo.

2.2. Descripción del entorno hardware

El entorno sobre el que se ha desarrollado todo el trabajo es el siguiente:

Componente	**Valor**
Placa base	ASUS TUF GAMING X670E-PLUS WIFI (con iGPU integrada)
CPU	AMD Ryzen 9 7950X 16-Core Processor
Hipervisor	Proxmox VE 9.2.0
GPU 1	NVIDIA Tesla C1060 (GT200, cómputo)
GPU 2	ASUS ENGTS250 (G92b, consumo)
Driver objetivo	NVIDIA 340.108

Cuadro 2.1: Entorno hardware del trabajo

El detalle de las especificaciones de las GPUs se presentó en la tabla 1 del capítulo de preliminares. Tanto la placa base como la CPU son componentes modernos (2022–2025), lo cual, como veremos, introduce problemas de compatibilidad con los sistemas operativos antiguos que el hardware *legacy* requiere.

2.2.1. Carriles PCIe disponibles: cuántas GPUs admite la plataforma

En una plataforma de consumo, el número de GPUs que pueden conectarse con un ancho de banda razonable lo limita el procesador, que es quien ofrece los carriles PCIe (sección 1.3). El Ryzen 9 7950X expone 28 carriles, no todos de la misma generación [1, 2]:

- 16 carriles PCIe 5.0 para la ranura gráfica principal, configurables como un único enlace x16 o como dos enlaces x8 (bifurcación x8/x8);
- 4 carriles PCIe 5.0 para la unidad NVMe principal;

- 8 carriles PCIe 4.0 adicionales: 4 para una segunda unidad NVMe y 4 para el enlace con el *chipset*.

La placa ASUS TUF GAMING X670E-PLUS WIFI reparte esos recursos en tres ranuras de expansión [8]:

- una ranura PCIe 5.0 x16 conectada directamente al procesador;
- una ranura PCIe 4.0 con formato físico x16, pero solo cuatro carriles eléctricos (x4), conectada a través del *chipset*;
- una ranura PCIe 4.0 x4, también conectada a través del *chipset*.

Esta distribución condiciona directamente cuántas GPUs puede alojar el sistema y en qué condiciones. Solo la ranura principal ofrece un enlace directo y completo con el procesador. Cualquier GPU adicional debe conectarse a una ranura del *chipset*, con cuatro carriles como máximo y compartiendo con discos, red y puertos USB el único enlace x4 entre el *chipset* y el procesador. La ranura principal puede dividirse en dos enlaces x8 mediante la bifurcación, pero eso reduce a la mitad el ancho de banda de cada una. A todo ello se suma una limitación física: las tarjetas del caso de estudio ocupan dos ranuras de ancho, de modo que no todas las ranuras pueden usarse a la vez sin recurrir a cables alargadores (*risers*). En la práctica, por tanto, la plataforma admite pocas GPUs, y solo una de ellas con el enlace completo.

Existe una forma de ampliar ese número: dividir los dieciséis carriles de la ranura principal en cuatro enlaces x4 (bifurcación x4/x4/x4/x4), siempre que la placa ofrezca ese modo, y conectar a ella un adaptador o divisor (*splitter*) que reparta esos enlaces entre cuatro GPUs mediante *riser*. Junto con las ranuras del *chipset*, el sistema podría alojar así cuatro o cinco tarjetas. Esta ampliación tiene, sin embargo, un precio:

- **Ancho de banda.** La bifurcación multiplica el número de enlaces, pero no los carriles: cada GPU dispone de un enlace x4, el mismo ancho que una ranura del *chipset*, y el ancho de banda total de la ranura principal sigue siendo el mismo, ahora repartido entre cuatro tarjetas.
- **Fiabilidad del enlace.** Los *risers* y los divisores añaden longitud y conectores al enlace, lo que dificulta la negociación de su velocidad, precisamente la fase en la que estas tarjetas antiguas ya presentan problemas. En este trabajo, la tercera tarjeta no llegó a ser detectada a través de un cable alargador.
- **Alimentación y espacio.** Cada tarjeta añade su propio consumo, hasta 188 W en el caso de la Tesla C1060 [27], lo que exige una fuente de alimentación dimensionada para ello y un chasis o un soporte abierto donde montar las tarjetas fuera de la placa.

Añadir tarjetas es, por tanto, posible, pero cada una aporta menos que la anterior y aumenta el riesgo de que el conjunto falle.

2.2.2. Configuración de firmware de la plataforma

En este apartado se detallan los parámetros modificados en el firmware de la placa base con el fin de acondicionar el arranque del sistema a las restricciones operativas de un hardware gráfico tan antiguo como el que aquí acontece.

Como primer objetivo, estaba lograr que la plataforma superase con éxito la fase de inicialización POST (*Power-On Self-Test*), etapa de autodiagnóstico en la que el firmware escanea, asigna recursos y enumera los dispositivos conectados a los buses antes de transferir el control al cargador de arranque.

Durante la configuración inicial, mientras la salida de vídeo dependía de una de las tarjetas *legacy*, fue necesario habilitar el módulo de compatibilidad CSM (*Compatibility Support Module*) con prioridad de ejecución para memorias *Legacy OpROM*. La *OpROM* es la memoria no volátil donde se encuentra la VBIOS (*Video BIOS*) de la GPU, el código mínimo que la placa base ejecuta para inicializar la tarjeta como dispositivo de vídeo durante el arranque. Dada la generación a la que se remonta el hardware de este trabajo, su VBIOS carece del protocolo GOP (*Graphics Output Protocol*), el estándar con el que las plataformas UEFI inicializan la salida de vídeo durante el POST.

Como consecuencia, con un arranque en modo UEFI nativo puro la placa base no puede ejecutar la VBIOS de la tarjeta para mostrar imagen, y el arranque se detenía en el POST. La activación del módulo CSM solventa esta incompatibilidad emulando un entorno BIOS tradicional, lo que permite ejecutar el código de 16 bits de la VBIOS.

Esta limitación solo afecta a la tarjeta que proporciona la imagen durante el arranque. En la configuración final, la salida de vídeo la proporciona la iGPU integrada en el procesador, que es moderna y dispone de GOP, y las dos tarjetas *legacy* se usan únicamente como aceleradores de cómputo: la GTS 250 cumple el mismo papel que la Tesla C1060, que ni siquiera dispone de salidas de vídeo. Como GOP solo interviene en la salida de vídeo del firmware, su ausencia no debería afectar a ninguna de las dos: durante el POST el firmware se limita a enumerarlas y asignarles recursos, y es el driver de NVIDIA quien las prepara para el cómputo una vez arrancado el sistema operativo. De hecho, la instalación de Proxmox VE dispone de cargadores de arranque para ambos modos, y el registro del kernel muestra que el sistema arrancó en modo UEFI con el kernel empleado en las pruebas, sin necesidad del CSM.

En segundo lugar, dado que este es un sistema multiGPU, es necesario configurar también la distribución eléctrica de las líneas de datos (*lanes*) para evitar que el firmware asigne la totalidad del ancho de banda al bus principal, manteniendo operativas ambas ranuras,

principal y secundaria de forma concurrente.

Durante las pruebas se forzó manualmente a PCIe Gen 1 el enlace de las ranuras de la placa (`PCIEX16_1`, `PCIEX16_2` y `PCIEX4`). Si bien esto supone una pérdida considerable de ancho de banda, asegura una comunicación más tolerante y evita que el bus actual descarte las tarjetas antiguas ante discrepancias en los tiempos de respuesta.

2.2.3. Arranque dual LXC/VFIO

Las configuraciones de LXC y VFIO no pueden convivir en el mismo arranque: la primera necesita que el anfitrión cargue el driver NVIDIA, mientras que la segunda necesita reservar las GPUs para `vfio-pci`. Por ello se implantó un arranque dual mediante GRUB, el gestor que presenta el menú inicial del sistema y selecciona el kernel y sus parámetros.

Este arranque dual no formaba parte del planteamiento inicial. Surgió al pasar de la vía de máquinas virtuales a la de contenedores, cuando se comprobó que cada cambio de enfoque obligaba a revertir la configuración del host (capítulo 4). El arranque por defecto corresponde al modo LXC, por ser la configuración más estable. Para retomar los experimentos con máquinas virtuales se añadió una entrada personalizada de GRUB que arranca el mismo kernel con parámetros orientados a VFIO:

```
pcie_port_pm=off pcie_aspm=off amd_iommu=on iommu=pt pci=noaer \
  vfio_pci.disable_idle_d3=1 vfio_pci.disable_vga=1 vfio_dynamic
```

Además, los módulos `vfio`, `vfio_iommu_type1`, `vfio_pci` y `vfio_virqfd` quedan disponibles para que el hardware pueda vincularse a `vfio-pci`. La palabra `vfio_dynamic` no es una opción estándar del kernel: es una marca local introducida expresamente para identificar este arranque. Cuando está presente, un script ejecutado durante el arranque desvincula las GPUs NVIDIA del driver del host y las vincula a `vfio-pci` a través de la interfaz `sysfs` del kernel, obteniendo sus identificadores en ese momento. Esto sustituye a la reserva estática por identificadores empleada en la primera fase (capítulo 3), que excluía el driver NVIDIA del host y obligaba a revertir la configuración para volver a usar contenedores. En el modo LXC, el script no actúa y el host carga sus drivers con normalidad.

La selección de VFIO es de un solo arranque. El comando `boot-vfio` fija dicha entrada únicamente para el próximo reinicio y cancela la operación si GRUB no acepta la selección; `GRUB_DEFAULT` permanece en la entrada estable de LXC. Un servicio *one-shot* limpia después la variable interna `next_entry`, de modo que los reinicios siguientes vuelven automáticamente al modo LXC. El comando `check-mode` inspecciona `/proc/cmdline` y muestra las GPUs detectadas con su driver activo.

Con esta base común de plataforma ya estabilizada, el primer experimento consistió en probar la vía más directa: un sistema Ubuntu con soporte nativo para el driver antiguo.

2.3. Primer acercamiento: Ubuntu 18.04 LTS con el driver 340.40

La primera aproximación al problema consistió en instalar un sistema operativo relativamente antiguo —Ubuntu 18.04 LTS (núcleo 4.15, GCC 7.x)— por contar con soporte directo para el controlador requerido por las tarjetas. En esta versión, el paquete `nvidia-340` de los repositorios oficiales de Canonical se instalaba sin fricción, ya que la distribución mantenía binarios y parches preparados específicamente para dicho *kernel*.

La instalación resultó sencilla y el controlador detectó ambas GPUs sin inconvenientes. Sin embargo, este acercamiento pronto mostró sus limitaciones. Tanto la placa base como el procesador del equipo de pruebas son componentes posteriores a esta versión de Ubuntu, por lo que un *kernel* anterior carece del soporte adecuado para la gestión del *chipset* moderno.

A esto se sumó el aspecto verdaderamente crítico para los objetivos del trabajo: Ubuntu 18.04, con su *kernel* base 4.15, no permitía un particionamiento limpio de los grupos IOMMU ni ofrecía una pila de virtualización suficientemente actualizada para gestionar los dispositivos PCIe con garantías. Al no reconocer la topología de interconexión de la placa base moderna, el *kernel* antiguo agrupaba múltiples periféricos independientes dentro de un mismo grupo IOMMU por motivos de seguridad de memoria. Dado que el paso directo mediante VFIO exige aislar el grupo al completo, resultaba inviable desacoplar una única tarjeta sin arrastrar otros componentes del sistema.

Por otro lado, actualizar dicho núcleo a una versión más reciente para solventar este problema rompía de inmediato el módulo `nvidia-340`, cuyo código fuente no estaba adaptado a las interfaces internas de memoria de los *kernels* posteriores. Ante este conflicto, mantener el sistema anfitrión en esta distribución impedía disponer simultáneamente del soporte moderno para aislar los dispositivos por hardware y del entorno necesario para operar las tarjetas.

Capítulo 3

Aislamiento de GPUs con máquinas virtuales

En este capítulo se aborda el segundo de los escenarios planteados en el capítulo 2: encapsular cada una de las tarjetas en una máquina virtual, cada una con su sistema operativo y su versión de driver correspondiente. Se describen los fundamentos del aislamiento de dispositivos, los obstáculos encontrados y el fallo documentado que llevó a descartar esta vía.

3.1. Planteamiento

Recordemos la situación de partida. Se disponía de tres GPUs NVIDIA que solo admiten la rama de driver 340.xx, aunque la configuración experimental empleó las dos tarjetas detectadas, y de un sistema moderno (Proxmox VE) que difícilmente puede ejecutar esa rama sin parches. La idea de este capítulo es dar la vuelta al problema: en lugar de hacer que el host ejecute el driver antiguo, se crean máquinas virtuales con sistemas operativos con compatibilidad para ello, cada una de las cuales recibe una GPU en exclusiva mediante el paso directo de dispositivos (VFIO). Cada máquina virtual carga su propio driver, y la limitación del driver único queda superada por construcción.

Para que el *passthrough* funcione, se necesita que el host cumpla tres condiciones [40, 23]:

1. Disponer de IOMMU activada, de forma que los accesos DMA de cada dispositivo queden restringidos a su espacio de direcciones virtual.
2. Que la GPU objetivo constituya por sí sola un grupo IOMMU (o que todos los miembros del grupo puedan cederse), de modo que el kernel permita el aislamiento individual del dispositivo.

3. Que el dispositivo soporte correctamente el **Function Level Reset**, de modo que al reiniciar la máquina virtual el dispositivo pueda devolverse a un estado limpio y listo para el próximo arranque.

3.1.1. Ajustes de configuración específicos para VFIO

Para el despliegue de esta fase experimental se diferenciaron dos niveles complementarios de virtualización asistida por hardware:

- **SVM / AMD-V:** *Secure Virtual Machine* (SVM) es el nombre de la arquitectura de instrucciones que conforma la tecnología comercial AMD-V (*AMD Virtualization*). Proporciona las extensiones de hardware en el procesador necesarias para que el hipervisor KVM ejecute el código de las máquinas virtuales de forma nativa en la CPU física.
- **AMD-Vi:** Constituye la implementación de AMD para la IOMMU (*Input-Output Memory Management Unit*), el circuito integrado encargado de traducir y restringir los accesos directos a memoria física (*Direct Memory Access*, DMA) emitidos por los periféricos.

En la configuración del firmware UEFI se habilitaron explícitamente ambas características: el modo SVM/AMD-V para permitir la instanciación de máquinas virtuales completas, y AMD-Vi para que el *kernel* de Linux estructurase los grupos IOMMU necesarios para el aislamiento de cada GPU mediante el controlador `vfio-pci`.

Asimismo, fue necesario neutralizar *Nouveau* —el controlador libre y de código abierto para GPUs NVIDIA incluido por defecto en el núcleo de Linux—, ya que de lo contrario interferirá en la asignación de las tarjetas a *vfio-pci*. Para impedirlo, se añadió el módulo a la lista negra (*blacklist*) [6].

En un primer momento, las GPUs se reservaron para `vfio-pci` desde el arranque mediante sus identificadores de dispositivo. Esta reserva impide que el host cargue el driver NVIDIA, por lo que tuvo que revertirse al pasar a la vía de contenedores (capítulo 4). El proceso no fue, por tanto, lineal: para poder retomar las pruebas con máquinas virtuales sin volver a revertir la configuración del host, se diseñó después el arranque dual descrito en el capítulo 2, cuya entrada VFIO incorpora los parámetros del kernel que se describen en este capítulo. Su uso operativo se detalla en el material complementario (sección A.3).

3.2. Evaluación experimental de passthrough mediante VFIO

El objetivo de esta fase era comprobar si podíamos aislar cada una de nuestras tarjetas gráficas y pasárselas de forma exclusiva a una máquina virtual con KVM/QEMU vinculándolas al módulo `vfio-pci`.

3.2.1. El primer tropiezo: la ASUS ENGTS250 no responde

Con las GPUs reservadas para `vfio-pci`, comprobamos qué dispositivos se vinculan correctamente al controlador `vfio-pci`. Lo que cabía esperar era que al escanear el bus PCIe aparecieran las dos tarjetas conectadas, una de las Tesla C1060, en la ranura principal, y la ASUS en la ranura secundaria. La sorpresa fue que en la lista únicamente figuraba la Tesla C1060; la ASUS ENGTS250 no aparecía.

Consultando los registros del *kernel*, encontramos el origen del problema: un fallo durante la fase de negociación de la velocidad y anchura del bus PCIe (*link training*). Este paso ocurre en la capa de enlace puramente física, mucho antes de que ningún controlador entre en juego. Al intentar aplicar un reinicio sobre la tarjeta, esta simplemente no contestó a tiempo, y el *chipset* de la placa base la catalogó directamente como rota (*broken device*) para no comprometer la estabilidad del resto del bus.

Este estado de bloqueo es crítico: una vez que el sistema coloca al dispositivo con esta marca, cesan todos los reintentos de comunicación. Ni siquiera conmutando entre configuraciones lograríamos devolver la tarjeta a su estado operativo. La tarjeta queda eléctricamente aislada donde un reinicio por software resulta inútil, obligando a recurrir a un apagado manual completo (corte físico de corriente). Este debería ser siempre el último recurso, pues en un escenario sin acceso presencial al equipo dejaría el nodo totalmente inutilizable.

3.2.2. Comprobaciones y ajustes en busca de una solución

Antes de resignarnos con el peor escenario, se explora qué margen de maniobra exponen las tarjetas mediante software y configuración:

1. Inspección de los mecanismos de reinicio (Reset): Consultamos qué métodos de reinicio soportaba la tarjeta a través de *sysfs*:

```
$ cat /sys/bus/pci/devices/0000:08:00.0/reset_method
pm bus
```

Dado que la arquitectura de la familia GT200 carece de soporte por hardware para FLR (*Function-Level Reset*), el *kernel* debe recurrir a los mecanismos de contingencia expuestos por el dispositivo. La consulta a `reset_method` confirmó que las únicas alternativas disponibles eran la transición de energía (`pm`) y el reinicio del bus, en este caso, (`bus`, correspondiente a *Secondary Bus Reset* o SBR) por estar en la ranura compartida con el *chipset*.

El método `pm` se apoya en el subsistema de gestión de energía PCI: fuerza al dispositivo a entrar en estado de bajo consumo `D3hot` (con alimentación presente pero relojes detenidos y pérdida del contexto interno) para devolverlo de inmediato a `D0` (rendimiento máximo), confiando en que la tarjeta se reinicie de forma limpia durante la transición [21]. Sin embargo, en hardware de esta generación, dicha transición describe un comportamiento conocido, y por tanto, mucho menos fiable. Ante esta incertidumbre, el *kernel* se ve forzado a aplicar el método `bus` (SBR): una maniobra sustancialmente más agresiva que, al no poder aislar la GPU, induce una interrupción eléctrica abrupta sobre el enlace PCIe completo para forzar su reinicialización.

Este comportamiento resulta llamativo en lo relativo a las diferencias de comportamiento de las tarjetas durante las pruebas: a pesar de compartir una arquitectura muy similar, la inestabilidad y el colapso del bus se manifestaron de forma crítica en la ASUS ENGTS250 y no en la Tesla C1060. Como hipótesis de trabajo, la causa más plausible apunta a la topología física de conexión de cada tarjeta en la placa base. Mientras que la ranura principal de la Tesla se comunica a través de líneas PCIe dedicadas conectadas directamente a la CPU, la ASUS se encontraba instalada en una ranura secundaria subordinada a las líneas del *chipset*. Al inducir un reinicio secundario de bus (SBR) sobre un enlace intermediado por el *chipset*, la perturbación eléctrica no solo afectó a la GPU, sino que desestabilizó el conmutador interno de dicho puente, arrastrando consigo a otros periféricos multiplexados en el mismo bus —incluida la controladora de almacenamiento del *host*— y provocando el bloqueo total del sistema.

2. Control eléctrico del slot (Hotplug Power): Comprobamos si la placa base permite cortar la corriente de la ranura PCIe por software para emular el tirón de cable:

```
$ ls /sys/bus/pci/slots/
```

El resultado fue negativo. La gestión de energía independiente por ranura es habitual en placas de servidor, pero inexistente en placas de consumo convencionales.

Estas dos comprobaciones permiten evaluar la componente $F_{\text{recuperación}}$ del filtro de estabilidad (sección 1.5), que exige al menos una de dos vías de recuperación por *software*. La primera no se cumple: `flr` no figura entre los métodos de reinicio de la tarjeta (`pm`

`bus`), como corresponde a una arquitectura sin soporte de FLR. La segunda tampoco: el directorio `/sys/bus/pci/slots/` está vacío, de modo que la placa no ofrece control de alimentación por ranura. Por tanto, $F_{\text{recuperación}} = \text{falso}$: si una de estas tarjetas se cuelga, no existe ningún mecanismo para recuperarla sin intervenir físicamente sobre la máquina.

3. Desactivación de mecanismos de ahorro de energía: La hipótesis de trabajo era que las funciones de gestión de energía del PCIe estaban mandando a la tarjeta a estados de reposo de los que luego era incapaz de despertar a tiempo. Por ello, aplicamos una serie de directivas agresivas en la línea de comandos del *kernel* (`cmdline`) y en la BIOS:

- **`vfio_pci.disable_idle_d3=1`:** Evita que el controlador ponga la GPU en reposo profundo ($D3_{hot}$) mientras la máquina virtual no la usa. Al ser la tarjeta tan antigua, los comportamientos no son predecibles.
- **`pcie_aspm=off`:** Deshabilita por completo el ahorro de energía en el propio enlace físico PCIe (*Active-State Power Management*).
- **`pcie_port_pm=off`:** Desactiva la gestión de energía en el puente PCIe que da acceso a la GTS250. Este parámetro fue determinante para evitar que el puente del que cuelga la GPU cortase la comunicación durante la reconfiguración.
- **`pci=noaer`:** Desactiva el reporte avanzado de errores para evitar que el sistema sobrerreaccione ante pequeñas inestabilidades del enlace. Este parámetro más que solucionar un problema en caso de haberlo, lo enmascararía.

Tras forzar la modificación de estos parámetros, se consiguió que ambas GPUs, Tesla C1060 y ahora también la ASUS ENGTS 250, se vinculasen al módulo *vfio*. De entre los parámetros modificados, la principal sospecha de éxito recayó en `vfio_pci.disable_idle_d3=1`: impedir que VFIO dejara la GPU en `D3hot` eliminaba una transición energética problemática. No obstante, como todas estas directivas se aplicaron conjuntamente, su efecto individual no quedó aislado; esta explicación es coherente con el momento del cuelgue, pero no demuestra por sí sola cuál fue el parámetro determinante. Sabiendo que con estas pruebas estábamos exponiendo al sistema a comportamientos eléctricos no conocidos, se decidió dar por válida la vinculación sin determinar cuál de los parámetros exactos nos hizo avanzar.

Así pues con estos cambios conseguimos el primer gran avance: la ASUS ENGTS250 completó el *binding* con `vfio-pci` de forma consistente. La negociación se estabilizó a menor velocidad ($\times 4$ a 2.5 GT/s en lugar de sus $\times 16$ nominales), lo que ralentizaba ligeramente el arranque, pero al menos habíamos eliminado el cuelgue eléctrico inicial.

3.2.3. La prueba de fuego en la máquina virtual y el cuelgue definitivo

Con el controlador `vfio-pci` adueñado de la tarjeta, lo siguiente era comprobar si podíamos levantar una máquina virtual funcional. Diseñamos un protocolo de validación adaptado a las limitaciones heredadas de las tarjetas:

1. Se configuró la máquina virtual con **SeaBIOS** (emulación de BIOS clásica de 16 bits) y tipo de máquina **i440fx**, debido a la ausencia de GOP en la VBIOS de la GPU.
2. Se instaló el sistema operativo base sin la GPU conectada para evitar interferencias.
3. Se añadió la tarjeta gráfica mediante la directiva `hostpci0: 08:00.0`, aplicando dos precauciones clave:
    - **Desactivar la lectura de la ROM (`rombar=0`):** Evita que QEMU exponga la memoria ROM de la tarjeta (*Expansion ROM BAR*) a la máquina virtual. En tarjetas antiguas, el acceso y mapeo de esta región de memoria entra en conflicto con los registros de control principales (BAR0/MMIO) del dispositivo una vez que este ya ha sido inicializado. Dado que la GPU se destina exclusivamente a tareas de cómputo y no a la salida gráfica del arranque de la VM, la exposición de la VBIOS resulta innecesaria y su desactivación elimina bloqueos por contención en el bus.
    - **Desactivación de la GPU primaria (`x-vga=0`):** Evita que el hipervisor trate a la GPU física como el adaptador VGA principal de la máquina virtual. En su lugar, se configuró una tarjeta gráfica virtual emulada (*VirtIO-GPU*), la cual redirige la salida de vídeo del sistema operativo invitado hacia la consola web del hipervisor mediante el protocolo noVNC (cliente VNC integrado en la interfaz de Proxmox). De este modo, la GTS250 queda liberada de cualquier función de salida de vídeo, quedando asignada como acelerador secundario dedicado exclusivamente a cómputo.

Pese a todas las precauciones, en el momento exacto en que la máquina virtual intentó arrancar e inicializar la tarjeta, el sistema anfitrión se congeló por completo, obligando de nuevo a forzar el apagado manual del equipo.

Como detalle relevante, la interfaz de Proxmox reportó la máquina virtual como iniciada con éxito mientras el equipo ya estaba totalmente bloqueado; un claro falso positivo en la monitorización. Esta prueba confirmó de manera empírica que, aun resolviendo los problemas del enlace físico a base de parámetros en el *kernel*, la incapacidad de estas tarjetas antiguas para recuperarse de un reinicio por software hace inviable su uso estable mediante VFIO en nuestra plataforma.

3.2.4. Un fallo que alcanzó al host completo

Una prueba posterior, esta vez con la Tesla C1060 asignada a una máquina virtual, mostró que las consecuencias de esta incapacidad pueden ir más allá de la propia tarjeta. Al arrancar la máquina virtual, el registro del *kernel* mostró lo siguiente:

```
vfio-pci 0000:01:00.0: Unable to change power state from D3cold to D0,
    device inaccessible
vfio-pci 0000:01:00.0: resetting
pcieport 0000:00:01.1: unlocked secondary bus reset via:
    pci_reset_bus_function+0x15c/0x180
vfio-pci 0000:01:00.0: reset done
vfio-pci 0000:01:00.0: Unable to change power state from D3cold to D0,
    device inaccessible
```

La tarjeta dejó de responder y el *kernel*, a falta de FLR, recurrió al reinicio del bus a través del puerto PCIe del que cuelga. El reinicio se completó (`reset done`), pero la tarjeta siguió sin responder: el enlace se reinicia, pero el chip no vuelve a un estado conocido.

Inmediatamente después, órdenes básicas del sistema como `tail` o `cat` empezaron a fallar con errores de entrada/salida, y el sistema de ficheros raíz del host apareció montado en modo de solo lectura de emergencia (`emergency_ro`): el propio sistema de ficheros había detectado una inconsistencia grave y se había protegido. Al reiniciar, el equipo no llegó a arrancar: no había imagen en ninguna salida de vídeo ni respuesta de red. Tras un corte completo de alimentación, la BIOS arrancó, pero no detectaba ningún disco de arranque. El sistema solo se recuperó tras **retirar físicamente ambas GPUs** y repetir el corte de alimentación.

No pudo determinarse con certeza la cadena causal. La hipótesis más plausible es que el reinicio del bus afectara de forma colateral a otros dispositivos de la plataforma, entre ellos la controladora del disco de sistema, y que alguna de las tarjetas quedara en un estado eléctrico anómalo que persistía tras los apagados normales e impedía detectar el disco durante el arranque. En cualquier caso, el incidente muestra que, en esta plataforma, un reinicio de bus forzado sobre estas tarjetas no es una operación aislada: puede comprometer la integridad del host completo.

3.3. Resultado y decisión

Tras múltiples intentos de configuración (activación de IOMMU en la BIOS, reserva de las GPUs con el módulo `vfio-pci`, pases de ID de dispositivo, etc.), el despliegue de las VMs con VFIO se descartó por inviable:

- La ausencia de **FLR** de ambas tarjetas impide la reutilización del dispositivo entre máquinas virtuales: el ciclo de vida de la VM (apagado → reinicio) deja la GPU inaccesible eléctricamente.

Como se mencionó anteriormente por integridad del sistema, se decidió no repetir sistemáticamente estos bloqueos eléctricos: cada cuelgue exigía un apagado manual y exponía el sistema de ficheros y el arranque a nuevos daños. La conclusión se apoya, por tanto, en la repetición del patrón de fallo y en su coherencia con la ausencia de recuperación por software ya que el apagado manual no es ni solución definitiva ni robusta.

3.4. Análisis comparativo: máquinas virtuales frente a contenedores

A la vista de lo experimentado en este capítulo, la tabla 3.1 resume la comparación cualitativa entre las dos vías de compartición de GPUs analizadas hasta ahora. Aunque el capítulo 4 y siguientes desarrollan la vía de los contenedores, anticipar aquí la comparación ayuda a situar el fallo de este capítulo en su contexto: no se trata de una vía mala en general, sino de una vía inadecuada para las características concretas de nuestro hardware.

Criterio	**Máquina virtual (VFIO)**	**Contenedor LXC**
Aislamiento del hardware	Total (dispositivo dedicado)	Compartido (*device nodes* del host)
Versiones de driver	Una distinta por VM	Una única para todo el sistema
Requisitos de plataforma	IOMMU activa, grupos IOMMU, FLR correcto	Ninguno
Reinicio del dispositivo	Necesita reset limpio (FLR); crítico para el ciclo de vida de la VM	No interviene: el driver sigue en el host
Coste de configuración	Alto (BIOS, `vfio-pci`, ROM de vídeo)	Bajo (*cgroup* + *bind mount*)
Uso conjunto con el host	Imposible (GPU exclusiva de la VM)	Posible (driver compartido)
Idoneidad con hardware *legacy*	Mala (FLR incompleto, VBIOS sin UEFI)	Buena (el driver lo resuelve el host)

Cuadro 3.1: Comparación cualitativa entre máquinas virtuales y contenedores para la compartición de GPUs

La conclusión de este capítulo es que la virtualización completa no es la vía adecuada para integrar GPUs *legacy* en sistemas modernos. No obstante, el análisis no fue en vano: delimitó con precisión las condiciones que debe cumplir cualquier vía alternativa. La vía de los contenedores LXC, que se estudia en el capítulo siguiente, no necesita ni IOMMU ni reset de dispositivos, ya que el contenedor comparte el kernel del host y, por tanto, el

propio driver que ya controla las tarjetas. La contrapartida es volver a la limitación del driver único: solo puede haber un driver propietario en el sistema.

Capítulo 4

Virtualización ligera con LXC: integración del driver legacy en Proxmox VE

En este capítulo se abandona la vía de las máquinas virtuales y se adopta la virtualización ligera con contenedores LXC, que comparten el kernel del anfitrión. El reto de este capítulo se traslada a la instalación del driver *legacy* NVIDIA 340.108 sobre un kernel moderno: se documentan los obstáculos encontrados, la solución mediante parches comunitarios y DKMS, y la validación del módulo resultante.

4.1. Por qué LXC

La tecnología de contenedores LXC presenta, frente a las máquinas virtuales, una diferencia fundamental: los contenedores comparten el kernel del host. Esta característica, que en el capítulo 3 nos parecía una limitación, es precisamente la clave de esta fase. Al compartir el kernel, el contenedor no necesita un driver propio: usa el módulo del driver NVIDIA que carga el host, y solo necesita montar los *character devices* correspondientes. No hay IOMMU, no hay grupos, no hay resets: el aislamiento a nivel de hardware no interviene.

La contrapartida es inmediata: volvemos a la limitación del driver único. Solo puede haber un driver propietario de NVIDIA a nivel de sistema, y ese driver debe ser capaz de controlar todas las GPUs. Como hasta ahora ambas tarjetas de pruebas comparten la misma rama *legacy* (340.xx), consideramos que valía la pena hacer la prueba. El problema se traslada entonces de la virtualización a la instalación del driver: conseguir que el driver 340.108 —con último release oficial en 2019— funcione sobre un kernel de 2026.

4.2. Diagnóstico de la incompatibilidad fundamental

Para entender el origen de la incompatibilidad conviene recordar cómo se integra un driver en Linux. Un driver como el de NVIDIA no es un programa independiente, sino un *módulo* que se carga dentro del propio kernel. Para construirlo hay que compilar su código fuente contra los *headers* del kernel —los ficheros que describen la estructura interna del mismo— y con un compilador (gcc) compatible con el que se usó para compilar ese kernel. Si alguna de estas tres piezas (código del driver, *headers* y compilador) no encaja con las demás, el módulo no puede compilarse. Además, como el módulo resultante queda ligado a la versión concreta del kernel para la que se compiló, debe recompilarse cada vez que el kernel se actualiza. De esta tarea se encarga habitualmente DKMS (*Dynamic Kernel Module Support*), una herramienta que recompila de forma automática los módulos externos al kernel tras cada actualización.

El driver NVIDIA 340.108 fue diseñado contra kernels de la serie 3.x/4.x. El kernel por defecto de Proxmox VE 9.2 (serie 6.x/7.x, de 2025–2026) presenta una diferencia de aproximadamente siete años de evolución de la API interna del kernel (gestión de memoria, macros de compilación, etc). No es de extrañar que el código fuente del driver no compile sin más contra los *headers* modernos.

Esta distancia temporal es especialmente problemática en Linux, las distintas versiones de kernel no mantienen un patrón estable en la interfaz interna para los módulos, ni a nivel de código fuente (API) ni a nivel binario (ABI). Cuando esta interfaz cambia, los drivers incluidos en el propio kernel se adaptan a la vez, pero un driver externo como el de NVIDIA depende de que su fabricante —o, en su defecto, la comunidad— lo actualice. Sin esa actualización, cuanto mayor es la distancia entre versiones, más llamadas del driver dejan de existir o cambian de forma, lo que explica la gran cantidad de errores de compilación que se describen más adelante.

Una comparación con el capítulo 2 ayuda a situar el problema: en Ubuntu 18.04 (kernel 4.15, gcc 7.x, apenas dos años de diferencia respecto al driver), el paquete `nvidia-340` instalaba sin complicaciones porque se mantienen binarios y parches adaptados para ello de forma oficial. Para combinaciones de kernel tan recientes como las de este trabajo, **ningún repositorio oficial mantiene el driver 340.xx**: la tabla de plazos de soporte de NVIDIA declara expresamente que la serie 340.* es la última que da soporte a las GPUs G8x, G9x y GT2xx, que la compatibilidad con kernels Linux se cerró en la versión 340.108 y que no está prevista ninguna publicación posterior de la serie [32, **?**]. Por tanto, la compilación debe abordarse manualmente o mediante parches de terceros mantenidos por la comunidad en caso de haberlos.

4.3. Bitácora de resolución de problemas

A continuación se resumen, siguiendo a grandes rasgos el orden en que se presentaron, los obstáculos encontrados hasta conseguir que el driver funcionase. El objetivo no es reproducir cada orden ejecutada, sino explicar qué falló, por qué, y qué enseña cada problema sobre la viabilidad de este tipo de sistemas. Los pasos para reproducir la instalación se resumen en el apéndice A.

No todos los obstáculos tuvieron el mismo peso. La mayoría fueron incidencias menores de configuración, resueltas con rapidez, mientras que uno de ellos —el fallo estructural de la compilación manual— obligó a cambiar por completo de estrategia.

4.3.1. Preparación del host y del entorno de compilación

El primer paso fue liberar las GPUs de la configuración heredada del capítulo 3. Durante los experimentos con VFIO, las tarjetas habían quedado reservadas desde el arranque para `vfio-pci`, el módulo que las aparta del host para cedérselas a una máquina virtual, de modo que ningún driver del host podía tomar su control. Este paso puso de manifiesto que ambos enfoques son incompatibles por naturaleza: VFIO necesita reservar el hardware del host, mientras que LXC necesita que el host lo gobierne. Por ello, además de revertir la configuración para aislamiento, se diseñó el sistema de arranque dual descrito en la sección 2.2.3, que permite alternar limpiamente entre ambas configuraciones en cada inicio.

A continuación se abordó la elección del kernel. El que Proxmox VE 9.2 instala por defecto (versión 7.0) era demasiado moderno para el driver, por lo que se instaló y fijó como kernel de arranque una versión anterior también soportada por Proxmox, la 6.14. Este cambio, sin embargo, no fue suficiente: incluso con la versión 6.14 seguía sin haber paquetes precompilados ni parches oficiales, y la diferencia entre driver y kernel seguía siendo demasiado grande para compilar el driver sin modificar su código.

Durante esta preparación surgieron además dos incidencias menores. Por un lado, el instalador no encontraba los *headers* del kernel porque Proxmox VE 9 había cambiado el nombre del paquete que los contiene (de `pve-headers` a `proxmox-headers`), un ejemplo de cómo buena parte de la documentación disponible para software antiguo ha quedado desactualizada. Por otro, el instalador de NVIDIA detenía la instalación al detectar que el kernel se había compilado con gcc 12.2 y el sistema disponía de gcc 12.4. Aunque se trata de una diferencia menor que el propio kernel solo trata como un aviso, fue necesario indicar expresamente al driver que ignorase esta comprobación, tanto en el instalador como, más adelante, en la configuración de DKMS.

4.3.2. Intentos de compilación manual sin parches

Con los requisitos previos resueltos, se intentó compilar el driver manualmente y sin modificar su código, aun sabiendo que era un camino costoso en complejidad y en tiempo. Algunos de los errores pudieron resolverse ajustando directivas de compilación, pero, como se verá, resolverlos todos de esta forma no era viable ni habría dado lugar a una solución robusta.

Los primeros errores fueron de configuración del entorno: la convivencia de varias versiones de gcc y la modificación manual de las opciones de compilación impedían localizar algunos ficheros, tanto del compilador como del propio driver, y se resolvieron con ajustes puntuales. El siguiente, en cambio, fue estructural: incluso con los anteriores resueltos, la compilación terminó en cientos de errores encadenados como los siguientes:

```
error: unknown type name 'nv_state_t'
error: request for member 'flags' in something not a structure or union
```

Su origen está en el mecanismo con el que el driver se adapta a cada kernel. Como el kernel cambia de una versión a otra, el driver incluye un script llamado `conftest` que, antes de compilar, realiza pequeñas pruebas sobre los *headers* del kernel de destino para averiguar qué estructuras y funciones ofrece y qué versión de gcc emplea, y con ello genera un fichero de configuración (`conftest.h`) con las definiciones que necesita el resto del código. Al haber modificado manualmente las opciones de compilación, este mecanismo no llegó a ejecutarse correctamente, y el driver se compiló sin información sobre la configuración del kernel de destino. El error se repitió en todos los intentos de compilación manual.

Este fallo marcó el final de la vía manual. Con tanta distancia entre versiones, no basta con ajustar la compilación: el propio código del driver debe modificarse para adaptarse a los cambios internos del kernel. Por ello se recurrió a un conjunto de parches ya mantenido por la comunidad.

4.4. Los parches comunitarios y la solución con DKMS

Si en Ubuntu 18.04 el mismo driver se instalaba sin dificultad (capítulo 2), es porque la propia distribución mantenía parches para adaptarlo a su kernel. Para un kernel actual, ese trabajo lo realizan repositorios mantenidos por la comunidad.

4.4.1. De la compilación manual a DKMS

La incorporación de los parches comunitarios se realizó en dos intentos, que se diferencian en dos aspectos: el repositorio de parches empleado y la forma de compilar el driver. Distinguir ambos aspectos es clave para entender por qué el primero fracasó y el segundo no.

En el primer intento se utilizó el repositorio `dkosmari/nvidia-340.108-updated` [12], que reúne todas las modificaciones en un único parche. El parche se aplicó sin problemas sobre el código original del driver [37], de modo que el código ya quedaba adaptado al kernel moderno. Sin embargo, la compilación se siguió realizando igual que en la sección 4.3.2: invocando directamente la orden de compilación con opciones ajustadas manualmente. Como consecuencia, el mecanismo `conftest` volvió a no ejecutarse correctamente y la compilación falló por el mismo motivo. Este intento demostró que parchear el código era necesario, pero no suficiente: también había que cambiar la forma de compilarlo.

En el segundo intento se cambiaron ambos aspectos. Por un lado, la compilación se delegó en DKMS. Además de recompilar el módulo tras cada actualización del kernel, DKMS ejecuta el proceso de compilación tal y como lo define el propio driver en su fichero de configuración (`dkms.conf`), sin intervenir manualmente en las opciones de compilación. De este modo, el mecanismo `conftest` se ejecuta como el driver espera, que era precisamente lo que había fallado en todos los intentos anteriores. Por otro lado, se sustituyó el parche único por el repositorio `MichelBoucey/nvidia-340xx` [9], derivado de `archlinux-jerry/nvidia-340xx` [7] y mantenido activamente. Frente a un parche único y genérico, este repositorio ofrece parches específicos para versiones concretas del kernel, que cubren la 6.14 empleada en este trabajo, lo que ofrecía mayores garantías de compatibilidad con el entorno. Por ese motivo, no se llegó a probar el primer repositorio con DKMS. Además, el nuevo repositorio está preparado para integrarse con DKMS: uno de sus parches deja lista la configuración que este necesita, a la que solo hubo que añadir la indicación de ignorar la comprobación de versión del compilador descrita en la sección 4.3.1.

Estos parches son incrementales: cada uno adapta el driver a los cambios de una versión concreta del kernel (de la 5.7 a la 7.0) o del compilador (gcc 14 y 15), y se aplican en orden. En un primer momento se aplicaron los 21 parches del repositorio, incluidos los destinados a kernels posteriores a la 6.14, lo que produjo errores: esos parches adaptan el código a cambios que el kernel instalado todavía no incluye. Se rectificó aplicando solo los necesarios hasta la versión fijada, es decir, los 18 primeros, que cubren los kernels de la 5.7 a la 6.8 y las dos versiones de gcc. Estos se integraron sin ningún fallo.

La tabla 4.1 resume la evolución de los intentos de compilación descritos hasta aquí.

Gracias a DKMS, el módulo se recompilará automáticamente con cada actualización del

Intento	Código del driver	Compilación	Resultado
Sin parches (sección 4.3.2)	Original	Manual	Falla: código no adaptado y `conftest` sin ejecutar
Repositorio `dkosmari`	Parche único	Manual	Falla: `conftest` sin ejecutar
Repositorio `MichelBoucey`	Parches incrementales	DKMS	Éxito, tras corregir una opción de compilación

Cuadro 4.1: Evolución de los intentos de compilación del driver

kernel, en lugar de quedar como un binario congelado, válido solo para la versión actual, que habría que recompilar manualmente.

Por último, se instalaron los componentes del driver que no forman parte del kernel (librerías y herramientas como `nvidia-smi`), indicando al instalador oficial que no volviera a compilar el módulo. Durante la instalación, se descartaron las librerías de 32 bits al ser todo el software de este trabajo —toolkit CUDA, benchmarks y herramientas— de 64 bits: instalarlas solo añadiría ficheros muertos y posibles conflictos de multilib. También se descartó la configuración del entorno gráfico (`nvidia-xconfig`/X11) porque el anfitrión es un servidor sin servidor gráfico en ejecución.

4.5. Verificación y consideraciones sobre el módulo UVM

Ejecutamos `nvidia-smi` y comprobamos que el driver detecta ambas GPUs sin problemas. La instalación quedó además verificada tras un ciclo completo de arranque, confirmando la persistencia del módulo vía DKMS.

Un aspecto técnico relevante de esta fase es la decisión deliberada de no compilar el segundo módulo que suele acompañar al driver, `nvidia-uvm.ko`. Como se explicó en los preliminares, UVM (*Unified Virtual Memory*) es un sistema de memoria virtual unificada compartido entre CPU y GPU [**?**]. Sin UVM, la RAM y la VRAM son dos espacios completamente aislados: el programador debe reservar memoria por separado en cada uno de ellos y copiar explícitamente los datos de la CPU a la GPU y de vuelta. Con UVM, ambos espacios se perciben como uno solo, y basta con una única reserva. El intercambio de datos se resuelve mediante *page faulting* por hardware, de forma muy similar a la paginación de los sistemas operativos: cuando se accede a una dirección que no está cargada, el módulo `uvm` intercepta el fallo y trae la página correspondiente, consultando unas tablas que indican dónde se encuentra cada una (CPU, GPU o ambas).

Las GPUs de las que disponemos son previas a la introducción de UVM en el hardware,

por lo que todas las limitaciones descritas nos afectan de primera mano: al no estar soportada la memoria unificada por estas tarjetas. La decisión de no compilar el módulo `nvidia-uvm.ko` se validó empíricamente en el capítulo 6, cuando `deviceQuery` confirmó que ambas tarjetas reportan `Device supports Unified Addressing (UVA): No`.

4.6. Discusión

La tabla 4.2 resume los obstáculos de esta fase agrupados por categoría, de modo que pueda apreciarse la naturaleza del problema en conjunto.

Categoría	**Problemas agrupados**
Configuración del host	Reserva de las GPUs heredada de la configuración VFIO (1 problema)
Empaquetado / documentación	Cambio de nombre del paquete de *headers* (`pve-headers` → `proxmox-headers`) (1)
Obsolescencia del software *legacy*	Driver de 2019 frente a kernel de 2026 (1)
Configuración de *toolchain*	Discrepancia de versión de gcc entre kernel y sistema; varias versiones de gcc instaladas a la vez; opciones de compilación sobrescritas manualmente (3)
Metodología de compilación	Fallo de `conftest` al compilar manualmente (1)
Mantenimiento de terceros	Dependencia de parches comunitarios (dos repositorios); parche pensado para gcc 15 aplicado sobre gcc 12 (3)

Cuadro 4.2: Causas raíz de la fase de instalación del driver, agrupadas por categoría

El resultado más relevante de este capítulo es que la viabilidad de ejecutar hardware NVIDIA de la arquitectura Tesla (2008–2009) en un host con kernel y *toolchain* de 2025–2026 **depende enteramente de la existencia y mantenimiento activo de parches de terceros**, dado que NVIDIA no ofrece soporte oficial para esta combinación desde hace varios años. La reproducibilidad del proceso está, por tanto, sujeta a la disponibilidad continuada de dichos parches, un riesgo que debe tenerse en cuenta al valorar este tipo de despliegues.

Desde el punto de vista metodológico, el proceso confirma la importancia de utilizar *toolchains* gestionados (DKMS) frente a invocaciones manuales de `make`, y la conveniencia de fijar versiones de compilador mediante `update-alternatives` para evitar interacciones entre múltiples versiones de gcc. En términos cualitativos, la conclusión general de la

fase es doble: el **obstáculo dominante** no es el hardware (que funciona correctamente bajo el driver correcto), sino la **obsolescencia del software**, y el **factor habilitante** es la existencia de una comunidad que mantiene vivo el soporte de estos drivers. Sin esa comunidad, el aprovechamiento de estas tarjetas sería directamente inviable, lo que sitúa el mantenimiento comunitario como un criterio de viabilidad de primer orden en la evaluación de sistemas con hardware *legacy*. El procedimiento completo y reproducible, sin la traza de errores, se recoge en el apéndice A.

Con el driver funcionando en el host y ambas GPUs detectadas, el siguiente paso, que se aborda en el capítulo 5, es proporcionar acceso a las tarjetas a un contenedor LXC e instalar el stack de cómputo CUDA compatible.

Capítulo 5

Contenedor LXC con acceso a GPU y stack CUDA 6.5

Al final del capítulo 4, el driver 340.108 quedó instalado en el host y `nvidia-smi` detectaba ambas tarjetas. Sin embargo, que el host detecte las GPUs no significa que se pueda ejecutar código sobre ellas. Este capítulo completa el camino hasta el cómputo real: dar acceso a las tarjetas desde un contenedor LXC, instalar en él el software necesario para ejecutar programas CUDA y validar el conjunto con un programa real.

5.1. Las capas del software de cómputo

En los preliminares se distinguieron dos niveles en el software de NVIDIA: el módulo de kernel, que controla físicamente las tarjetas, y el stack *userspace*, formado por las librerías y herramientas que se ejecutan en espacio de usuario. Para ejecutar programas CUDA hace falta un tercer nivel, el *toolkit* de CUDA. En este trabajo se denomina **stack CUDA** al conjunto de los dos niveles que se ejecutan en espacio de usuario, es decir, el stack *userspace* del driver más el *toolkit*. Las tres capas quedan así:

1. **Módulo de kernel** (`nvidia.ko`): se carga en el host y es el único componente que depende de la versión del kernel. Su instalación fue el objeto del capítulo 4.

2. **Stack *userspace* del driver**: librerías como `libcuda` y herramientas como `nvidia-smi`. Ofrece la interfaz de bajo nivel de CUDA, la *Driver API*, que da acceso directo a los contextos y módulos de la GPU. Debe coincidir exactamente en versión con el módulo de kernel, ya que ambos se comunican directamente.

3. ***Toolkit* de CUDA**: el compilador `nvcc`, las librerías de alto nivel y los programas de ejemplo (*samples*). Cada versión del *toolkit* incluye su propio compilador, de modo que la versión de `nvcc` coincide siempre con la del *toolkit* instalado. Incluye

la *Runtime API*, que se implementa por encima de la *Driver API* y simplifica la programación a cambio de un menor control [25]. Su versión viene limitada por el hardware, no por el kernel.

Esta división explica el reparto de la figura 5.1. Solo la primera capa reside en el host; las otras dos se instalan dentro del contenedor, que accede a las tarjetas a través del módulo del host. Por ello, todo el trabajo de adaptación al kernel moderno quedó concentrado en el capítulo 4, y en este capítulo los problemas son de otra naturaleza: dar acceso a las GPUs desde el contenedor y conseguir instalar un *toolkit* de 2014 en un sistema actual.

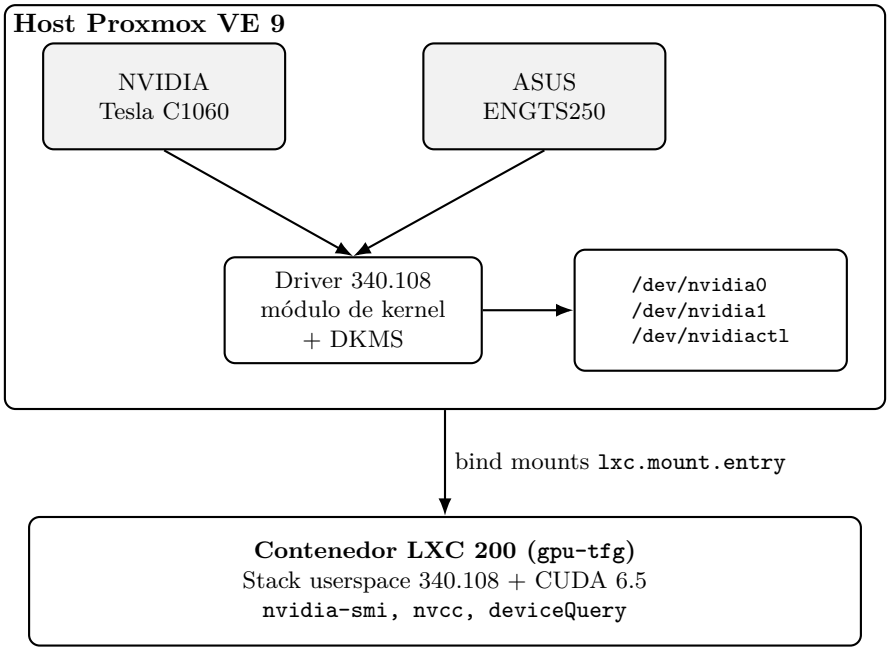


Figura 5.1: Arquitectura del sistema: host Proxmox con el driver, y contenedor LXC con acceso a ambas GPUs

5.2. Acceso a las GPUs desde el contenedor

El contenedor se creó a partir de una plantilla estándar de Debian 12 proporcionada por Proxmox. La única incidencia en este paso fue menor: el nombre exacto de la plantilla cambia con el tiempo según las versiones que Proxmox tenga publicadas, por lo que debe consultarse en el momento de la descarga en lugar de asumir uno fijo.

La decisión de diseño relevante fue crear un **contenedor privilegiado**. Los contenedores no privilegiados traducen los identificadores de usuario y de grupo entre el contenedor y el host [19, 20], lo que complica el acceso a los dispositivos de NVIDIA, ya que habría que traducir también los permisos de esos dispositivos. La vía privilegiada es más simple y directa, a costa de un aislamiento de seguridad menor, una contrapartida razonable en un entorno puramente experimental como el de este trabajo.

En Linux, los programas se comunican con el driver a través de unos ficheros especiales del directorio `/dev`, los *device nodes*. El driver de NVIDIA crea uno por cada GPU (`/dev/nvidia0` y `/dev/nvidia1`) y uno de control compartido (`/dev/nvidiactl`), todos ellos identificados por el mismo número de dispositivo, el 195. Como el contenedor comparte el kernel del host, darle acceso a las GPUs consiste únicamente en dos pasos: autorizarle a usar los dispositivos con ese número y hacer visibles dentro de él esos tres ficheros. La tabla 5.1 recoge la correspondencia entre ficheros y tarjetas. No existe un fichero adicional para la memoria unificada, puesto que el módulo `nvidia-uvm.ko` no se compiló (capítulo 4).

Device node	**GPU**	**Bus PCI**
`/dev/nvidia0`	Tesla C1060	`0000:01:00.0`
`/dev/nvidia1`	GeForce GTS 250 (ENGTS250)	`0000:08:00.0`
`/dev/nvidiactl`	(control, compartido)	—

Cuadro 5.1: Correspondencia entre *device nodes* y GPUs en el host

Con esta configuración, un mismo contenedor accede a ambas GPUs a la vez. Si se quisiera aislar cada tarjeta en su propio contenedor, por ejemplo para comparar su rendimiento por separado, bastaría con hacer visible en cada uno de ellos solo el fichero de la GPU correspondiente. Tras arrancar el contenedor se comprobó que se tenía visión de los tres ficheros desde el interior del mismo. La configuración exacta se resume en el apéndice A.

5.3. Instalación del stack CUDA

5.3.1. Stack *userspace* del driver

A diferencia del módulo de kernel, el stack *userspace* es un binario precompilado por NVIDIA que no depende de la versión del kernel, por lo que no necesitó ninguno de los parches del capítulo 4. Dentro del contenedor se utilizó el instalador oficial sin modificar, indicándole únicamente que no instalase el módulo de kernel, que ya proporciona el host. Al igual que en el host, se omitieron las librerías de 32 bits y la configuración del entorno gráfico. Tras la instalación, `nvidia-smi` mostró ambas GPUs desde dentro del contenedor con la misma versión de driver que el host.

5.3.2. *Toolkit* de CUDA 6.5

La versión del *toolkit* viene impuesta por el hardware. Las GPUs del proyecto tienen *compute capability* 1.3 (Tesla C1060) y 1.1 (GTS 250), y CUDA 6.5 es la última versión del *toolkit* que da soporte a *compute capabilities* inferiores a 2.0 [27]. Cualquier versión

posterior es, por tanto, incompatible con estas tarjetas. La descarga se realizó desde el archivo histórico de NVIDIA [36].

La instalación de un *toolkit* de 2014 en un sistema actual planteó dos obstáculos. El primero fue menor: el instalador rechazaba el compilador del sistema, ya que CUDA 6.5 solo declara soporte hasta gcc 4.8 y Debian 12 incluye gcc 12. Bastó con indicarle que omitiese esta comprobación, el mismo tipo de incompatibilidad por antigüedad ya encontrado con el driver en el capítulo 4.

El segundo obstáculo fue más relevante, porque el fallo no era visible. El instalador terminó sin mostrar ningún error, pero el *toolkit* no había quedado instalado. Solo el registro de la instalación revelaba la causa real:

```
Can't locate InstallUtils.pm in @INC (...) at ./install-linux.pl line 6.
BEGIN failed--compilation aborted at ./install-linux.pl line 6.
```

Para entenderlo hay que saber cómo está construido el instalador. El fichero que se descarga es en realidad un *meta-instalador*: un paquete que contiene a su vez tres instaladores independientes (driver, *toolkit* y ejemplos), cada uno de los cuales se descomprime y se ejecuta automáticamente en un directorio temporal. Al ejecutarse de esta forma encadenada, el instalador del *toolkit* no localizaba uno de sus propios componentes internos y abortaba, sin que el error llegase a mostrarse al usuario.

La solución fue prescindir de la ejecución automática y desmontar el instalador por capas: primero se extrajeron los tres instaladores del paquete; después se extrajo el del *toolkit* a un directorio fijo, sin ejecutarlo; y, por último, se lanzó su instalación manualmente desde ese directorio, indicándole explícitamente dónde encontrar sus componentes. Los ejemplos (*samples*) se instalaron siguiendo el mismo patrón, copiando directamente sus ficheros, ya que el script auxiliar previsto para ello asumía una estructura de directorios distinta a la real del paquete. Por último, se configuraron las variables de entorno del sistema para que el compilador `nvcc` y las librerías de CUDA fuesen accesibles, y `nvcc` confirmó la versión 6.5 instalada. El procedimiento se resume en el apéndice A.

5.4. Validación del cómputo real

`nvidia-smi` solo consulta al driver qué tarjetas hay; no ejecuta ningún código en ellas. La prueba definitiva de que las tres capas funcionan juntas es ejecutar un programa CUDA real. Para ello se utilizó `deviceQuery`, uno de los ejemplos del *toolkit*, que recorre la cadena completa: el programa usa la *Runtime API* del *toolkit*, esta se apoya en la *Driver API* del stack *userspace*, y esta, a su vez, se comunica con el módulo de kernel del host. Por eso, este programa valida también que la versión de las librerías del contenedor es compatible con

la del módulo del host, algo que `nvidia-smi` no puede garantizar por sí solo. El resultado fue el siguiente:

```
Detected 2 CUDA Capable device(s)
deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 6.5, CUDA Runtime
    Version = 6.5,
NumDevs = 2, Device0 = Tesla C1060, Device1 = GeForce GTS 250
Result = PASS
```

El resultado `Result = PASS` confirma que el cómputo CUDA funciona de extremo a extremo dentro del contenedor, lo que cierra el objetivo técnico principal de esta fase. La salida de `deviceQuery` confirma además que ninguna de las dos tarjetas admite direccionamiento unificado (`Device supports Unified Addressing (UVA): No`), lo que valida con datos reales la decisión, tomada en el capítulo 4, de no compilar el módulo `nvidia-uvm.ko`. La salida completa se proporciona como material complementario (apéndice A).

5.5. Valoración de la fase

Esta fase aporta dos resultados de interés para el objetivo general del trabajo. En primer lugar, demuestra que **dar acceso a las GPUs desde un contenedor es notablemente más simple que hacerlo desde una máquina virtual**: toda la complejidad del capítulo 3 (IOMMU, grupos, reinicios) desaparece, y el coste se traslada a la instalación del driver en el host, que ya quedó resuelta en el capítulo 4. En segundo lugar, la experiencia con CUDA 6.5 confirma un patrón recurrente en este trabajo: **los instaladores antiguos pueden fallar de forma silenciosa en entornos modernos**, y la vía más robusta es desmontarlos y ejecutarlos manualmente por capas. Este patrón, que se repetirá en el capítulo 6 con el compilador `nvcc`, es una lección transferible a cualquier proyecto que reutilice software *legacy*: los procesos de instalación automáticos de hace más de una década no pueden darse por supuestos.

Con el entorno de cómputo plenamente operativo, el siguiente capítulo aborda la experimentación: pruebas de rendimiento, ejecución concurrente y análisis de las limitaciones del hardware.

Capítulo 6

Experimentación y resultados

Con el entorno de cómputo operativo, este capítulo evalúa el sistema en funcionamiento. Se mide el rendimiento de cómputo y de transferencia de ambas GPUs, se comparan los resultados con los máximos teóricos de cada tarjeta y de cada configuración del bus PCIe, se estudia la ejecución simultánea en las dos tarjetas y se documentan dos limitaciones encontradas durante las pruebas: una condición de carrera en el driver y la ausencia de telemetría de consumo. El capítulo termina valorando, a partir de estos datos, en qué condiciones compensa el reaprovechamiento.

6.1. Metodología de evaluación

Como herramientas de evaluación se utilizaron dos ejemplos del *toolkit* de CUDA 6.5, ampliamente empleados como referencia para este tipo de medidas [35]:

- `matrixMul`: multiplica dos matrices en la GPU y mide el rendimiento de cómputo en GFLOP/s. Con la configuración por defecto para estas tarjetas, multiplica una matriz de 160×160 por otra de 160×320 y repite el cálculo 300 veces sobre los mismos datos. El tiempo y el rendimiento que reporta corresponden **únicamente a la ejecución en la GPU**: los datos se copian una sola vez antes de empezar y no se incluyen en la medida. Por tanto, su resultado no depende del bus PCIe.
- `bandwidthTest`: mide el ancho de banda de tres tipos de transferencia. Para cada una, copia repetidamente un bloque de datos de unos 32 MB y divide el volumen total copiado entre el tiempo empleado:
  - **Host to Device (H2D) o Device to Host(D2H):** copias entre la memoria del host y la de la GPU, en uno y otro sentido. Atraviesan el bus PCIe, por lo que miden el ancho de banda del enlace. Se realizaron con memoria del host fijada (*pinned*), que permite transferencias directas sin copias intermedias.

- **Device to Device (D2D):** Mide la velocidad de copia directa entre dos regiones de la memoria de la tarjeta (VRAM). Al ejecutarse exclusivamente dentro de la propia GPU, la transferencia no atraviesa el bus PCIe, aislando por completo el rendimiento de su subsistema de memoria. Esta métrica es fundamental, ya que los *kernels* leen y escriben datos continuamente en VRAM durante el cálculo, convirtiendo este ancho de banda en el principal cuello de botella para algoritmos con alta intensidad de datos. Dado que cada byte copiado implica tanto una lectura como una escritura, `bandwidthTest` suma ambas operaciones, permitiendo comparar directamente el resultado empírico con el ancho de banda teórico de la tarjeta.

Para interpretar los resultados, cada medida se compara con su máximo teórico:

1. el rendimiento pico de cálculo, para `matrixMul`;
2. el máximo del enlace PCIe, para las transferencias entre host y GPU;
3. el máximo de la memoria de la GPU, para las transferencias internas.

El cálculo de cada máximo se detalla junto a los resultados correspondientes. `bandwidthTest` expresa sus resultados en MB/s; en las comparaciones se toma 1 MB = 10^6 bytes, por lo que los porcentajes deben entenderse como aproximados.

Todas las pruebas se realizaron con el enlace de ambas ranuras fijado a PCIe Gen 1 desde la BIOS, para garantizar que las tarjetas cumplieran los tiempos de negociación del enlace (capítulo 2). Por este motivo no se realizaron mediciones en Gen 2, la generación máxima que admiten ambas tarjetas: forzarla habría comprometido la fiabilidad de la conexión, por lo que sus valores se incluyen únicamente como máximos teóricos. La Tesla C1060 ocupa la ranura principal, conectada directamente a la CPU, y la GTS 250 la secundaria, que opera eléctricamente a cuatro carriles (x4) a través del *chipset* (sección 1.3). El enlace realmente negociado en cada ranura se verificó con `lspci`, como se detalla en la sección 6.3.2.

6.2. Compilación de los benchmarks

A diferencia de `deviceQuery`, que solo contiene código para la CPU, `matrixMul` y `bandwidthTest` incluyen código que se ejecuta en la GPU y que debe compilarse con `nvcc`, el compilador de CUDA. El sistema dispone de `nvcc` 6.5, la versión incluida en el *toolkit* instalado en el capítulo 4, ya que cada versión incorpora su propio compilador.

Para entender el problema que surgió conviene saber cómo trabaja este compilador. `nvcc` no lo compila todo por sí mismo: primero analiza el fichero fuente completo para separar el código destinado a la GPU del código de la CPU (que delega en el compilador del

sistema, `gcc`). Durante esa primera lectura, `nvcc` debe preprocesar también las **cabeceras del sistema y de la biblioteca estándar de C++** (archivos como `<iostream>` o `<cuda_runtime.h>` que contienen las declaraciones y tipos fundamentales necesarios para compilar). Dado que estas cabeceras no forman parte de CUDA sino de la versión de `gcc` instalada en el sistema operativo, cualquier incompatibilidad sintáctica entre una versión moderna de `gcc` y el analizador de un `nvcc` antiguo provocará que la compilación falle antes de empezar.

En la comprobación de estas cabeceras se encontró el primer fallo. La compilación en el contenedor terminó con errores como los siguientes:

```
error: identifier "nullptr" is undefined
error: variable "std::constexpr" is not a type name
```

`nullptr` y `constexpr` son palabras clave introducidas por el estándar C++11 (2011). Los estándares de C++ son acumulativos: cada versión posterior (C++14, C++17...) incluye todo lo introducido en las anteriores, por lo que cualquier cabecera escrita para C++11 o un estándar más reciente puede contener estas palabras. Las cabeceras de gcc 12, incluidas en Debian 12, las utilizan con normalidad, pero el analizador de `nvcc` 6.5 se diseñó antes de que ese estándar se generalizase y no las reconoce. Es decir, el problema no está en el código de los ejemplos, sino en que `nvcc` 6.5 no es capaz de leer las cabeceras de un sistema actual. Por la misma razón, desactivar la comprobación de versión del compilador, como se hizo durante la instalación del *toolkit* en el capítulo 5, no sirve aquí: solo suprime el aviso, pero las cabeceras siguen siendo ilegibles para `nvcc`.

La solución, por tanto, era compilar con un gcc y unas cabeceras de la misma época que CUDA 6.5. Los repositorios de Debian 12 no ofrecen versiones de gcc tan antiguas y, aunque se instalase una por otra vía, las cabeceras de la biblioteca de C del sistema seguirían siendo modernas: hacía falta un entorno antiguo completo. Para ello se creó, dentro del contenedor, un entorno *chroot*: un directorio que contiene una instalación mínima de otro sistema operativo y que los programas ejecutados en su interior perciben como si fuese el sistema completo. Se eligió Debian 8 (2015). Lo determinante no es la versión de Debian en sí, sino el estándar de C++ que su gcc aplica por defecto, ya que es el que condiciona qué partes de sus cabeceras se activan. La tabla 6.1 resume esta relación: Debian 8 es la versión más reciente cuyo gcc aplica por defecto un estándar anterior a C++11, mientras que desde Debian 9 el estándar por defecto ya incluye C++11 y, con alta probabilidad, reproduciría el mismo problema. Aunque CUDA 6.5 solo declara soporte oficial hasta gcc 4.8, la versión 4.9 resultó compatible en la práctica una vez desactivada la comprobación de versión. Además, los repositorios de Debian 8 siguen disponibles en el archivo histórico de Debian [10]. En ese entorno se hizo visible el *toolkit* ya instalado y se compilaron los ejemplos, limitando el código generado a las dos arquitecturas reales del hardware (`sm_11`

y `sm_13`) en lugar de las seis o siete que genera por defecto el `Makefile` de los ejemplos.

Versión de Debian	**gcc**	**Estándar de C++ por defecto**	**Cabeceras legibles por `nvcc` 6.5**
Debian 8 (2015)	4.9	C++98, anterior a C++11	Sí (verificado)
Debian 9 (2017)	6	C++14, que incluye C++11	No (previsiblemente)
Debian 12 (2023)	12	C++17, que incluye C++11	No (verificado)

Cuadro 6.1: Estándar de C++ aplicado por defecto por el gcc de cada versión de Debian

El entorno de Debian 8 se utilizó **únicamente para compilar**. Los programas resultantes se ejecutaron sin ningún problema directamente en el contenedor principal de Debian 12. Esto es posible porque un programa compilado contra bibliotecas antiguas sigue funcionando sobre versiones más modernas de esas bibliotecas, pero no al revés. Este resultado confirma, además, que la incompatibilidad se limitaba a la discrepancia en la declaración de las cabeceras pero no a nivel de binario. El procedimiento se resume en el apéndice A.

6.3. Resultados de rendimiento

6.3.1. Cómputo: `matrixMul`

El rendimiento que reporta `matrixMul` corresponde al total de operaciones en coma flotante realizadas dividido entre el tiempo efectivo de cómputo en la GPU:

$$R = \frac{N_{\text{iter}} \cdot (2 \cdot M \cdot K \cdot N)}{T_{\text{GPU}} \cdot 10^9} \quad [\text{GFLOP/s}] \tag{6.1}$$

donde:

- $M \times K$ y $K \times N$: dimensiones de las dos matrices que se multiplican (por defecto, los tamaños en `matrixMul` son: $M = 160$, $K = 160$ y $N = 320$). El número de columnas de la primera coincide con el de filas de la segunda (K).
- $2 \cdot M \cdot K \cdot N$: operaciones en punto flotante por multiplicación. La matriz resultante contiene $M \cdot N$ elementos; cada uno requiere K multiplicaciones y K sumas.
- N_{iter}: número de repeticiones de la prueba (300).
- T_{GPU}: tiempo total de ejecución en la GPU (en segundos), excluyendo transferencias por el bus PCIe.

- 10^9: factor de escala para expresar el resultado en GFLOP/s (10^9 FLOP/s = 1 GFLOP/s).

Sustituyendo los valores fijos de la prueba, el numerador equivale exactamente a $300 \cdot (2 \cdot 160 \cdot 160 \cdot 320) = 4{,}9152 \cdot 10^9$ operaciones, de modo que la expresión se reduce a $R = 4{,}9152 / T_{\text{GPU}}$ (con T_{GPU} en segundos).

El programa reporta el tiempo medio por multiplicación, el cual, multiplicado por las 300 repeticiones, proporciona T_{GPU}:

- **Tesla C1060:** $300 \times 0{,}106$ ms $= 31{,}8$ ms $= 0{,}0318$ s $\implies R \approx 155$ GFLOP/s
- **GTS 250:** $300 \times 0{,}204$ ms $= 61{,}2$ ms $= 0{,}0612$ s $\implies R \approx 80$ GFLOP/s

El máximo teórico, o rendimiento pico, se obtiene a partir de las especificaciones de cada tarjeta:

$$P_{\text{pico}} = N_{\text{núcleos}} \times f \times 2 \tag{6.2}$$

donde:

- $N_{\text{núcleos}}$: número de núcleos CUDA (240 en la Tesla y 128 en la GTS 250).
- f: frecuencia de los núcleos (1.296 GHz en la Tesla y 1.62 GHz en la GTS 250).
- El factor 2: cada núcleo puede completar en cada ciclo una multiplicación y una suma combinadas, es decir, dos operaciones.

Para la Tesla, 240 núcleos $\times$ 1,296 GHz $\times$ 2 $\approx$ 622 GFLOP/s, y para la GTS 250, 128 núcleos $\times$ 1,62 GHz $\times$ 2 $\approx$ 415 GFLOP/s. La tabla 6.2 recoge ambos valores junto con el porcentaje del pico alcanzado, que es el cociente entre los dos (por ejemplo, $155{,}26/622 \approx 0{,}25$ para la Tesla). Como `matrixMul` no mide transferencias por el bus, este resultado es el mismo con independencia de la generación PCIe o de la ranura que ocupe cada tarjeta.

GPU	**Medido**	**Tiempo**	**Pico teórico**	**% del pico**
Tesla C1060	155.26 GFLOP/s	0.106 ms	622 GFLOP/s	≈25 %
GeForce GTS 250	80.49 GFLOP/s	0.204 ms	415 GFLOP/s	≈19 %

Cuadro 6.2: Resultados de `matrixMul` frente al rendimiento pico teórico. El tiempo indicado es el de una multiplicación.

Ambas tarjetas quedan lejos de su pico. Es lo esperable: `matrixMul` es un ejemplo didáctico, no una implementación optimizada, y las matrices empleadas son pequeñas. Lo relevante para este trabajo es la comparación entre ambas tarjetas, que se analiza en la sección 6.3.4.

Una repetición posterior de la prueba en la Tesla C1060 dio 145.16 GFLOP/s, un 6.5 % menos que la primera medida. Como `matrixMul` no depende del bus, esta diferencia no se debe a la ranura, sino a la variabilidad normal entre ejecuciones. Los resultados de este capítulo proceden de ejecuciones individuales, por lo que las diferencias de unos pocos puntos porcentuales deben interpretarse con cautela. Repetir cada prueba un número suficiente de veces y expresar los resultados como una media con su intervalo de confianza permitiría cuantificar esta variabilidad; pero el deterioro del hardware descrito en la sección 6.6 no permitió abordarlo en este trabajo, por lo que queda como mejora para trabajos futuros.

6.3.2. Transferencias entre host y GPU

Esta sección analiza las transferencias que atraviesan el bus PCIe (H2D y D2H). Para interpretarlas es necesario conocer primero el enlace con el que trabajó realmente cada tarjeta, que no tiene por qué coincidir con el máximo de la ranura, y calcular a partir de él su máximo teórico.

Enlace negociado por cada tarjeta. La velocidad de cada carril se expresa en GT/s (gigatransferencias por segundo), lo que equivale a miles de millones de bits brutos transmitidos por segundo en la línea física. Según la información del bus obtenida con `lspci`, durante las pruebas los enlaces fueron los siguientes:

- Tesla C1060, en la ranura principal: x8 a 2.5 GT/s, aunque tanto la ranura como la tarjeta admiten x16.
- GTS 250, en la ranura secundaria: x4 a 2.5 GT/s, el máximo que permite esa ranura, que solo dispone de cuatro carriles procedentes del *chipset*.

Por qué la Tesla funcionó a x8. El enlace x8 de la Tesla tiene dos causas superpuestas. La primera es la configuración de la BIOS durante las pruebas: la ranura principal estaba en modo de bifurcación x8/x8, que divide sus dieciséis carriles en dos enlaces x8 independientes, pensados para tarjetas adaptadoras de unidades M.2 [8], y una GPU solo puede utilizar uno de ellos. Este modo se eligió porque, durante la configuración inicial, se observó que con la configuración automática la ranura secundaria no resultaba operativa, y que este modo lo resolvía (capítulo 2).

La causa de ese comportamiento no está documentada por el fabricante ni se ha podido determinar. No se explica por un reparto de carriles, ya que la ranura secundaria depende del *chipset* y no de los carriles que reparte la bifurcación, que son los de la CPU. Tampoco parece deberse a una falta de espacio de direcciones para las tarjetas: los BAR (*Base*

Address Registers, los registros con los que la CPU mapea la memoria del dispositivo) de la Tesla C1060 ocupan en total unos 112 MB y se asignan sin errores. La explicación más probable es que el modo x8/x8 prolongue la inicialización de los puertos PCIe durante el arranque. Tras cada reinicio, los dispositivos PCIe disponen de un plazo para estar listos antes de responder a las consultas del sistema, que además pueden prolongar si necesitan más tiempo, como recoge el propio código del kernel de Linux [46], y una tarjeta antigua puede necesitar más tiempo del habitual; una inicialización más larga le daría ese tiempo adicional antes de que la BIOS compruebe qué dispositivos hay conectados. Esta hipótesis es coherente con los problemas de negociación del enlace que motivaron el forzado a Gen 1, aunque no ha podido verificarse.

La segunda causa se descubrió al volver posteriormente a la configuración automática: aunque el puerto de la CPU pasa entonces a ofrecer x16, la Tesla sigue conectándose a x8. Al establecerse un enlace, la tarjeta y el puerto comprueban el funcionamiento de cada carril y, si alguno falla, reducen el enlace al siguiente ancho estándar (x16, x8, x4...) en el que todos los carriles utilizados funcionan. Que la Tesla no recupere x16 indica, por tanto, que parte de sus carriles no funcionan correctamente, por un problema físico en los contactos de la tarjeta o de la ranura cuya localización exacta queda pendiente. Cada enlace se establece de forma independiente: el x4 de la GTS 250 no está relacionado con este problema, sino que se debe a que su ranura solo dispone de cuatro carriles.

Máximo teórico de cada enlace. El máximo teórico de cada enlace depende de su generación y de su número de carriles (sección 1.3):

$$AB_{\text{enlace}} = \frac{v \times c}{8} \times N_{\text{carriles}} \tag{6.3}$$

donde:

- AB_{enlace}: ancho de banda máximo del enlace, en cada sentido.
- v: velocidad de señal de cada carril: 2.5 GT/s en Gen 1, es decir, $2{,}5 \cdot 10^9$ bit/s, y el doble en Gen 2.
- c: fracción de los bits transmitidos que transporta datos. Gen 1 y Gen 2 emplean la codificación 8b/10b, en la que de cada 10 bits transmitidos 2 se dedican a codificar la señal, por lo que $c = 8/10$.
- El divisor 8: convierte bits en bytes.
- N_{carriles}: número de carriles del enlace (8 en la Tesla y 4 en la GTS 250).

Cada carril en Gen 1 ofrece, por tanto, $(2{,}5 \cdot 10^9 \times 0{,}8)/8 = 250$ MB/s por sentido. Así, el enlace x8 de la Tesla ofrece $8 \times 250 = 2000$ MB/s, y el x4 de la GTS 250,

$4 \times 250 = 1000$ MB/s. Gen 2, la generación máxima que admiten ambas tarjetas, duplicaría la velocidad de señal con la misma codificación y, con ella, estos valores.

Resultados. La tabla 6.3 compara estos máximos con el ancho de banda medido por `bandwidthTest`. El porcentaje indicado entre paréntesis es el cociente entre ambos; por ejemplo, 1603,3/2000 ≈ 0,80 para la Tesla en H2D.

GPU (ranura)	**Enlace**	**Máximo teórico**	**H2D**	**D2H**
Tesla C1060 (principal)	Gen 1 x8	2000 MB/s	1603.3 MB/s (≈80 %)	1635.6 MB/s (≈82 %)
GTS 250 (secundaria)	Gen 1 x4	1000 MB/s	778.0 MB/s (≈78 %)	817.6 MB/s (≈82 %)

Cuadro 6.3: Ancho de banda medido entre host y GPU frente al máximo teórico del enlace negociado, por sentido (`bandwidthTest`, memoria *pinned*)

Las dos tarjetas aprovechan en torno al 80 % de su enlace, un valor coherente con la eficiencia habitual del protocolo PCIe ($\eta \approx 0{,}75$–$0{,}8$): ambos enlaces están, en la práctica, saturados. Por tanto, la diferencia de rendimiento entre ambas se debe únicamente al ancho del enlace: la Tesla dispone del doble de carriles y transfiere el doble de datos.

La misma tarjeta en ambas ranuras. Para confirmar que la diferencia se debe a la ranura y no a la tarjeta, se repitió la prueba con la Tesla C1060 instalada en la ranura secundaria, la misma que ocupaba la GTS 250. La tabla 6.4 recoge el resultado.

Tarjeta y ranura	**Enlace**	**H2D**	**D2H**	**D2D**
Tesla C1060, principal	Gen 1 x8	1603.3 MB/s	1635.6 MB/s	75 661.9 MB/s
Tesla C1060, secundaria	Gen 1 x4	777.5 MB/s	815.0 MB/s	75 117 MB/s
GTS 250, secundaria	Gen 1 x4	778.0 MB/s	817.6 MB/s	54 603.4 MB/s

Cuadro 6.4: Resultados de `bandwidthTest` con la Tesla C1060 en cada ranura, frente a la GTS 250

En la ranura secundaria, la Tesla obtiene en las transferencias por el bus prácticamente los mismos valores que la GTS 250 (777.5 frente a 778.0 MB/s): con el mismo enlace, ambas tarjetas rinden igual. Al pasar de la ranura secundaria a la principal, la Tesla duplica su ancho de banda, lo mismo que se duplica el número de carriles. Las transferencias internas (D2D), en cambio, apenas varían al cambiar de ranura, como corresponde a una medida que depende de la memoria de la GPU y no del bus.

Margen de mejora según la configuración. La tabla 6.5 completa el análisis con el máximo teórico de cada ranura según su ancho y su generación, tanto para las tarjetas del

caso de estudio como para una GPU moderna capaz de aprovechar la generación máxima de cada ranura. Como ambas tarjetas son PCIe 2.0 x16 nativas, en las dos primeras columnas el máximo depende solo de la ranura y de su configuración, no de la tarjeta que la ocupe.

Ranura y ancho del enlace	**Gen 1**	**Gen 2**	**Máximo de la ranura (GPU moderna)**
Secundaria (*chipset*), x4	**1.0 GB/s***	2.0 GB/s	≈7.9 GB/s (PCIe 4.0)
Principal (CPU), x8	**2.0 GB/s***	4.0 GB/s	≈31.5 GB/s (PCIe 5.0)
Principal (CPU), x16	4.0 GB/s	8.0 GB/s	≈63.0 GB/s (PCIe 5.0)

Cuadro 6.5: Máximo teórico por sentido del enlace PCIe de cada ranura según su ancho y su generación. Gen 2 es la generación máxima de las tarjetas del caso de estudio; la última columna corresponde a la generación máxima de cada ranura de la placa, con la codificación 128b/130b de PCIe 4.0 y 5.0. Los valores marcados con * corresponden a la configuración de las pruebas.

Cada tarjeta dispone del valor de la fila correspondiente a la ranura que ocupa y de la columna de la generación configurada. En las pruebas, la GTS 250 disponía de 1.0 GB/s (ranura secundaria, Gen 1) y la Tesla, de 2.0 GB/s (ranura principal a x8, Gen 1). La tabla permite separar los dos factores que determinan el ancho de banda de cada GPU:

- **La generación**: pasar de Gen 1 a Gen 2 duplicaría el ancho de banda de ambas ranuras sin cambiar ningún componente, ya que las dos tarjetas admiten Gen 2. Si no se hizo fue para garantizar que cumplieran los tiempos de negociación del enlace: es una pérdida aceptada a cambio de fiabilidad, y la primera que convendría recuperar si la conexión en Gen 2 resultase estable. Más allá de Gen 2 no hay margen con estas tarjetas: la ranura principal admite PCIe 5.0, pero ellas no pueden aprovecharlo.

- **Los carriles**: a diferencia de la generación, el número de carriles no puede aumentarse, porque lo fija la placa y debe repartirse entre las ranuras. La ranura principal dispone de los dieciséis carriles de la CPU, mientras que la secundaria solo cuenta con cuatro, procedentes del *chipset*. Existe, por tanto, un compromiso entre el número de GPUs y el ancho de banda de cada una: cada GPU adicional obliga a repartir los carriles disponibles o a relegarla a una ranura del *chipset*, con un enlace más estrecho y compartido. Este es el límite verdaderamente estructural de una plataforma de consumo (sección 1.3).

- **La comparación con una GPU moderna**: la última columna muestra que, con

hardware de esta generación, la plataforma queda muy infrautilizada. Una GPU actual en la ranura principal a x16 dispondría de unos 63 GB/s, unas treinta veces más que la Tesla en las pruebas.

Todos los valores de la tabla son cotas superiores: a juzgar por las pruebas, el ancho de banda real se situaría en torno al 80 % de cada uno.

6.3.3. Transferencias internas en la GPU

La transferencia D2D mide el ancho de banda de la memoria de cada tarjeta. Su máximo teórico se obtiene a partir de los datos de la memoria que reporta `deviceQuery`:

$$AB_{\text{VRAM}} = f_{\text{mem}} \times 2 \times \frac{B}{8} \tag{6.4}$$

donde:

- AB_{VRAM}: ancho de banda máximo de la memoria de la GPU.
- f_{mem}: frecuencia de la memoria (800 MHz en la Tesla y 1000 MHz en la GTS 250).
- El factor 2: la memoria GDDR3 realiza dos transferencias por ciclo de reloj.
- B: ancho del bus de memoria, en bits (512 en la Tesla y 256 en la GTS 250); al dividirlo entre 8 se obtienen los bytes que se mueven en cada transferencia.

Para la Tesla, 800 MHz$\times$2$\times$64 B = 102,4 GB/s, y para la GTS 250, 1000 MHz$\times$2$\times$32 B = 64,0 GB/s. La tabla 6.6 compara estos valores con los medidos:

GPU	**Memoria**	**Máximo teórico**	**Medido**
Tesla C1060	512 bits, 800 MHz	102.4 GB/s	75.66 GB/s (≈74 %)
GeForce GTS 250	256 bits, 1000 MHz	64.0 GB/s	54.60 GB/s (≈85 %)

Cuadro 6.6: Ancho de banda de memoria interna medido frente al máximo teórico (`bandwidthTest`, D2D)

6.3.4. Comparativa entre ambas tarjetas

Esta sección compara ambas tarjetas métrica a métrica. En cada tabla se recoge el valor de cada tarjeta y el cociente entre ambos, que se obtiene dividiendo el valor de la Tesla entre el de la GTS 250. Los valores proceden de las tablas anteriores de este capítulo y de la tabla 1 de los preliminares. Como se verá, solo las transferencias por el bus se ajustan a la predicción teórica; en el resto, los cocientes medidos se desvían de ella, y cada desviación tiene una explicación distinta.

Métrica	Tesla C1060	GTS 250	Cociente
Núcleos CUDA	240	128	1.875
Frecuencia de los núcleos	1296 MHz	1620 MHz	0.80
Rendimiento, teórico	622 GFLOP/s	415 GFLOP/s	1.50
Rendimiento, medido	155.26 GFLOP/s	80.49 GFLOP/s	1.93

Cuadro 6.7: Comparativa de cómputo entre la Tesla C1060 y la GTS 250

Cómputo. Aunque la GTS 250 tiene menos núcleos, estos funcionan a mayor frecuencia, por lo que el cociente teórico es de solo 1.5×. La Tesla obtiene, sin embargo, 1.93×. La explicación más probable está en las diferencias entre ambos chips: según `deviceQuery`, la Tesla (GT200) dispone del doble de registros por bloque (16 384 frente a 8192) y admite más hilos por multiprocesador (1024 frente a 768), lo que le permite mantener ocupados más núcleos en este tipo de cálculo.

Métrica	Tesla C1060	GTS 250	Cociente
Capacidad	4096 MB	1024 MB	4.00
Ancho del bus de memoria	512 bits	256 bits	2.00
Frecuencia de la memoria	800 MHz	1000 MHz	0.80
Ancho de banda, teórico	102.4 GB/s	64.0 GB/s	1.60
Ancho de banda, medido	75.66 GB/s	54.60 GB/s	1.39

Cuadro 6.8: Comparativa de la memoria interna entre la Tesla C1060 y la GTS 250

Memoria interna. La Tesla tiene el doble de ancho de bus, pero la memoria de la GTS 250 funciona a mayor frecuencia, por lo que el cociente teórico es de 1.6×, no de 2×. El medido es algo menor (1.39×) porque la GTS 250 aprovecha mejor su memoria (en torno al 85 % de su máximo, frente al 74 % de la Tesla).

Métrica	Tesla C1060	GTS 250	Cociente
Carriles del enlace	8	4	2.00
H2D, teórico	2000 MB/s	1000 MB/s	2.00
H2D, medido	1603.3 MB/s	778.0 MB/s	2.06
D2H, teórico	2000 MB/s	1000 MB/s	2.00
D2H, medido	1635.6 MB/s	817.6 MB/s	2.00

Cuadro 6.9: Comparativa de las transferencias por el bus entre la Tesla C1060 y la GTS 250

Transferencias por el bus. El cociente medido (≈2×) coincide con el cociente entre los enlaces negociados (x8 frente a x4), ya que ambas tarjetas saturan el suyo. Al colocar la

Tesla en la misma ranura que la GTS 250, ambas obtienen el mismo resultado (tabla 6.4), lo que confirma que este cociente refleja la diferencia entre ranuras y no entre tarjetas.

En conjunto, estos resultados muestran que, con GPUs heterogéneas, el rendimiento relativo no puede deducirse directamente de las especificaciones: depende de la arquitectura de cada chip, de la ranura que ocupa y de la configuración del enlace.

6.4. Ejecución concurrente

Para evaluar si el uso simultáneo de ambas tarjetas degrada su rendimiento, se lanzaron dos instancias de `matrixMul` a la vez, una en cada GPU. La tabla 6.10 compara el resultado con el obtenido en solitario.

GPU	**En solitario**	**Bajo concurrencia**
Tesla C1060	155.26 GFLOP/s	≈155.2–155.3 GFLOP/s
GTS 250	80.49 GFLOP/s	≈81.3 GFLOP/s

Cuadro 6.10: Rendimiento de `matrixMul` en solitario y bajo ejecución concurrente

No se observa ninguna degradación: las diferencias están dentro de la variación normal entre ejecuciones. Ambas tarjetas pueden, por tanto, utilizarse simultáneamente sin que una penalice a la otra. Conviene, no obstante, precisar el alcance de este resultado: como `matrixMul` apenas utiliza el bus (sección 6.8), el experimento demuestra la ausencia de interferencias en el cómputo, pero no somete al bus a una carga real. El comportamiento con dos cargas que transfieran grandes volúmenes de datos a la vez queda sin medir.

Que ambas tarjetas puedan trabajar a la vez no significa que sumen su rendimiento sin más: el resultado depende de cómo se reparta el trabajo entre ellas (sección 1.1). Con los rendimientos medidos, si el trabajo se reparte en proporción a la capacidad de cada tarjeta (ecuación (1.7)), la aceleración de la parte paralela es $A_{\mathrm{m}} = (155{,}26 + 80{,}49)/155{,}26 \approx 1{,}52$, en lugar de 2. Si, por el contrario, se reparte a partes iguales (ecuación (1.8)), la GTS 250 marca el ritmo de ambas y $A_{\mathrm{m}} = 2 \cdot 80{,}49/155{,}26 \approx 1{,}04$: la segunda tarjeta apenas aporta aceleración. Con los rendimientos pico teóricos (622 y 415 GFLOP/s), los valores serían $\approx 1{,}67$ y $\approx 1{,}33$. Si se incorporase la segunda Tesla C1060 del caso de estudio y rindiera igual que la primera, el sistema de tres GPUs alcanzaría $A_{\mathrm{m}} \approx 2{,}52$ con reparto proporcional, en lugar de 3. En un sistema heterogéneo, por tanto, repartir el trabajo según la capacidad de cada tarjeta es imprescindible para aprovecharlas.

6.5. Condición de carrera en la inicialización

6.5.1. Qué se observó

En el primer lanzamiento simultáneo, el proceso de la segunda GPU falló al inicializarse, mientras que el de la primera terminó correctamente. El error devuelto por CUDA no fue siempre el mismo: en una reproducción fue `cudaErrorNoDevice` (código 38, "no se encuentra el dispositivo") y en otra, `cudaErrorInvalidDevice` (código 10, "el dispositivo indicado no es válido") [30]. Que un mismo escenario produzca errores distintos es un indicio de que no se trata de un fallo determinista, sino de una *condición de carrera*: un error que solo aparece cuando dos operaciones coinciden en el tiempo de una forma concreta.

Para caracterizar el fallo se repitió el lanzamiento simultáneo varias veces. Tras el primer fallo, cinco repeticiones inmediatas no produjeron ningún fallo adicional. Tras reiniciar el contenedor, de quince repeticiones solo falló la primera. El fallo aparece, por tanto, únicamente en el primer lanzamiento simultáneo tras cada carga del driver, y las inicializaciones posteriores son estables.

6.5.2. Un punto de entrada compartido

Para entender cómo pueden interferir dos procesos que trabajan sobre GPUs distintas, hay que recordar los *device nodes* descritos en el capítulo 5. Además de un fichero por cada GPU, el driver expone uno de control, `/dev/nvidiactl`, que es común a todas. Cualquier proceso, se dirija a la GPU que se dirija, pasa primero por él para consultar qué GPUs existen y para registrar el contexto con el que va a trabajar. El driver mantiene esos registros en una estructura interna única, compartida por todos los procesos. Así, dos procesos que en apariencia no tienen nada que ver, porque cada uno usa su propia GPU, comparten en realidad un punto de paso común.

6.5.3. La pista del registro del kernel

El registro del kernel mostró, en el momento del fallo, el siguiente aviso:

```
WARNING: Flushing system-wide workqueues will be prohibited in near future.
os_flush_work_queue+0x4a/0x70 [nvidia]
rm_disable_adapter+0x7b/0x110 [nvidia]
nvidia_close+0x461/0x4d0 [nvidia]
```

Las tres últimas líneas indican qué estaba haciendo el driver: cerrar el contexto de una GPU (`nvidia_close`). Para interpretar la primera hay que saber qué es una *cola de trabajo* (*workqueue*). En Linux, cuando una parte del kernel tiene que realizar una tarea que no es urgente, en lugar de hacerla en el momento la deja anotada en una cola para que un proceso del propio kernel la ejecute poco después. El kernel ofrece una cola compartida por defecto, que utilizan a la vez muchos subsistemas: red, almacenamiento, USB y otros drivers.

Al cerrar el contexto de una GPU, el driver necesita asegurarse de que no queda ninguna tarea pendiente relacionada con ella antes de liberarla. La problemática reside en que el driver 340.108 espera solo a sus propias tareas, vaciando la cola compartida de todo el sistema, sin importar en qué estado se encuentren.

6.5.4. Interpretación

Con estos elementos, la explicación más coherente con lo observado es la siguiente. El proceso de la primera GPU termina y el driver empieza a cerrar su contexto, vaciando la cola compartida. Si en ese mismo instante el proceso de la segunda GPU está a mitad de su inicialización, que también depende de esa cola y de la estructura de registro común, puede encontrarse con un estado a medio actualizar. El driver no encuentra entonces lo que espera y responde con uno de los dos errores observados, según el punto exacto de la inicialización en el que se produzca la interferencia. Esto explicaría que el código de error varíe entre repeticiones.

El hecho de que el fallo solo aparezca la primera vez tras cargar el driver encaja con esta interpretación. En la primera inicialización tras la carga, ninguna GPU tiene aún estado previo y el driver debe preparar desde cero sus estructuras internas, un proceso más largo que en las inicializaciones siguientes. Cuanto más dura una operación, mayor es la probabilidad de que coincida con el cierre del otro proceso. Una vez el driver ha completado la primera inicialización de cada GPU, las siguientes son más rápidas y la coincidencia deja de producirse en la práctica.

6.5.5. Por qué el kernel moderno advierte de esta práctica

Vaciar la cola compartida del sistema era una práctica habitual y relativamente inofensiva en la fecha de salida de este driver. Con el tiempo, sin embargo, ha planteado dos problemas cada vez más graves. Por un lado, el tiempo de espera es impredecible: depende de todo lo que el resto del sistema tenga pendiente en la cola, no de la GPU, y en un servidor moderno, con más dispositivos y más drivers cargados, esa espera tiende a crecer. Por otro, provoca interferencias entre partes del sistema que no tienen relación entre sí, como la observada entre las dos GPUs.

Por ello, las versiones recientes del kernel muestran el aviso anterior, que anuncia que esta práctica dejará de estar permitida, y empujan a los desarrolladores a utilizar colas de trabajo propias de cada dispositivo, cuyo vaciado no afecta al resto del sistema. El driver 340.108 nunca se adaptó a este patrón y, al no recibir mantenimiento de NVIDIA, previsiblemente nunca lo hará. Es un buen ejemplo de cómo la evolución del kernel, que en el capítulo 4 impidió compilar el driver sin parches, también saca a la luz en tiempo de ejecución prácticas que hace una década se consideraban aceptables.

6.5.6. Mitigación

La mitigación es sencilla: basta con escalonar ligeramente el primer lanzamiento simultáneo tras cada arranque, o con reintentarlo en caso de fallo, tal como se recoge en el material complementario (sección A.3). No obstante, el hallazgo tiene un valor que va más allá de este caso concreto: muestra que un driver *legacy* puede funcionar correctamente en las pruebas individuales y, aun así, esconder defectos que solo aparecen bajo uso concurrente, precisamente el escenario que persigue un sistema multi-GPU.

6.6. Prueba de estabilidad bajo carga sostenida

6.6.1. Planteamiento

Las pruebas anteriores fueron breves: cada ejecución de `matrixMul` dura apenas unas décimas de segundo. Para evaluar el filtro $F_{\text{estabilidad}}$ (sección 1.5) era necesario comprobar el comportamiento del sistema bajo un uso prolongado. Para ello, con la misma configuración que en el resto del capítulo —la Tesla C1060 en la ranura principal y la GTS 250 en la secundaria—, se lanzó `matrixMul` en bucle continuo en ambas GPUs a la vez, con matrices de 4096×4096 para que cada ejecución durase más, y se programó una monitorización de diez minutos que registraba cada diez segundos la temperatura y la utilización de cada tarjeta mediante `nvidia-smi`. Al terminar, estaba previsto revisar el registro del kernel en busca de errores del driver.

6.6.2. Resultado: reinicio completo del host

La prueba no llegó a completarse. Durante su ejecución se perdió la conexión con el sistema y, al recuperarla, tanto el host como el contenedor llevaban encendidos apenas siete minutos: no se había caído solo la conexión, sino que la máquina física entera se había reiniciado. El registro del kernel del arranque anterior, conservado en disco, mostraba en los minutos previos al reinicio mensajes como el siguiente:

```
watchdog: BUG: soft lockup - CPU#4 stuck for 231s! [kworker/4:4:124258]
Workqueue: pm pm_runtime_work
RIP: 0010:pci_mmcfg_read+0xcb/0x110
```

Un *soft lockup* es la situación en la que un núcleo de la CPU queda atrapado ejecutando una misma tarea sin ceder el control al resto del sistema. El kernel dispone de un vigilante (*watchdog*) que detecta estas situaciones y las registra como un fallo grave. En este caso, la misma tarea apareció bloqueada en la misma CPU de forma repetida durante más de tres minutos y medio, hasta alcanzar los 231 segundos, y poco después el sistema se reinició. El registro no permite determinar con certeza qué desencadenó el reinicio, aunque, como se señaló en el capítulo 1, los vigilantes del kernel pueden configurarse para reiniciar el sistema ante este tipo de bloqueos [22].

6.6.3. Qué estaba haciendo el sistema

La tarea bloqueada pertenecía a la gestión de energía de los dispositivos PCI: el kernel estaba restaurando el estado de un dispositivo tras su salida de un modo de bajo consumo, y para ello leía su espacio de configuración (`pci_mmcfg_read`). Esa lectura nunca terminó. Además, en el momento del bloqueo, el registro donde se almacena el valor leído contenía `0xffffffff`, que es precisamente lo que se obtiene al intentar leer un dispositivo PCI que no responde. Otras trazas del mismo periodo muestran también funciones del manejador de interrupciones del driver NVIDIA, lo que sitúa el bloqueo en plena actividad de las GPUs.

6.6.4. Interpretación

La explicación más coherente con estos indicios es que, bajo carga sostenida, uno de los dispositivos del bus —previsiblemente una de las GPUs— dejó de responder, y el kernel quedó atrapado intentando leer su configuración, bloqueando la CPU hasta que el sistema se reinició. El registro no identifica qué dispositivo concreto dejó de responder. Varias observaciones posteriores apuntan, sin embargo, a un deterioro físico del hardware: la GTS 250 dejó de ser detectada por el sistema tras estas pruebas. Esta observación no permite, por sí sola, establecer una relación causal con el bloqueo, pero resulta coherente con un hardware de 2008–2009 que, sometido a un uso intensivo y prolongado, empieza a fallar.

6.6.5. Implicaciones

Este resultado es la evidencia más relevante del trabajo para el filtro $F_{\text{estabilidad}}$. A diferencia de la condición de carrera de la sección 6.5, que afectaba a un único proceso y tenía una mitigación sencilla, este fallo no quedó contenido en el dispositivo: bloqueó parte del host y terminó provocando el reinicio de la máquina completa, con la consiguiente interrupción de todos los servicios que se ejecutaban en ella. Se trata, por tanto, de un fallo propagado en el sentido definido en la sección 1.5. En términos del marco, esta prueba es el *burn-in* de la componente $F_{\text{carga sostenida}}$, y el sistema no lo superó: el fallo apareció antes de completar los diez minutos de monitorización previstos, de modo que la componente es falsa para cualquier duración T razonable de uso continuado. Su importancia es aún mayor porque se produjo con la configuración basada en contenedores, que había superado todas las pruebas anteriores, y no con la vía VFIO, cuya inestabilidad ya se conocía. Las pruebas breves demuestran, por tanto, que el sistema funciona, pero no que sea estable: solo un uso prolongado pone de manifiesto este tipo de fallos.

6.7. Consumo eléctrico

6.7.1. Ausencia de telemetría

Al consultar el consumo de las tarjetas con `nvidia-smi` [31], todos los campos de potencia devolvieron `N/A` en ambas GPUs. Estas generaciones de chips (GT200 y G92b) carecen del subsistema de gestión de energía que `nvidia-smi` necesita para informar del consumo, introducido en generaciones posteriores [38]. No es un fallo de configuración, sino una limitación de estas tarjetas, con una implicación directa para el objetivo del trabajo: un sistema de este tipo no puede informar de su propio consumo, y conocer su coste energético exige instrumentación externa.

6.7.2. Protocolo de medición

Dado que no se disponía de un vatímetro en el momento de la redacción, se definió el siguiente protocolo, que queda pendiente como trabajo futuro:

1. Medir el consumo del servidor completo en reposo, como línea base, tras dejar estabilizar térmicamente el equipo y con el resto de la carga del sistema en mínimos.
2. Generar una carga sostenida en una GPU, ejecutando `matrixMul` de forma continua con matrices grandes.

3. Medir el consumo durante varios minutos de carga sostenida, repitiendo cada medida al menos tres veces y descartando la primera si el equipo aún no ha alcanzado el régimen térmico.

4. Estimar el consumo atribuible a esa GPU como la diferencia entre el consumo bajo carga y el consumo en reposo.

5. Repetir para la segunda GPU y para ambas simultáneamente.

Esta diferencia debe interpretarse como una **cota superior** del consumo de la GPU, no como una medida exacta: al cargar la tarjeta también aumentan la actividad de la CPU, la velocidad de los ventiladores y las pérdidas de la fuente de alimentación, cuya eficiencia varía con la carga. Aun así, es la mejor estimación obtenible sin instrumentación por GPU y basta para decidir si el reaprovechamiento compensa en términos energéticos.

6.8. Influencia del bus PCIe y rentabilidad

Esta sección aplica los fundamentos de la sección 1.3 y el modelo económico de la sección 1.4 a los resultados medidos, para responder a dos preguntas: cuánto penaliza la topología PCIe de una plataforma de consumo y cuándo compensa el reaprovechamiento.

6.8.1. Por qué el bus no influyó en las pruebas de cómputo

En la ejecución de `matrixMul`, por el bus solo pasan las tres matrices, cada una formada por números de 4 bytes: las dos de entrada hacia la GPU y la de resultado de vuelta.

$$\begin{aligned} \text{Hacia la GPU:} &\quad (160 \cdot 160 + 160 \cdot 320) \times 4 \text{ B} = 307\,200 \text{ B} \\ \text{Hacia el host:} &\quad 160 \cdot 320 \times 4 \text{ B} = 204\,800 \text{ B} \\ \text{Total:} &\quad 512\,000 \text{ B} \approx 0{,}5 \text{ MB} \end{aligned}$$

Dividiendo cada volumen entre el ancho de banda medido en su sentido, estas transferencias tardan 307 200 B/1603,3 MB/s + 204 800 B/1635,6 MB/s ≈ 0,32 ms en la Tesla, y unos 0.65 ms en la GTS 250 con el mismo cálculo. El cálculo, en cambio, se repite 300 veces: 300 × 0,106 ms ≈ 32 ms en la Tesla y 300 × 0,204 ms ≈ 61 ms en la GTS 250. Las transferencias suponen, por tanto, en torno al 1 % del tiempo total (0,32/(32 + 0,32) y 0,65/(61 + 0,65)).

El modelo Roofline [52] llega a la misma conclusión. La intensidad aritmética respecto al bus es el cociente entre las operaciones realizadas y los bytes transferidos:

$$I_{\text{A}} = \frac{300 \times 16{,}38 \cdot 10^6 \text{ FLOP}}{512\,000 \text{ B}} \approx 9600 \text{ FLOP/byte}$$

El punto a partir del cual una carga deja de estar limitada por el bus es el cociente entre el pico de cálculo y el ancho de banda del bus: 622 GFLOP/s/1,60 GB/s ≈ 390 FLOP/byte para la Tesla y 415 GFLOP/s/0,78 GB/s ≈ 530 FLOP/byte para la GTS 250. La carga se encuentra muy por encima de ambos valores: es claramente *compute-bound*, y por eso ni la generación PCIe ni la ranura afectan a su rendimiento.

Estos puntos de cresta, calculados con el ancho de banda medido, son algo mayores que los teóricos de cada enlace (622/2,0 ≈ 311 FLOP/byte para la Tesla en su enlace x8 en Gen 1, y 415/1,0 = 415 FLOP/byte para la GTS 250 en su enlace x4 en Gen 1), porque el bus real rinde en torno al 80 % de su máximo. En cualquier caso, ambos están muy lejos de la cresta de la memoria interna de cada tarjeta (622/102,4 ≈ 6 y 415/64,0 ≈ 6,5 FLOP/byte): en términos de intensidad aritmética, el bus es entre 50 y 64 veces más exigente que la VRAM, casi dos órdenes de magnitud. La configuración de las pruebas acentúa esta diferencia: con su enlace nativo (Gen 2 x16, 8 GB/s), la cresta de la Tesla bajaría a unos 78 FLOP/byte, cuatro veces menos.

6.8.2. Qué ocurriría con una carga limitada por el bus

La situación cambia por completo con una carga que deba transferir grandes volúmenes de datos. Para transferir 2 GB hacia la GPU, por ejemplo un conjunto de imágenes, la tabla 6.11 muestra el tiempo necesario en cada configuración, calculado como el cociente entre el volumen y el ancho de banda (por ejemplo, 2 GB/0,78 GB/s ≈ 2,6 s para la GTS 250). Para las dos primeras filas se emplea el ancho de banda medido, y para las dos últimas, el máximo teórico, por lo que estas representan el mejor caso posible.

Configuración	**Ancho de banda**	**Tiempo para 2 GB**
GTS 250, ranura del *chipset*, Gen 1 (medido)	0.78 GB/s	≈2.6 s
Tesla C1060, ranura principal, Gen 1 (medido)	1.60 GB/s	≈1.25 s
Cualquiera de las dos en la ranura principal con sus 16 carriles, Gen 2 (cota teórica)	8 GB/s	≥0.25 s
GPU moderna en servidor, PCIe 4.0 x16 (cota teórica) [51]	32 GB/s	≥0.06 s

Cuadro 6.11: Tiempo de transferencia de 2 GB según la configuración del bus

En una carga así, el tiempo de transferencia domina sobre el de cálculo y, siguiendo a Amdahl [5], el beneficio de añadir una segunda GPU queda limitado por la parte que no se acelera, que ahora es la transferencia por el bus. La topología de consumo penaliza además de forma desigual: la tarjeta conectada a través del *chipset* tarda el doble que la de la ranura principal, por lo que, si el trabajo se reparte a partes iguales, la más lenta marca el ritmo de ambas [2, 8].

6.8.3. Cuándo compensa: comparativa con una alternativa moderna

Aplicando el modelo económico (sección 1.4) al caso de estudio, la tabla 6.12 compara el sistema reaprovechado con una GPU actual de gama media. Para que la comparación sea homogénea, el rendimiento y la eficiencia se comparan pico con pico.

Concepto	**Reaprovechamiento (C1060 + GTS 250)**	**Alternativa moderna (RTX 4060)**
Coste del hardware	0 EUR (rescatado)	∼350 EUR
Rendimiento pico	≈1037 GFLOP/s (622 + 415)	∼15 000 GFLOP/s
Rendimiento medido (`matrixMul`)	235.75 GFLOP/s (155.26 + 80.49)	—
Consumo máximo (TDP)	≈338 W (188 + 150)	115 W
Eficiencia pico	≈3.1 GFLOP/W	∼130 GFLOP/W
Versión máxima de CUDA	6.5 (CC 1.1/1.3)	13.x (CC 8.9)
Horas de puesta a punto	15–25 h (cap. 4–5) + riesgo de los parches	∼1 h (driver oficial)

Cuadro 6.12: Comparativa de rentabilidad. La alternativa moderna ofrece unas 14 veces más rendimiento pico y unas 40 veces más eficiencia energética. TDPs según [49, 48, 26].

Los valores de la tabla se obtienen de la siguiente forma. El rendimiento pico del sistema reaprovechado es la suma de los picos de ambas tarjetas (ecuación 6.2), y su consumo máximo, la suma de sus potencias de diseño (TDP). La eficiencia pico es el cociente entre ambos: 1037 GFLOP/s/338 W ≈ 3,1 GFLOP/W para el sistema reaprovechado, frente a 15 000 GFLOP/s/115 W ≈ 130 GFLOP/W para la alternativa moderna. De ahí salen los dos cocientes de la comparación: 15 000/1037 ≈ 14 en rendimiento y 130/3,1 ≈ 42 en eficiencia.

La comparación es clara en términos de rendimiento: la alternativa moderna ofrece unas 14 veces más capacidad de cálculo con un tercio del consumo, lo que se traduce en unas 40 veces más eficiencia. El reaprovechamiento solo compensa cuando el hardware tiene coste nulo y el valor del cálculo no exige eficiencia energética ni versiones modernas de CUDA.

La ley de Gustafson [13] matiza esta conclusión para el aprovechamiento conjunto de las tarjetas. Si el problema crece con los recursos —por ejemplo, pasando de las matrices de 160×320 del ejemplo a matrices de 4096×4096—, la intensidad aritmética aumenta, la parte paralelizable domina y el sistema escala pese a la heterogeneidad. Si el problema es fijo y pequeño, en cambio, domina Amdahl y no compensa el esfuerzo de adaptar siete años de evolución del kernel. En un contexto formativo, donde el objetivo es aprender las

técnicas de VFIO, LXC y DKMS, la rentabilidad es formativa y el reaprovechamiento sí compensa; en producción, donde cada kWh cuenta y se necesitan versiones modernas de CUDA, no.

Para cerrar la ecuación de ganancia con datos reales falta medir el consumo siguiendo el protocolo de la sección 6.7.2. Hasta entonces, la decisión debe apoyarse en el marco $V = F_{\text{driver}} \wedge F_{\text{aislamiento}} \wedge F_{\text{carga}} \wedge F_{\text{coste}} \wedge F_{\text{estabilidad}}$ de la figura 6.1.

6.9. Aplicación del marco de viabilidad al caso de estudio

Con los resultados de este capítulo y de los anteriores, puede aplicarse ya el marco de viabilidad del capítulo 1 al caso de estudio. La figura 6.1 recorre los cinco filtros en orden; basta con que uno falle para que el despliegue no compense y deba descartarse o replantearse.

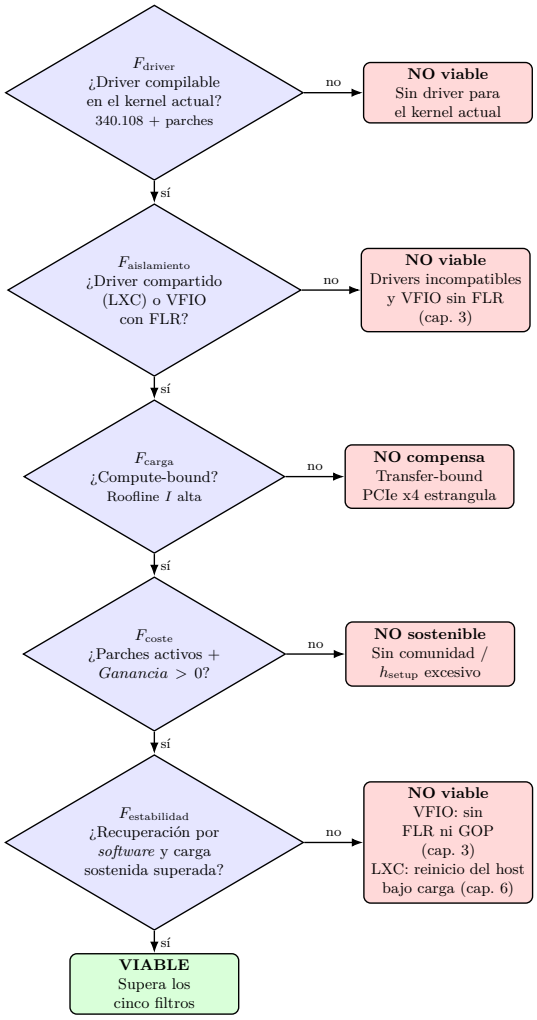


Figura 6.1: Árbol de decisión de viabilidad $V = F_{\text{driver}} \wedge F_{\text{aislamiento}} \wedge F_{\text{carga}} \wedge F_{\text{coste}} \wedge F_{\text{estabilidad}}$. Cada filtro desfavorable conduce a una salida de no viabilidad, y solo superando los cinco se alcanza la salida viable. Se resalta el recorrido del caso de estudio, que supera los cuatro primeros filtros y no supera $F_{\text{estabilidad}}$; a la izquierda se indica el capítulo en el que se obtuvo la evidencia de cada paso.

F1. Soporte del driver [33, 32]. ¿Existe un driver para las GPUs que pueda compilarse y cargarse en un kernel actual? En nuestro caso, el driver 340.108 carece de soporte oficial para los kernels actuales, y solo pudo compilarse gracias a los parches mantenidos por la comunidad (capítulo 4) [9]. El filtro se supera, por tanto, pero condicionado a la existencia de esos parches: sin ellos habría sido necesario adaptar el código del driver por cuenta propia. Esa dependencia es la que recoge el filtro de coste.

F2. Homogeneidad del driver y aislamiento [23, 40]. ¿Comparten todas las GPUs el mismo driver? Si es así, pueden convivir en el kernel del *host* y el aislamiento no es necesario: basta con contenedores LXC, que comparten ese driver. Si necesitan drivers incompatibles entre sí, no pueden compartir el kernel, por la limitación del driver único descrita en el capítulo 2 [39], y deben aislarse en máquinas virtuales, lo que requiere IOMMU activa, un grupo IOMMU dedicado [45] y FLR correcto [3]. En nuestro caso,

ambas tarjetas funcionan con el driver 340.108, de modo que basta la vía LXC. La vía VFIO, en cambio, sería inviable, porque las tarjetas no implementan FLR (capítulo 3).

F3. Perfil de carga [52, 5]. ¿La aplicación es *compute-bound*? Con el modelo Roofline (sección 1.2), si $T_{\text{cómputo}} \gg B/\text{BW}_{\text{PCIe}}$, el número de carriles no importa. En este capítulo se ha comprobado que `matrixMul` lo es (sección 6.8), por lo que el enlace estrecho de la ranura del *chipset* no la penaliza. Para una carga *transfer-bound*, en cambio, esa ranura x4 [8, 1] haría que la plataforma de consumo no compensara frente a una de servidor.

F4. Coste y sostenibilidad [9, 7]. ¿Existe mantenimiento comunitario y la ganancia de la ecuación (1.16) es positiva? El capítulo 4 cuantifica h_{setup}: siete años de diferencia entre driver y kernel, 18 parches aplicados y un *toolchain* adaptado. Sin los repositorios comunitarios, $C_{\text{riesgo}} \to \infty$. Este es el filtro que convierte un éxito técnico puntual en un despliegue sostenible.

F5. Estabilidad y contención del fallo [3, 47]. ¿Quedan los fallos contenidos en el dispositivo, sin propagarse al resto del sistema? Como se formaliza en la sección 1.5, este filtro se evalúa en dos partes. La primera, $F_{\text{recuperación}}$, comprueba si existe algún mecanismo por *software* capaz de recuperar una GPU colgada, y resultó falsa: las tarjetas no implementan FLR y la placa no ofrece control de alimentación por ranura (capítulo 3). La segunda, $F_{\text{carga sostenida}}$, exige superar una prueba de carga prolongada sin fallos que se propaguen al *host*, y también resultó falsa: el *host* completo se reinició durante la prueba (sección 6.6). Por tanto, $F_{\text{estabilidad}} = \text{falso}$.

La utilidad del marco es que anticipa buena parte del resultado sin necesidad de experimentar: nuestro caso supera F1–F4, y $F_{\text{recuperación}}$ descarta la vía VFIO, que LXC evita al no necesitar reinicios de dispositivo; un caso con una GPU Kepler (driver 470) junto a una GT200 (driver 340) fallaría ya en F1, y una carga de *streaming* fallaría en F3. La excepción es $F_{\text{carga sostenida}}$, que solo puede comprobarse con una prueba de campo y que, en nuestro caso, resultó desfavorable también para LXC.

6.10. Discusión de resultados

Los resultados de este capítulo permiten extraer varias conclusiones sobre la viabilidad de un sistema multi-GPU heterogéneo:

- **El aprovechamiento conjunto es posible en pruebas breves**: ambas GPUs pueden utilizarse simultáneamente sin que una degrade el rendimiento de la otra, al menos en cargas de cálculo intensivo. La heterogeneidad no impide el uso conjunto, aunque sí obliga a repartir el trabajo según la capacidad de cada tarjeta.
- **El sistema no es estable bajo uso sostenido**: al mantener ambas GPUs en carga continua, un bloqueo del kernel terminó provocando el reinicio del host completo

(sección 6.6). Es el resultado más relevante del capítulo, por dos motivos. Por un lado, el fallo no quedó contenido en ningún dispositivo, sino que afectó a toda la máquina, por lo que el sistema no supera el filtro $F_{\text{estabilidad}}$. Por otro, se produjo con la configuración basada en contenedores, que había superado todas las pruebas anteriores, lo que demuestra que las pruebas breves no bastan para validar un sistema con hardware *legacy*: solo un uso prolongado pone de manifiesto este tipo de fallos.

- **Las especificaciones no bastan para predecir el rendimiento relativo**: en el cómputo y en la memoria interna, los cocientes medidos se desvían de los teóricos. La arquitectura de cada chip, la ranura que ocupa y la configuración del enlace pesan tanto como el número de núcleos o el ancho del bus.

- **La topología PCIe de consumo es decisiva para cargas limitadas por el bus**: en las pruebas de cálculo no tuvo efecto, pero la tarjeta conectada a través del *chipset* dispone de la mitad de carriles que la de la ranura principal y lo satura, lo que la convierte en el cuello de botella de cualquier carga que transfiera grandes volúmenes de datos.

- **El software *legacy* impone un coste elevado**: compilar código nuevo exige un entorno adicional con un compilador de hace una década, mucho más complejo que ejecutar binarios ya compilados. Además, el driver presenta defectos latentes, como la condición de carrera de la sección 6.5, que, aunque tiene una mitigación sencilla, debe tenerse en cuenta en cualquier despliegue.

- **El hardware *legacy* tiene límites físicos**: la ausencia de telemetría de potencia impide conocer el consumo sin instrumentación externa, un factor relevante en cualquier despliegue de estas características. A ello se suman los indicios de deterioro observados durante las pruebas —una tarjeta que dejó de ser detectada, el sobrecalentamiento de otra y un enlace que solo funciona con la mitad de sus carriles—, coherentes con la inestabilidad descrita.

Conclusiones

Este trabajo se planteó como un estudio experimental para responder a una pregunta concreta: *¿es posible aprovechar múltiples GPUs heterogéneas —de años, generaciones y propósitos distintos— dentro de una misma máquina?* A lo largo de los capítulos anteriores hemos construido un sistema real con tres tarjetas NVIDIA disponibles de la microarquitectura Tesla (2008–2009) y hemos documentado, paso a paso, qué ha sido posible hacer con ellas y qué no. La configuración experimental empleó dos tarjetas, una Tesla C1060 y la ENGTS250, porque la tercera no fue detectada a través del *riser* PCIe. En esta sección final recogemos las conclusiones del trabajo, organizadas en torno a las fases del proyecto, y las líneas de trabajo futuro.

Conclusiones por fase

Sobre la virtualización completa (capítulo 3). El aislamiento de cada GPU en una máquina virtual mediante VFIO resultó inviable para este hardware. En las pruebas realizadas, ninguna de las dos tarjetas pudo recuperarse por software: el caso documentado con mayor detalle fue la ASUS ENGTS250, que solo expone los métodos `pm` y `bus`, sin FLR, y quedó eléctricamente colgada. La VBIOS sin soporte UEFI impide además un paso directo estable con QEMU/KVM. La conclusión técnica es que la virtualización completa no es la vía adecuada para integrar estas GPUs *legacy* en la plataforma empleada, y que la documentación de este fallo fue tan valiosa como lo habría sido un éxito: delimitó con precisión las condiciones que debe cumplir cualquier vía alternativa.

Sobre los contenedores LXC (capítulos 4 y 5). La vía de los contenedores resultó la adecuada para integrar las tarjetas. Al compartir el kernel del host, los contenedores no necesitan ni IOMMU ni resets de dispositivo: basta con montar los *character devices* del driver. La contrapartida —la limitación del driver único— se convirtió en el problema central, y su solución constituye el principal resultado técnico del trabajo: es posible ejecutar el driver *legacy* NVIDIA 340.108 (2019) sobre un kernel de 2026 mediante parches comunitarios y DKMS. La validación final fue contundente: código CUDA real (`deviceQuery`) ejecutándose de extremo a extremo dentro del contenedor, con ambas tarjetas operativas simultáneamente.

Sobre el rendimiento (capítulo 6). Los benchmarks confirman que el aprovechamiento conjunto de las tarjetas es posible y no degrada el rendimiento individual en cargas de cálculo intensivo: ambas GPUs ejecutan cargas concurrentes sin contención apreciable. El análisis de los cocientes entre ambas tarjetas revela, sin embargo, que el rendimiento relativo no puede deducirse directamente de las especificaciones: en el cómputo y en la memoria interna, los cocientes medidos se desvían de los teóricos por diferencias de arquitectura entre ambos chips, y en las transferencias por el bus la diferencia no se debe a las tarjetas, sino al enlace de la ranura que ocupa cada una.

Sobre la estabilidad (capítulo 6). Las pruebas breves no reflejan el comportamiento del sistema bajo un uso prolongado. Al someter ambas GPUs a carga continua, el sistema sufrió un bloqueo del kernel que terminó con el reinicio del host completo: un fallo que no quedó contenido en ningún dispositivo, sino que se propagó a toda la máquina. Junto con otros indicios de deterioro físico observados durante el trabajo —una de las tarjetas dejó de ser detectada, otra se sobrecalentó y el enlace de la Tesla C1060 solo funciona con la mitad de sus carriles—, este resultado muestra que la principal limitación de este hardware no está en su integración, sino en su fiabilidad tras más de quince años de vida.

Sobre las limitaciones. El trabajo ha identificado tres tipos de limitaciones:

- *Limitaciones de software*: la instalación de software NVIDIA/CUDA de más de diez años de antigüedad sobre sistemas modernos requiere sistemáticamente extracción y ejecución manual por capas, y el desarrollo de código CUDA nuevo requiere un entorno de compilación adicional (*chroot* con una distribución de 2015).
- *Limitaciones de mantenimiento*: la viabilidad de este tipo de despliegues depende enteramente del mantenimiento comunitario de los parches del driver, un riesgo que debe asumirse al adoptar esta vía.
- *Limitaciones físicas*: el hardware carece de telemetría de potencia, de memoria unificada y de soporte para arquitecturas de cómputo modernas, lo que restringe los *toolkits* utilizables (CUDA 6.5 como límite) y dificulta la monitorización energética; además, su envejecimiento compromete la estabilidad del sistema bajo uso sostenido.

Respuesta a la pregunta de viabilidad

A la vista de los resultados, la respuesta a la pregunta inicial debe distinguir entre dos aspectos. En cuanto a la *integración*, la respuesta es *sí, con condiciones*: es posible aprovechar de forma conjunta GPUs heterogéneas en una misma máquina cuando todas las tarjetas comparten familia de driver (misma rama *legacy*) y se aceptan las restricciones de software y mantenimiento derivadas de la obsolescencia. La heterogeneidad de propósito

(cómputo frente a consumo) y de rendimiento no supone un obstáculo técnico: el sistema trata las tarjetas como dos dispositivos independientes del mismo driver, y la vía LXC supera los filtros de driver, aislamiento y carga del marco propuesto.

En cuanto a la *estabilidad*, en cambio, la respuesta es negativa para el caso de estudio: el sistema no superó una prueba de carga sostenida sin fallos que afectasen al host completo, por lo que no cumple el filtro $F_{\text{estabilidad}}$. En consecuencia, el reaprovechamiento de este hardware es viable para usos puntuales, experimentales o formativos, en los que un fallo ocasional es asumible, pero no para un servicio continuo. Este resultado ilustra, además, una de las conclusiones metodológicas del trabajo: con hardware *legacy*, la viabilidad no puede decidirse por completo sobre el papel, y el marco propuesto debe completarse siempre con una prueba de campo prolongada.

El caso de estudio es, en realidad, el caso más favorable dentro de la heterogeneidad: dos tarjetas de la misma microarquitectura y con la misma rama de driver. Su generalización plantea dos escenarios distintos. Con tarjetas del **mismo fabricante** pero de generaciones que requieran ramas incompatibles del driver propietario de NVIDIA, la vía de los contenedores no es aplicable: el kernel del host solo puede cargar un módulo de ese driver, de modo que una de las tarjetas quedaría sin soporte y sería necesario recurrir a la virtualización completa. Con tarjetas de **fabricantes distintos** (por ejemplo, NVIDIA y AMD), en cambio, cada una utiliza su propio módulo, y ambos pueden coexistir en el mismo kernel, como ocurre en el propio sistema del caso de estudio, donde el driver de NVIDIA convive con el de la gráfica integrada de AMD. En ese caso, la dificultad se traslada a mantener en un mismo sistema dos entornos de cómputo distintos (CUDA y OpenCL o ROCm) y a garantizar la compatibilidad de cada driver con el kernel. Ambos escenarios quedan fuera del alcance de este trabajo y constituyen líneas de trabajo futuro naturales.

Cabe pensar, por último, que buena parte de las incompatibilidades encontradas se deben a la antigüedad extrema del hardware empleado, de más de quince años, y que con GPUs más recientes el marco podría arrojar un veredicto distinto. La sección siguiente concreta esta idea.

Configuración mínima para un sistema realista

El marco de viabilidad se construyó a partir de las dificultades encontradas, pero también puede aplicarse en sentido inverso: recorrer sus filtros para determinar qué condiciones mínimas debe cumplir el hardware para que un sistema de este tipo resulte útil. El resultado sirve como primer requisito a la hora de buscar o adquirir tarjetas de segunda mano.

Al reaprovechar hardware, lo más probable es que las tarjetas disponibles no compartan driver. Por ello, la configuración mínima se plantea para el caso más restrictivo: cada

GPU asignada a una máquina virtual mediante paso directo de dispositivos. La vía de los contenedores, que exige que todas las tarjetas compartan driver, es un caso más favorable que no puede darse por supuesto. Los requisitos se dividen en tres grupos: los que dependen de la generación de la tarjeta y pueden comprobarse antes de adquirirla, los que solo pueden validarse con cada unidad y los que impone la plataforma.

Requisitos de la tarjeta, comprobables antes de adquirirla.

- **Driver con soporte en un kernel actual** (F_{driver}). Cada driver que necesiten las tarjetas, sea uno o varios, debe poder compilarse y cargarse en un kernel actual. Si solo es posible mediante parches de la comunidad, como en el caso de estudio, el filtro se supera, pero a costa del riesgo que recoge F_{coste}.
- **Uso en máquinas virtuales permitido por el fabricante** ($F_{\text{aislamiento}}$). En las GPUs de consumo de NVIDIA, el paso directo a máquinas virtuales está admitido oficialmente desde la rama 465 del driver, de 2021 [17].
- **Telemetría de consumo** (F_{coste}). Sin ella, el coste energético solo puede conocerse con instrumentación externa, como ocurrió en el caso de estudio. Conviene comprobar que la tarjeta informa de su consumo a través de `nvidia-smi` [31], algo que las generaciones consideradas a continuación sí hacen.
- **Soporte de una versión de CUDA mantenida**. Además del driver, el *toolkit* de CUDA determina qué compiladores y bibliotecas pueden emplearse; en el caso de estudio, quedar limitado a CUDA 6.5 fue una restricción tan relevante como la del propio driver.
- **Firmware UEFI (GOP)**: recomendable, pero no imprescindible. Una GPU destinada solo a cómputo no necesita proporcionar imagen durante el arranque, y la máquina virtual puede utilizar un firmware BIOS clásico, como se hizo en el capítulo 3.

Aplicados a las generaciones de NVIDIA, estos requisitos sitúan dos umbrales. Maxwell, Pascal y Volta (2014–2017) dependen de la rama 580 del driver, la última que les da soporte y que pasa a ser una rama *legacy* [18]. Turing (2018) y las generaciones posteriores, en cambio, utilizan la rama actual del driver y las versiones actuales de CUDA. De ahí se obtienen dos referencias:

- **Configuración mínima viable hoy: Pascal**, que cumple todos los requisitos con soporte oficial, pero con fecha de caducidad: en pocos años quedará en la situación de las tarjetas del caso de estudio.
- **Configuración mínima sostenible: Turing o posterior**, que los cumple sin fecha de caducidad a la vista.

Si, más allá del alcance de este trabajo, el sistema se quisiera destinar a aprendizaje profundo, el umbral se desplazaría antes hacia Turing, puesto que las versiones recientes de los entornos más extendidos ya han dejado de distribuir binarios para Maxwell, Pascal y Volta [43].

Requisitos que solo pueden validarse con cada tarjeta.

- **Reinicio por *software*** ($F_{\text{recuperación}}$). No puede deducirse de la generación: existen casos documentados de tarjetas recientes que anuncian FLR y cuyo reinicio, aun así, falla [42]. Debe comprobarse que la tarjeta anuncia FLR (`reset_method`) y, después, que el reinicio funciona realmente, asignándola y liberándola varias veces de una máquina virtual.
- **Estabilidad bajo carga** ($F_{\text{carga sostenida}}$). Con hardware de segunda mano, el estado físico de cada unidad es desconocido, por lo que cada tarjeta debe superar una prueba de carga prolongada antes de incorporarse al sistema.

Esta división coincide con la del propio filtro de estabilidad: el marco no elimina la incertidumbre del hardware reutilizado, pero la reduce a aquello que solo puede comprobarse con la tarjeta delante.

Requisitos de la plataforma: cuántas GPUs admite. Un centro de cálculo de bajo coste necesita varias GPUs, y su número lo limitan el procesador y la placa base:

- **Un grupo IOMMU propio para cada GPU**, condición necesaria para asignarlas a máquinas virtuales por separado. Las ranuras conectadas directamente al procesador lo garantizan casi siempre; las del *chipset*, solo en algunas placas. En la placa del caso de estudio, la ranura del *chipset* sí permitió asignar la GTS 250 a `vfio-pci`, pero comparte con otros dispositivos tanto el ancho de banda como el puente PCIe, lo que expone al resto del sistema ante un reinicio de bus (capítulo 3).
- **Carriles suficientes para cada GPU** (F_{carga}). Las tarjetas de estas generaciones están diseñadas para enlaces x16, pero funcionan con menos carriles a costa de ancho de banda. Si se fija como mínimo un enlace de unos 8 GB/s, una tarjeta PCIe 3.0, como las de Turing, necesita ocho carriles, mientras que una PCIe 4.0, de Ampere en adelante, se conforma con cuatro. El número máximo de GPUs es, aproximadamente, el cociente entre los carriles del procesador disponibles para ellas y los carriles que necesita cada una. La generación de la tarjeta determina, por tanto, también cuántas caben.

Con este criterio, una plataforma de consumo como la del caso de estudio, con dieciséis carriles del procesador para las GPUs, admite dos tarjetas PCIe 3.0 en x8/x8, o hasta cuatro PCIe 4.0 en x4/x4/x4/x4 si la placa ofrece ese modo de bifurcación. Para más tarjetas es necesario recurrir a plataformas de estación de trabajo o de servidor de segunda mano, que ofrecen entre 40 y 128 carriles directos al procesador. Los montajes con alargadores de un solo carril, habituales en minería, permiten conectar muchas tarjetas, pero no superan F_{carga} salvo con cargas de intensidad aritmética muy elevada. A todo ello se suman los requisitos físicos: una fuente de alimentación con margen sobre el consumo conjunto y una separación suficiente entre ranuras, ya que la mayoría de GPUs ocupan el espacio de dos y pueden bloquear la contigua o dificultar su refrigeración.

Configuración resultante. La configuración mínima viable parte de GPUs Pascal, con fecha de caducidad, y la mínima sostenible, de Turing en adelante. En ambos casos, el reinicio por *software* y la estabilidad bajo carga deben verificarse en cada tarjeta. En una plataforma de consumo caben dos GPUs PCIe 3.0, o cuatro PCIe 4.0 con bifurcación; un centro de cálculo de más de cuatro GPUs exige una plataforma de estación de trabajo o de servidor, con carriles directos al procesador para todas ellas.

Inteligencia artificial y mantenimiento de hardware heredado

Una parte importante de las dificultades encontradas no se debió a límites del hardware, sino al desconocimiento previo de sus compatibilidades y requisitos, y al esfuerzo de adaptar software sin mantenimiento a un sistema actual. Cabe preguntarse, por tanto, en qué medida un sistema de inteligencia artificial con acceso a compiladores, al código fuente del kernel y a las especificaciones de los dispositivos podría reducir ese esfuerzo, por ejemplo como un servicio abierto a la comunidad que facilitase el reaprovechamiento de hardware antiguo. A la vista de los problemas encontrados en este trabajo, un servicio así podría abordar, al menos, las siguientes tareas:

- **Diagnóstico previo de compatibilidad.** A partir de la descripción del equipo (GPUs, placa base y procesador), indicar qué rama del driver y qué versión de CUDA admite cada tarjeta, si todas pueden compartir driver y cuáles de los requisitos de la sección anterior se cumplen. Parte de lo que en este trabajo se descubrió mediante pruebas podría haberse anticipado de este modo.
- **Adaptación del driver a nuevos kernels.** El kernel de Linux no mantiene una interfaz interna estable para los módulos externos, por lo que cada nueva versión puede impedir la compilación de un driver sin mantenimiento. Los parches comunitarios

del capítulo 4 son precisamente adaptaciones de este tipo. Se trata de una tarea repetitiva cuyo resultado puede comprobar el propio compilador, lo que la hace adecuada para su automatización.

- **Preparación de entornos de compilación.** En el capítulo 6, `nvcc` 6.5 no pudo compilar con las cabeceras de un gcc actual y fue necesario un entorno con Debian 8 y gcc 4.9. Identificar esa combinación, prepararla y ajustar las opciones de compilación (`-gencode`) a las tarjetas instaladas son tareas bien acotadas.
- **Configuración del arranque y corrección de prácticas obsoletas.** Proponer los parámetros del kernel adecuados para cada plataforma, como los de gestión de energía de PCIe empleados en el capítulo 2, o modificar, en la parte abierta del driver, prácticas que el kernel actual desaconseja, como el vaciado de la cola de trabajo global al que se atribuye la condición de carrera de la sección 6.5.

Existen, sin embargo, límites que un servicio de este tipo no podría superar:

- **Código cerrado.** En el driver de NVIDIA, solo la capa de interfaz con el kernel se distribuye como código fuente; el núcleo del driver y las bibliotecas de CUDA son binarios cerrados. Sin acceso a ese código, no es posible llevar versiones actuales de CUDA a tarjetas que el fabricante ha dejado de admitir. Con drivers de código abierto el margen de actuación sería mayor, aunque seguiría limitado por las capacidades del propio hardware.
- **Hardware.** El software no puede añadir capacidades que la tarjeta no tiene, como el FLR, el firmware UEFI o la telemetría que no expone, ni corregir su deterioro físico, como el posible fallo de contactos que limitó la Tesla C1060 a x8 o el que podría explicar el bloqueo bajo carga sostenida del capítulo 6.
- **Validación sobre hardware real.** Una máquina virtual permite comprobar que un driver compila y se carga, pero no que funcione con una tarjeta concreta ni que se mantenga estable durante horas de carga. Para ello harían falta las tarjetas físicas, ya sea en un laboratorio del propio servicio o en los equipos de los usuarios.
- **Confianza y licencias.** Un módulo del kernel se ejecuta con los máximos privilegios del sistema, de modo que un error que no impida la compilación puede provocar fallos difíciles de detectar; los cambios generados tendrían que revisarse antes de distribuirse. Además, la licencia del driver de NVIDIA condiciona su redistribución, por lo que el servicio debería ofrecer parches y procedimientos en lugar de binarios, como ya hace la comunidad.

En términos del marco, un servicio así actuaría sobre F_{driver}, en la parte abierta del código, y sobre F_{coste}, al reducir el tiempo de puesta en marcha y la dependencia de parches

mantenidos por particulares. Los filtros $F_{\text{aislamiento}}$, F_{carga} y $F_{\text{estabilidad}}$ dependen sobre todo del hardware, y apenas se verían afectados. Una herramienta de este tipo podría, por tanto, alargar la vida útil del software, pero no la del hardware.

Trabajo futuro

Las líneas de trabajo futuro que se desprenden de este proyecto son las siguientes:

- **Diagnóstico de la estabilidad**: repetir la prueba de carga sostenida con las tarjetas revisadas o sustituidas por unidades equivalentes, registrando de forma continua la temperatura y el registro del kernel en un almacenamiento persistente, para determinar si los fallos observados se deben al deterioro de unidades concretas o son propios de esta generación de hardware sobre una plataforma moderna.
- **Medición del consumo eléctrico** con instrumentación externa (vatímetro o enchufe inteligente), siguiendo el protocolo definido en el capítulo 6, para cuantificar el coste energético real de este tipo de despliegues.
- **Ampliación de la heterogeneidad**: incorporar al sistema tarjetas de generaciones distintas dentro de la misma rama de drivers y estudiar configuraciones que respeten la limitación del driver único. Si las nuevas tarjetas exigieran ramas incompatibles entre sí, sería necesario un aislamiento real por máquina virtual, ya que asignar cada GPU a un contenedor distinto no basta: todos los contenedores comparten el mismo módulo del kernel del anfitrión.
- **Heterogeneidad de marcas**: estudiar la integración de GPUs AMD (OpenCL) junto a las NVIDIA (CUDA), analizando las diferencias de *runtime* y las alternativas de abstracción entre ambos ecosistemas.
- **Frameworks multi-GPU**: evaluar el rendimiento conjunto con cargas de trabajo que distribuyan computación entre ambas tarjetas de forma coordinada, más allá de la ejecución concurrente independiente estudiada en este trabajo.
- **Alternativas a la virtualización completa**: evaluar con el marco otras formas de dar a cada driver su propio kernel o de evitar la limitación del driver único. Entre ellas, las máquinas virtuales ligeras, que siguen requiriendo VFIO pero con menor sobrecarga; el reparto de las GPUs entre varios equipos conectados por red, con herramientas de cálculo distribuido o de uso remoto de GPUs, que sustituye el aislamiento por más hardware y una red más lenta que el bus PCIe; y los drivers libres que admiten varias generaciones de tarjetas en un mismo kernel, a cambio de renunciar a CUDA.

- **Servicio de compatibilidad asistido por inteligencia artificial**: explorar la viabilidad del servicio descrito en la sección anterior, empezando por el diagnóstico previo de compatibilidad, que no requiere distribuir código, y evaluando la calidad de los parches generados frente a los mantenidos por la comunidad.
- **Automatización del despliegue**: convertir en un script el procedimiento de instalación del driver documentado en el material complementario y extenderlo a la configuración del contenedor y a la gestión del arranque dual LXC/VFIO, de modo que el procedimiento reproducible documentado en el apéndice pueda ejecutarse de forma idempotente sobre un segundo host con hardware equivalente.

En definitiva, este trabajo demuestra que el aprovechamiento de GPUs heterogéneas es un problema real, con solución parcial y con muchas aristas técnicas abiertas. La documentación exhaustiva de los obstáculos y de sus causas raíz, que constituye el cuerpo de esta memoria, es la aportación principal del proyecto: permite a cualquier persona interesada en construir un sistema de este tipo conocer de antemano dónde están los límites y cómo salvar los problemas documentados.

Sobre la novedad y originalidad del trabajo

Cabe preguntarse si un sistema con 235.75 GFlop/s medidos (unos 1037 GFlop/s de pico teórico, capítulo 6) tiene novedad frente a una sola GPU moderna de 15 TFlop/s de pico. La respuesta es que la aportación no reside en el *speedup* bruto, sino en la **sistematización de las condiciones de viabilidad**. A diferencia de la mayoría de guías que reportan solo el éxito, este trabajo documenta 15 modos de fallo reproducibles y su causa raíz (capítulo 4), cuantifica el impacto de la topología PCIe de consumo (x4 vía chipset frente a x8 en la ranura conectada a la CPU, sección 1.3) y propone un marco formal $V = F_{\text{driver}} \wedge F_{\text{aislamiento}} \wedge F_{\text{carga}} \wedge F_{\text{coste}} \wedge F_{\text{estabilidad}}$ (secciones 1 y 6.9) basado en Amdahl [5], Gustafson [13] y Roofline [52] que permite descartar *a priori* buena parte de los reaprovechamientos que no compensan, y que señala qué debe verificarse necesariamente mediante una prueba de campo. Ese marco, junto con la validación de que la heterogeneidad no introduce contención en cargas *compute-bound* pero sí estrangula las *transfer-bound* (1603 vs. 778 MB/s medidos), constituye la contribución original y transferible más allá del hardware concreto GT200.

Apéndice A

Procedimiento reproducible

Este apéndice resume, sin la traza de errores documentada en los capítulos 4 y 5, cómo reproducir el sistema final. Se presenta como una secuencia de fases de alto nivel; las órdenes exactas y la explicación detallada de cada paso se recogen en las guías técnicas del repositorio del trabajo (sección A.3).

Las versiones y ajustes descritos constituyen la **configuración de referencia** validada a lo largo de este trabajo. No obstante, no todos estos valores son estrictamente vinculantes: mientras que algunos vienen condicionados por las limitaciones del propio hardware, otros respondieron a decisiones de diseño y admiten alternativas equivalentes, La tabla A.1 distingue ambos casos.

Componente	Configuración de referencia	Margen de elección
Driver NVIDIA	340.108	Ninguno: es la versión final de la última rama del driver con soporte para la microarquitectura Tesla (capítulo 4).
CUDA Toolkit	6.5	Ninguno: es la última versión con soporte para *compute capabilities* inferiores a 2.0 (capítulo 5).
Kernel del host	`6.14.11-6-bpo12-pve`	Previsiblemente, cualquier versión cubierta por los parches de `MichelBoucey/nvidia-340xx` (de la 5.7 a la 7.0), aplicando los parches hasta esa versión.
Compilador del host	gcc 12	El del sistema. Si no coincide con el que compiló el kernel, debe indicarse al driver que ignore la comprobación (capítulo 4).
Hipervisor	Proxmox VE 9.2	No impuesto por el hardware; la vía elegida solo requiere un host Linux con contenedores LXC.
Sistema del contenedor	Debian 12	No impuesto por el hardware.
Compilador para CUDA	gcc 4.9 (*chroot* con Debian 8)	Una versión antigua de gcc compatible con CUDA 6.5, que declara soporte oficial hasta gcc 4.8 (capítulo 6).

Cuadro A.1: Componentes de la configuración de referencia y margen de elección de cada uno

A.1. Configuración del firmware

- **Salida de vídeo:** debe proporcionarla una GPU con GOP; en la configuración de referencia, la iGPU del procesador. Las GPUs *legacy* se usan únicamente para cómputo y no necesitan GOP (capítulo 2).
- **CSM:** solo es necesario si el sistema operativo del host arranca en modo BIOS heredado.
- **Topología PCIe:** en la placa empleada, la ranura secundaria solo resultó operativa con la ranura principal en modo de bifurcación x8/x8 (capítulo 6). En otras placas este ajuste puede ser distinto o innecesario.
- **Generación del enlace PCIe:** en la configuración de referencia se fijaron a Gen 1 (2.5 GT/s) todos los ajustes de *Link Mode* disponibles en la BIOS, para garantizar

que las tarjetas cumplan los tiempos de negociación del enlace. Previsiblemente bastaría con fijar las ranuras ocupadas por las GPUs. Gen 2, la generación nativa de las tarjetas, no se ha probado.

- ***Above 4G Decoding*:** indiferente. Las ventanas de memoria de las tarjetas son pequeñas y se asignan sin problemas por debajo de los 4 GB, y el sistema arranca correctamente también con la opción habilitada.

Los ajustes empleados únicamente durante los experimentos VFIO se documentan en el capítulo 3 y no forman parte de esta línea base.

A.2. Fases del procedimiento

1. **Preparación del host.** Revertir cualquier reserva previa de las GPUs para VFIO en caso de ser necesario, desactivar el driver libre *nouveau* e instalar un kernel compatible con los parches del driver, junto con sus *headers*, fijándolo como kernel de arranque.

2. **Compilación e instalación del driver.** Aplicar al código del driver oficial 340.108 los parches comunitarios correspondientes al kernel instalado, corregir las incompatibilidades con el compilador del sistema y compilar el módulo mediante DKMS, que lo recompila automáticamente ante futuras actualizaciones del kernel. Después, instalar los componentes de espacio de usuario del driver sin recompilar el módulo y comprobar que ambas GPUs son detectadas.

3. **Contenedor y acceso a las GPUs.** Crear un contenedor LXC y darle acceso a los ficheros de dispositivo de las GPUs. Dentro del contenedor, instalar los componentes de espacio de usuario del driver, respetando la misma versión que en el host.

4. **CUDA 6.5.** Instalar el *toolkit* extrayendo sus instaladores por capas y validar el cómputo con `deviceQuery`. Para compilar programas nuevos, preparar un entorno de compilación con una versión antigua de gcc; los programas resultantes se ejecutan en el contenedor principal.

5. **Arranque dual (opcional).** Configurar una entrada de arranque para el modo VFIO que se utilice una sola vez y devuelva automáticamente el sistema al modo LXC en el siguiente reinicio.

Una vez instalado, el sistema permite usar ambas GPUs a la vez en ejecuciones cortas, escalonando ligeramente el primer lanzamiento simultáneo tras cada arranque para evitar la condición de carrera de la sección 6.5. No se recomienda, en cambio, para cargas sostenidas (sección 6.6).

A.3. Material complementario

El repositorio del trabajo reúne esta memoria y tres guías técnicas con las órdenes completas de cada fase, los problemas encontrados en cada una y su solución:

- **Instalación del driver en el host** (fases 1 y 2).
- **Contenedor LXC y CUDA** (fases 3 y 4), que incluye además las pruebas de rendimiento y estabilidad y las precauciones de uso.
- **Arranque dual LXC/VFIO** (fase 5), con los comandos para comprobar y cambiar el modo de arranque.

Bibliografía

[1] AMD: AMD Ryzen 9 7950X Specifications. URL https://www.amd.com/en/products/processors/desktops/ryzen/7000-series/amd-ryzen-9-7950x.html

[2] AMD: AMD Socket AM5 Chipset: carriles PCIe y conectividad. URL https://www.amd.com/en/products/processors/chipsets/am5.html

[3] AMD: Function Level Reset (PG344). URL https://docs.amd.com/r/en-US/pg344-pcie-dma-versal/Function-Level-Reset

[4] AMD, UEFI Forum: Graphics Output Protocol (GOP) Driver for UEFI. URL https://uefi.org/sites/default/files/resources/UPFS11_P4_UEFI_GOP_AMD.pdf

[5] Amdahl, G.M.: Validity of the single processor approach to achieving large scale computing capabilities. In: AFIPS Conference Proceedings, pp. 483–485 (1967). URL https://doi.org/10.1145/1465482.1465560

[6] Arch Linux: Nouveau — Arch Linux Wiki. URL https://wiki.archlinux.org/title/Nouveau

[7] archlinux-jerry: Repositorio en GitHub de archlinux-jerry/nvidia-340xx. URL https://github.com/archlinux-jerry/nvidia-340xx

[8] ASUS: TUF GAMING X670E-PLUS WIFI: especificaciones. URL https://www.asus.com/motherboards-components/motherboards/tuf-gaming/tuf-gaming-x670e-plus-wifi/techspec/

[9] Boucey, M.: Repositorio en GitHub de MichelBoucey/nvidia-340xx. URL https://github.com/MichelBoucey/nvidia-340xx

[10] Debian: Archivo histórico de Debian (archive.debian.org). URL https://archive.debian.org/

[11] DKMS Project: DKMS: Dynamic Kernel Module Support. URL https://github.com/dkms-project/dkms

[12] dkosmari: Repositorio en GitHub de dkosmari/nvidia-340.108-updated. URL https://github.com/dkosmari/nvidia-340.108-updated

[13] Gustafson, J.L.: Reevaluating Amdahl's law. Communications of the ACM **31**(5), 532–533 (1988). URL https://doi.org/10.1145/42411.42415

[14] Harris, M.: Unified Memory for CUDA Beginners (2017). URL https://developer.nvidia.com/blog/unified-memory-cuda-beginners/

[15] IBM: ¿Qué es la computación de alto rendimiento (HPC)? URL https://www.ibm.com/think/topics/hpc

[16] Kanter, D.: NVIDIA's GT200: Inside a Parallel Processor. Real World Tech (2008). URL https://www.realworldtech.com/gt200/

[17] Larabel, M.: NVIDIA Now Allows GeForce GPU Pass-Through For Windows VMs On Linux. Phoronix (2021). URL https://www.phoronix.com/news/NVIDIA-GeForce-Virt-On-Linux

[18] Larabel, M.: NVIDIA Confirms 580 Linux Driver Is The Last For Maxwell / Pascal / Volta. Phoronix (2025). URL https://www.phoronix.com/news/NVIDIA-580-Linux-Driver-Last-HW

[19] Linux Containers: LXC: Security. URL https://linuxcontainers.org/lxc/security/

[20] Linux Containers: lxc.container.conf: LXC container configuration file (manpage) (2021). URL https://linuxcontainers.org/lxc/manpages/man5/lxc.container.conf.5.html

[21] Linux Kernel Documentation: PCI Power Management (Documentación del kernel Linux). URL https://docs.kernel.org/power/pci.html

[22] Linux Kernel Documentation: Softlockup detector and hardlockup detector (nmi_watchdog) (Documentación del kernel Linux). URL https://docs.kernel.org/admin-guide/lockup-watchdogs.html

[23] Linux Kernel Documentation: VFIO: "Virtual Function I/O" (Documentación del kernel Linux). URL https://docs.kernel.org/driver-api/vfio.html

[24] NVIDIA: CUDA C++ Programming Guide. URL https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[25] NVIDIA: Difference between the Driver API and Runtime API. URL https://docs.nvidia.com/cuda/cuda-driver-api/driver-vs-runtime-api.html

[26] NVIDIA: GeForce RTX 4060 Family: especificaciones. URL https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4060-4060ti/

[27] NVIDIA: Legacy CUDA GPU Compute Capability. URL https://developer.nvidia.com/cuda/gpus/legacy

[28] NVIDIA: NVIDIA Accelerated Linux Graphics Driver README and Installation Guide (versión 340.108). URL https://download.nvidia.com/XFree86/Linux-x86_64/340.108/README/index.html

[29] NVIDIA: NVIDIA Ada GPU Architecture. URL https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf. Whitepaper, versión 2.02

[30] NVIDIA: NVIDIA CUDA Runtime API. URL https://docs.nvidia.com/cuda/cuda-runtime-api/index.html

[31] NVIDIA: NVIDIA System Management Interface (nvidia-smi). URL https://developer.nvidia.com/system-management-interface

[32] NVIDIA: Support timeframes for Unix legacy GPU releases. URL https://nvidia.custhelp.com/app/answers/detail/a_id/3142

[33] NVIDIA: Unix Drivers (archivo de drivers para Linux). URL https://www.nvidia.com/en-us/drivers/unix/

[34] NVIDIA: NVIDIA's Next Generation CUDA Compute Architecture: Fermi (2009). URL https://www.nvidia.com/content/PDF/fermi_white_papers/NVIDIA_Fermi_Compute_Architecture_Whitepaper.pdf. Whitepaper, versión 1.1

[35] NVIDIA: CUDA 6.5 Performance Report (2014). URL https://developer.download.nvidia.com/compute/cuda/6_5/rel/docs/CUDA_6.5_Performance_Report.pdf

[36] NVIDIA: CUDA Toolkit 6.5 (instalador para Linux x86_64) (2014). URL https://developer.download.nvidia.com/compute/cuda/6_5/rel/installers/cuda_6.5.14_linux_64.run

[37] NVIDIA: NVIDIA-Linux-x86_64-340.108.run (instalador del driver) (2016). URL https://us.download.nvidia.com/XFree86/Linux-x86_64/340.108/NVIDIA-Linux-x86_64-340.108.run

[38] NVIDIA Developer Forums: Power Consumption Unavailable (2013). URL https://forums.developer.nvidia.com/t/power-consumption-unavailable/31588

[39] NVIDIA Developer Forums: Possible to use multiple drivers for multi GPU? (2023). URL https://forums.developer.nvidia.com/t/possible-to-use-multiple-drivers-for-multi-gpu/269500

[40] Proxmox: PCI(e) Passthrough — Proxmox VE Wiki. URL https://pve.proxmox.com/wiki/PCI(e)_Passthrough

[41] Proxmox: Proxmox VE Wiki. URL https://pve.proxmox.com/wiki/Main_Page

[42] Proxmox Support Forum: RTX 5070 Ti GPU passthrough fails in Proxmox: stuck in D3/FLR reset loop. URL https://forum.proxmox.com/threads/rtx-5070-ti-gpu-passthrough-fails-in-proxmox-%E2%80%93-stuck-in-d3-flr-reset-loop-%E2%86%92-device-becomes-unresponsive-ffff.183656/

[43] PyTorch Developers: Notice: CUDA 12.6 wheels will no longer be published from PyTorch 2.15 (drops Maxwell / Pascal / Volta) (2026). URL https://dev-discuss.pytorch.org/t/notice-cuda-12-6-wheels-will-no-longer-be-published-from-pytorch-2-15-drops-maxwell-pascal-volta/3432

[44] Red Hat: Active-State Power Management. URL https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/power_management_guide/aspm

[45] Sieger, H.: IOMMU Groups: What You Need to Consider (2017). URL https://www.heiko-sieger.info/iommu-groups-what-you-need-to-consider/

[46] The Linux Kernel: drivers/pci/pci.c (código fuente del kernel Linux 6.14). URL https://github.com/torvalds/linux/blob/v6.14/drivers/pci/pci.c

[47] The Linux Kernel Documentation: ABI: sysfs-bus-pci (reset_method y ranuras PCI). URL https://www.kernel.org/doc/Documentation/ABI/testing/sysfs-bus-pci

[48] VideoCardz.net: NVIDIA GeForce GTS 250 Specs. URL https://videocardz.net/nvidia-geforce-gts-250

[49] VideoCardz.net: NVIDIA Tesla C1060 Data Center Accelerator Specs. URL https://videocardz.net/nvidia-tesla-c1060

[50] VideoCardz.net: NVIDIA GT200 Graphics Processing Unit (GPU) (2014). URL https://videocardz.net/gpu/nvidia-gt200

[51] Wikipedia: PCI Express. URL https://en.wikipedia.org/wiki/PCI_Express

[52] Williams, S., Waterman, A., Patterson, D.: Roofline: an insightful visual performance model for multicore architectures. Communications of the ACM **52**(4), 65–76 (2009). URL https://doi.org/10.1145/1498765.1498785